Introduction to Deep Learning

Caltech EE/CS/CNS 148a - Winter 2026
P. Perona

Lecture 3

Stochastic Gradient Descent, Batching, Momentum, GPUs

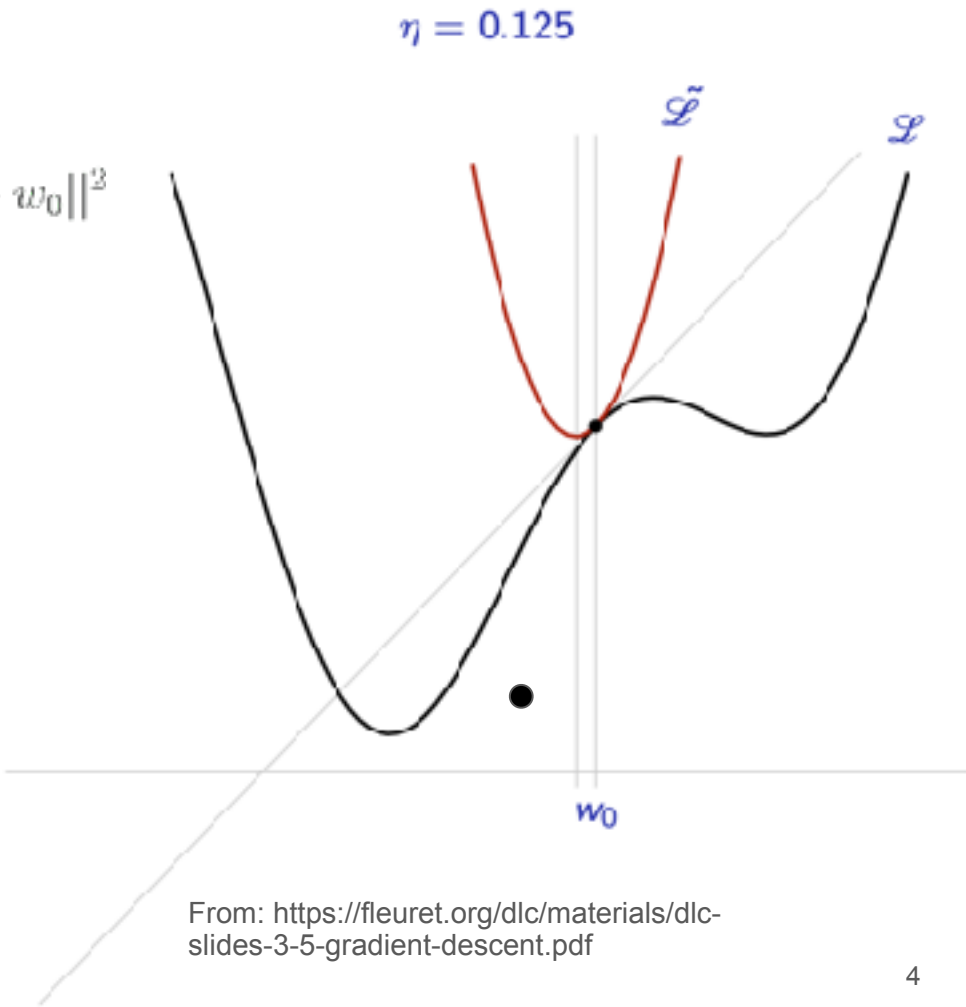
Selection criterion:
Minimizing the loss function

Gradient descent

$$\tilde{\mathcal{L}}(w) = \mathcal{L}(w_0) + \nabla\mathcal{L}(w_0)^T(w - w_0) + \frac{1}{2\eta}||w - w_0||^2$$

Gradient descent

$$\tilde{\mathcal{L}}(w) = \mathcal{L}(w_0) + \nabla\mathcal{L}(w_0)^T(w - w_0) + \frac{1}{2\eta}||w - w_0||^2$$

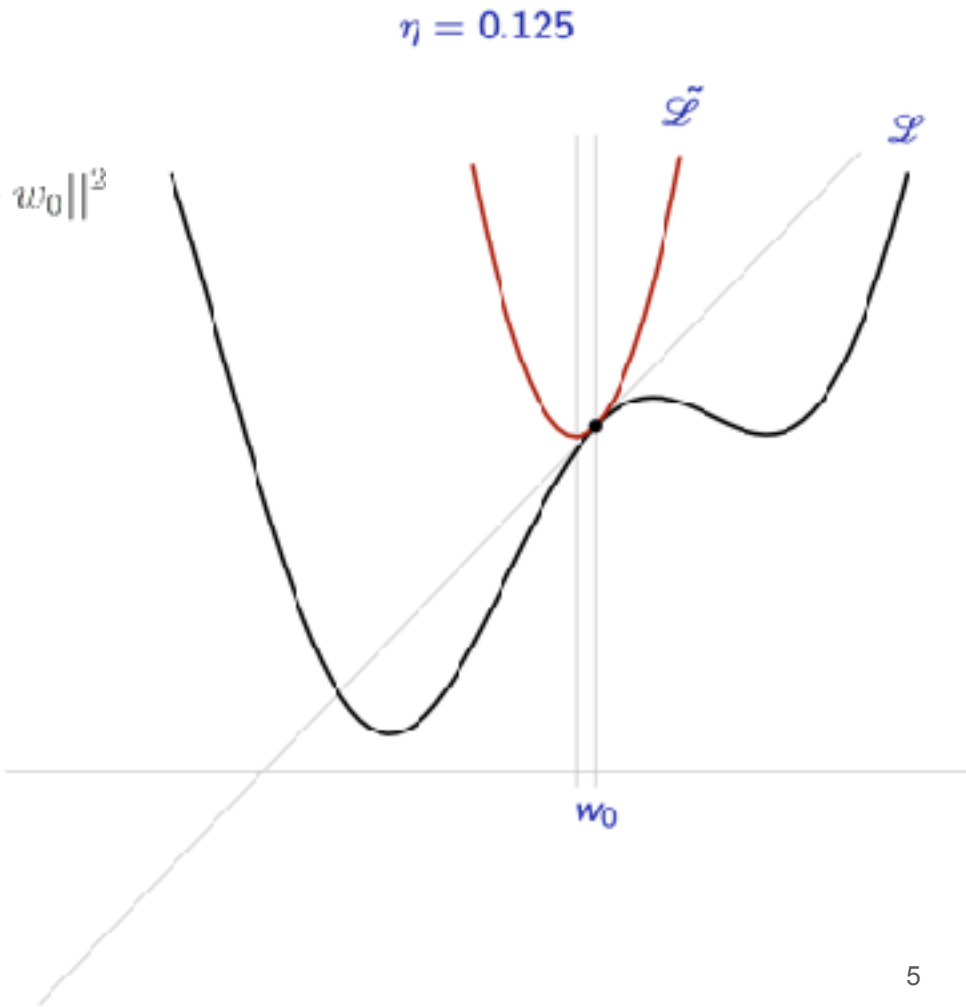


From: https://fleuret.org/dlc/materials/dlc-slides-3-5-gradient-descent.pdf

Gradient descent

$$\tilde{\mathcal{L}}(w) = \mathcal{L}(w_0) + \nabla\mathcal{L}(w_0)^T(w - w_0) + \frac{1}{2\eta}||w - w_0||^2$$

$$\nabla\tilde{\mathcal{L}}(w) = \nabla\mathcal{L}(w_0) + \frac{1}{\eta}(w - w_0)$$

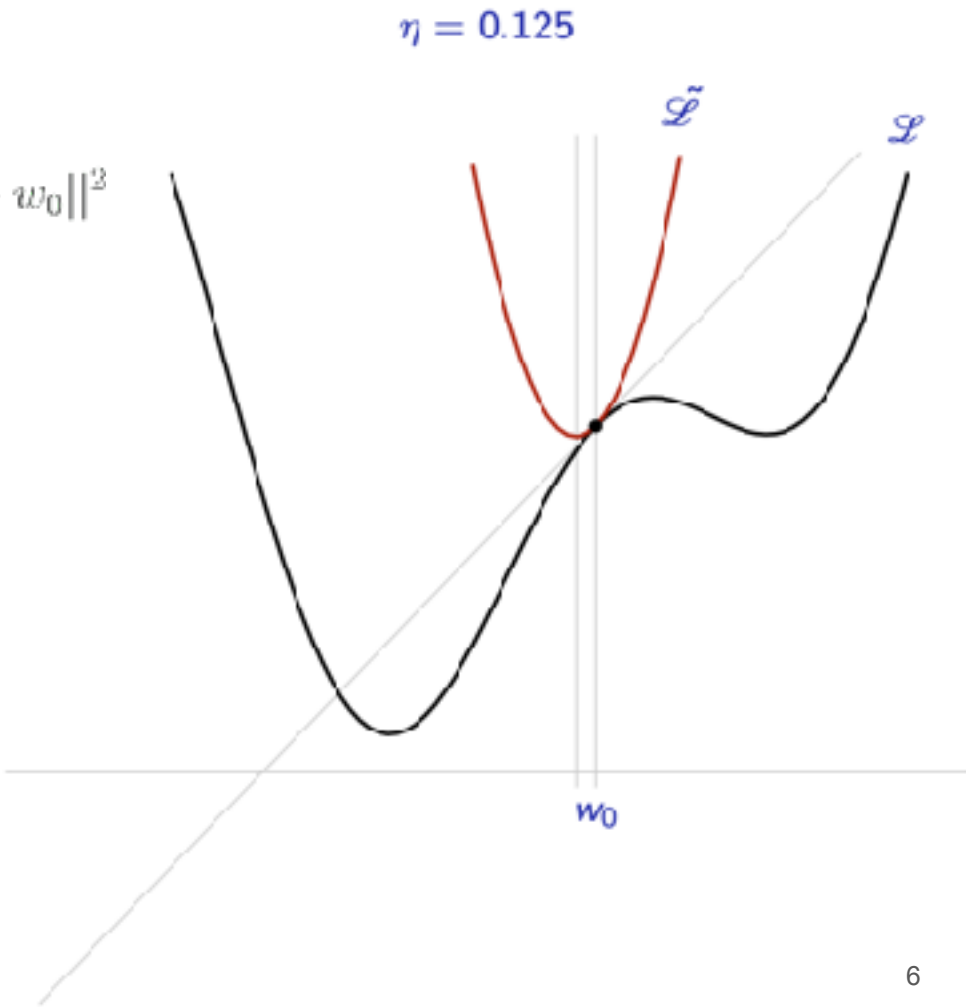


Gradient descent

$$\tilde{\mathcal{L}}(w) = \mathcal{L}(w_0) + \nabla\mathcal{L}(w_0)^T(w - w_0) + \frac{1}{2\eta}||w - w_0||^2$$

$$\nabla\tilde{\mathcal{L}}(w) = \nabla\mathcal{L}(w_0) + \frac{1}{\eta}(w - w_0)$$

$$argmin_w\tilde{\mathcal{L}}(w) = w_0 - \eta\nabla\mathcal{L}(w_0)$$

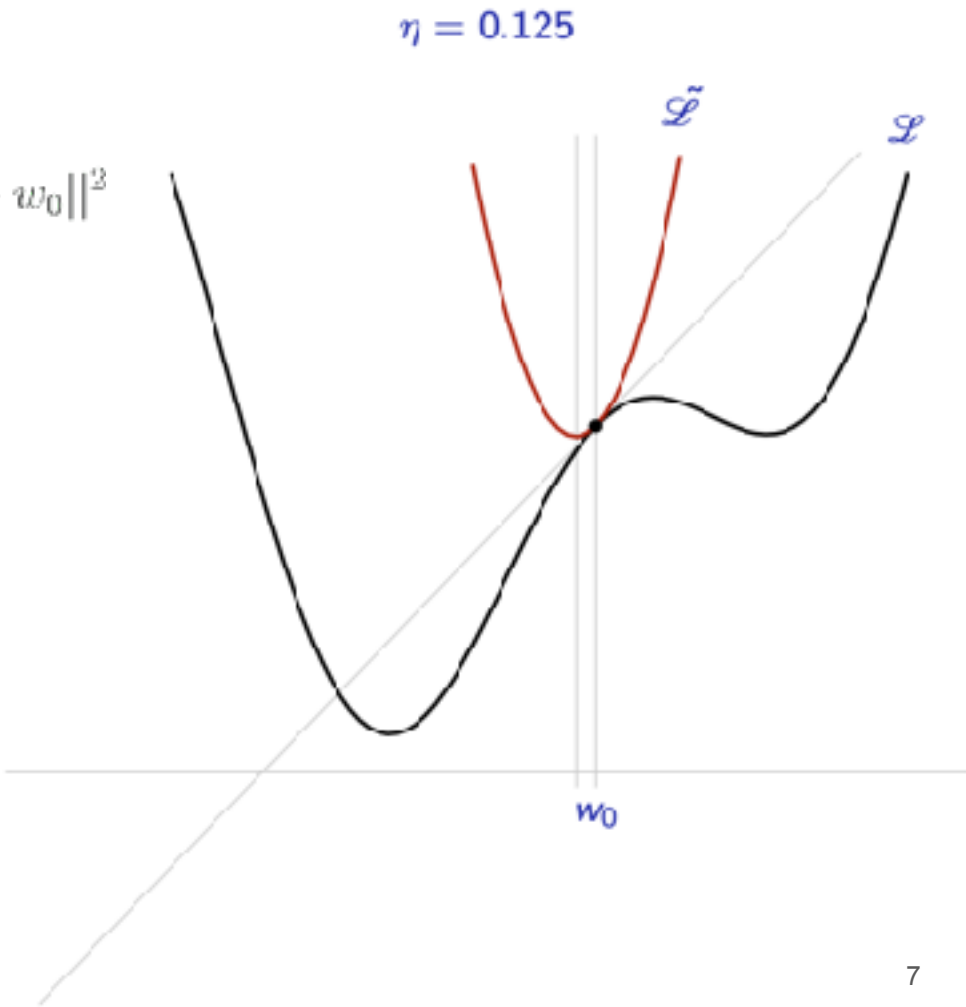


Gradient descent

$$\tilde{\mathcal{L}}(w) = \mathcal{L}(w_0) + \nabla\mathcal{L}(w_0)^T(w - w_0) + \frac{1}{2\eta}||w - w_0||^2$$

$$\nabla\tilde{\mathcal{L}}(w) = \nabla\mathcal{L}(w_0) + \frac{1}{\eta}(w - w_0)$$

$$argmin_w\tilde{\mathcal{L}}(w) = w_0 - \eta\nabla\mathcal{L}(w_0)$$

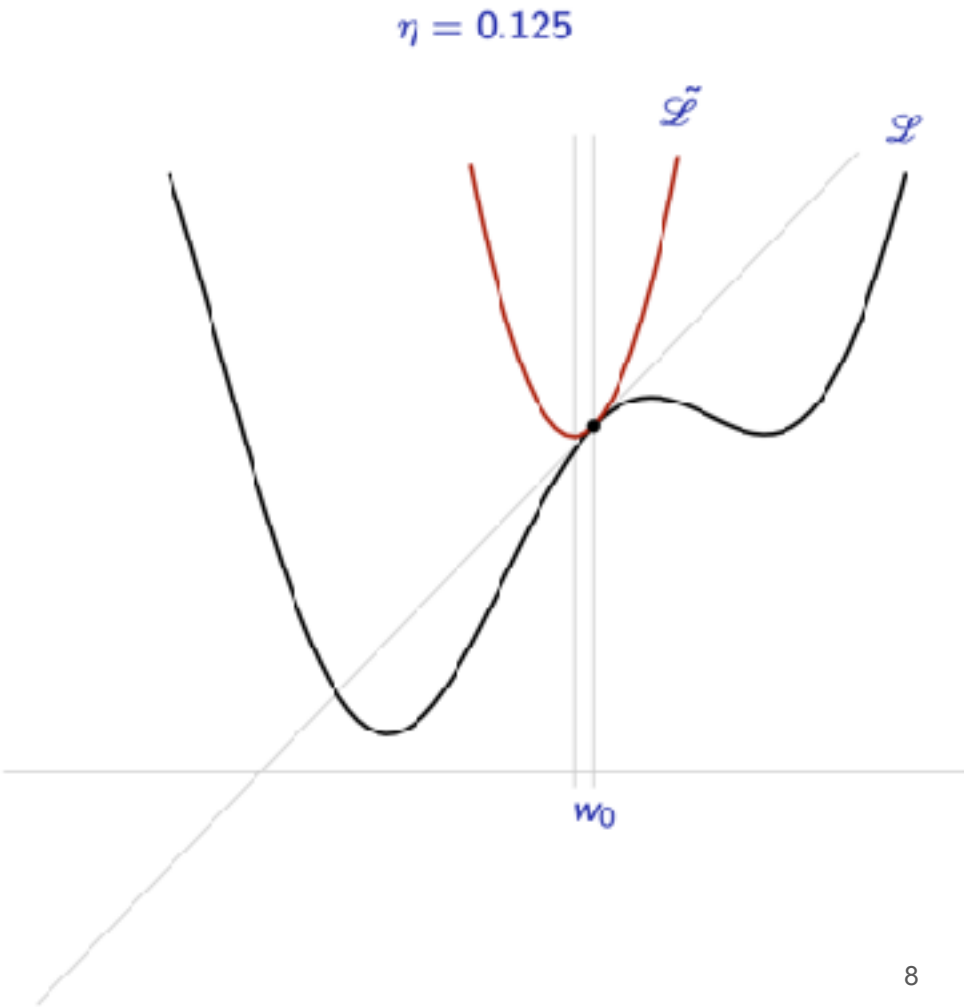


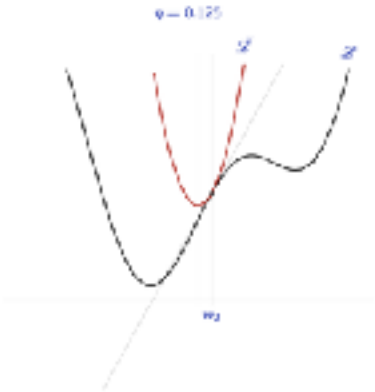
Gradient descent

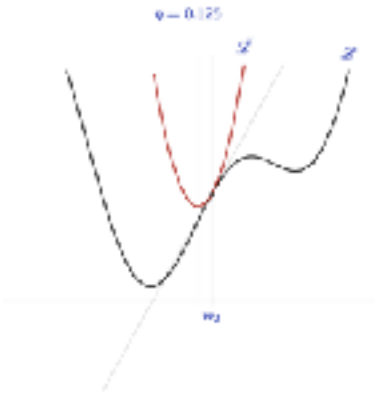
$$\mathcal{L}(w) = \frac{1}{K} \sum_{(\bar{x},\bar{y}) \in \mathcal{D}} \mathcal{L}(w, \bar{x}, \bar{y})$$

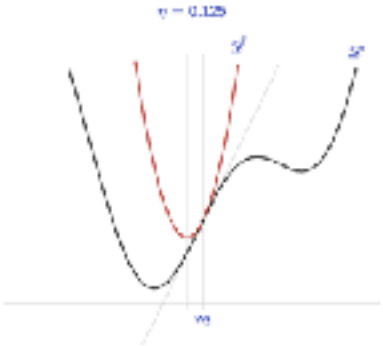
Update rule:

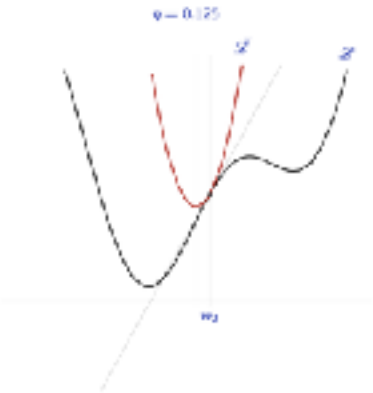
$$w_{t+1} = w_t - \eta \nabla \mathcal{L}(w_t)$$

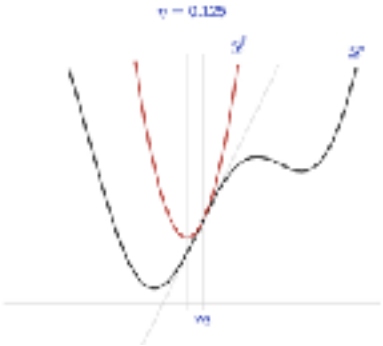


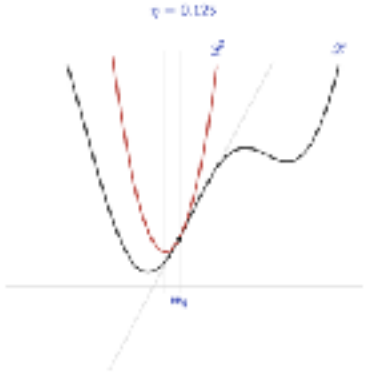

$\varphi = 0.125$
$\mathcal{L}^t$
$\mathcal{L}^s$
w_2

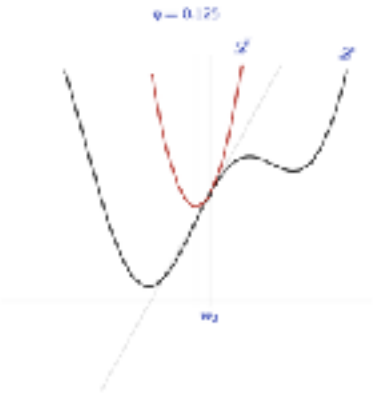


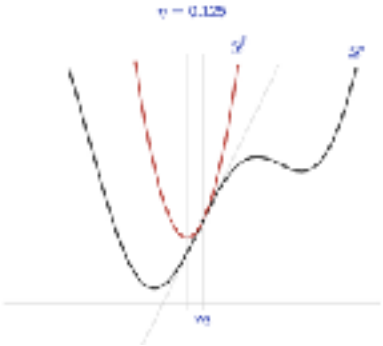


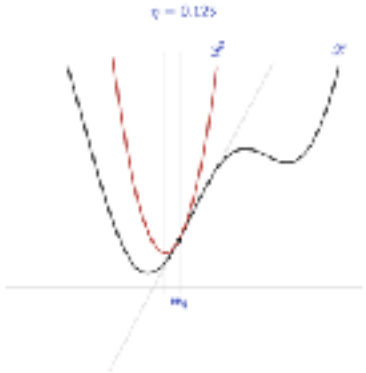


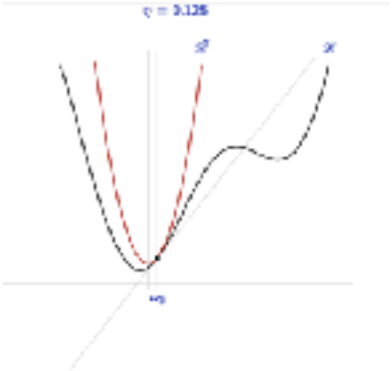


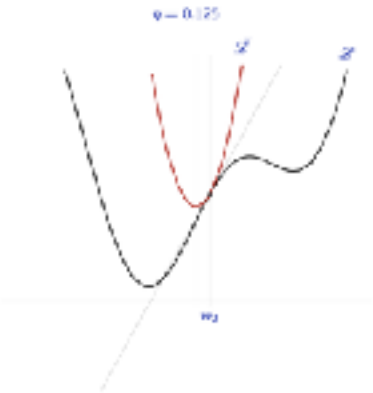


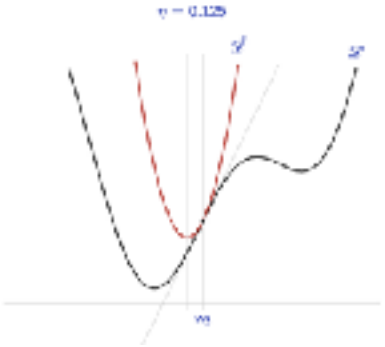


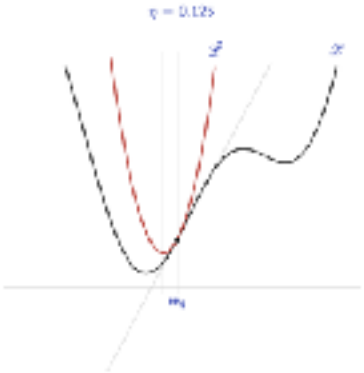


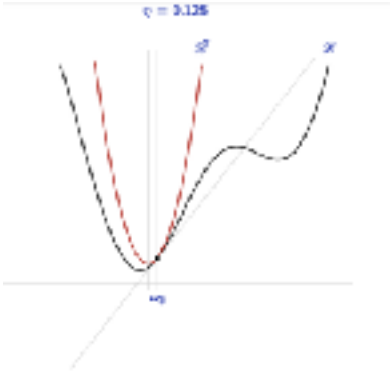


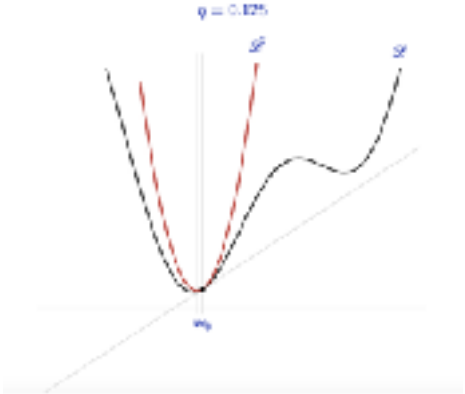


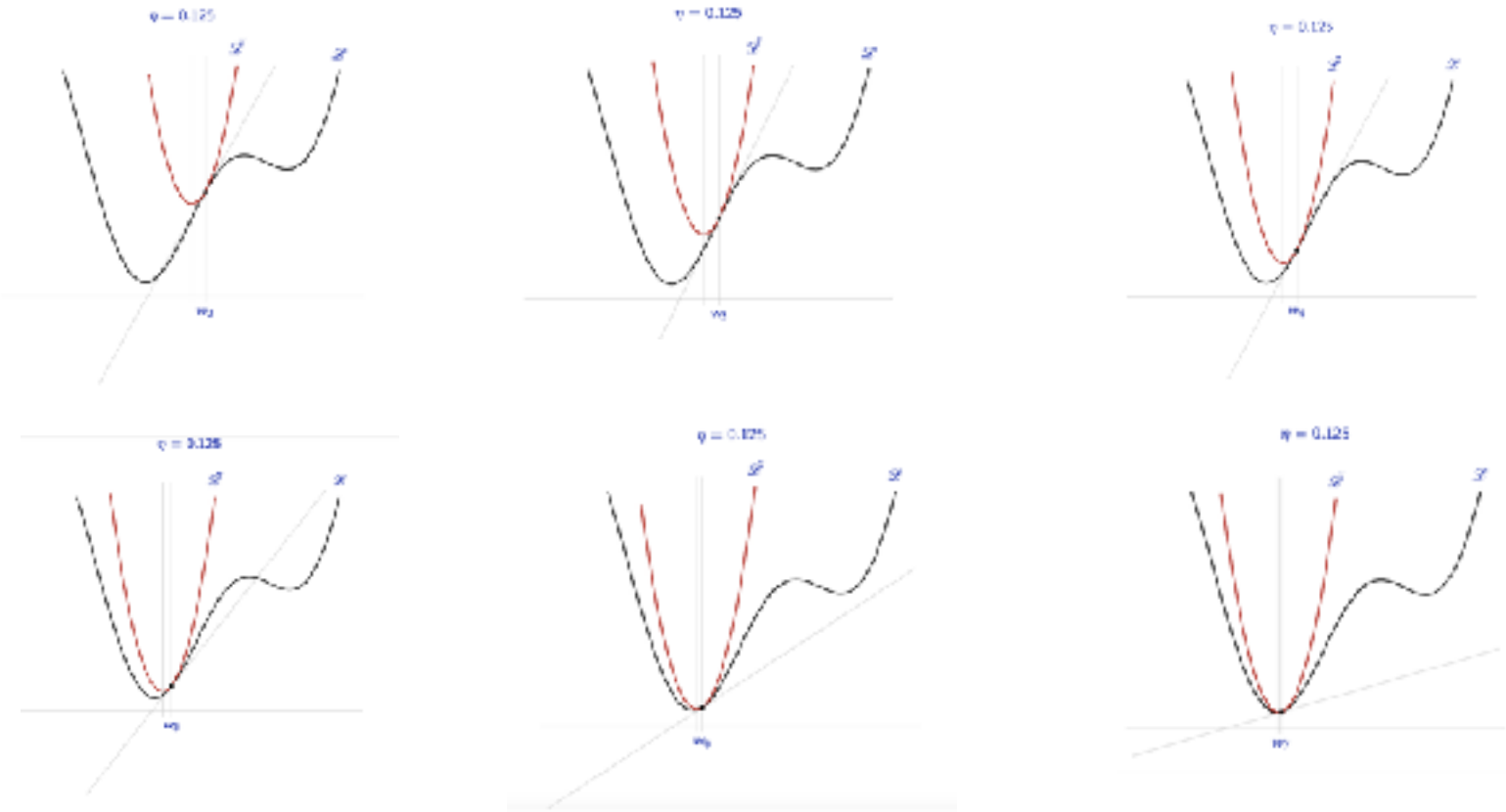




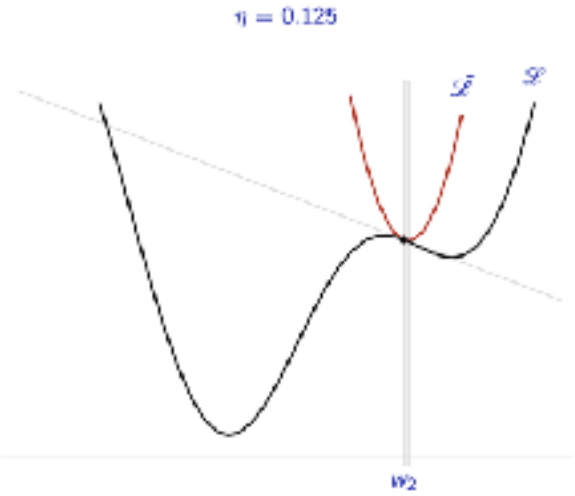


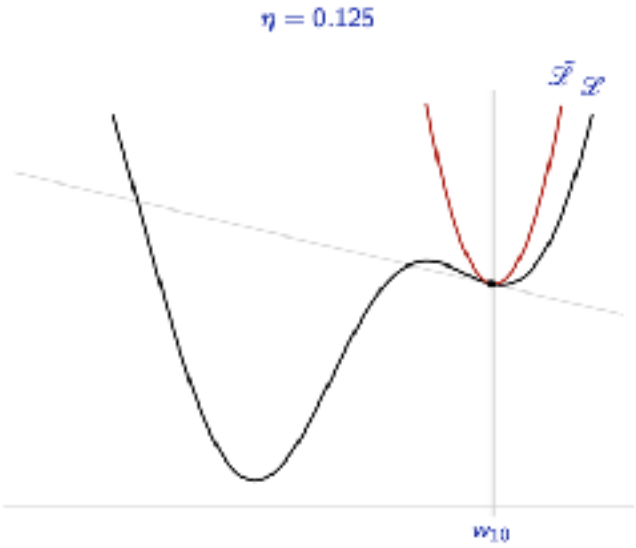




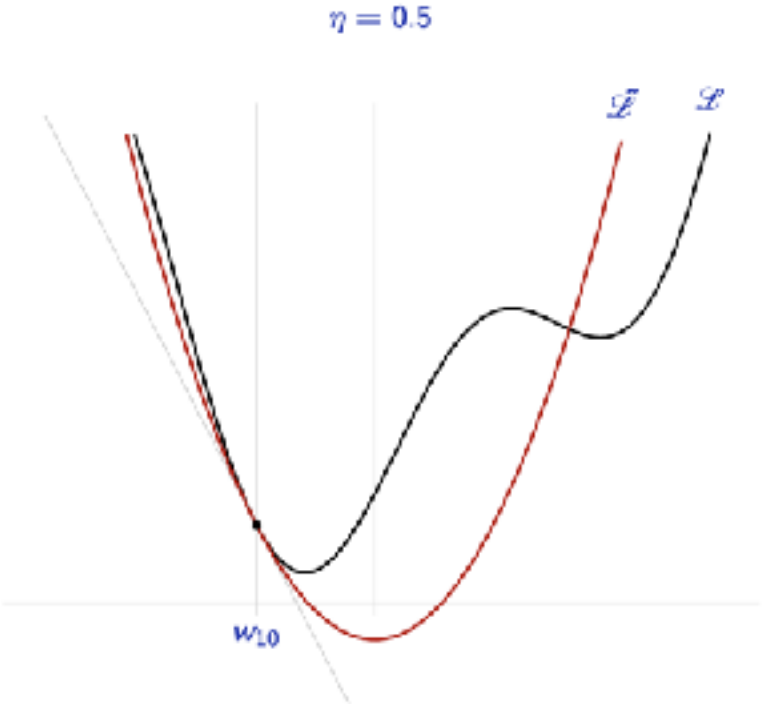


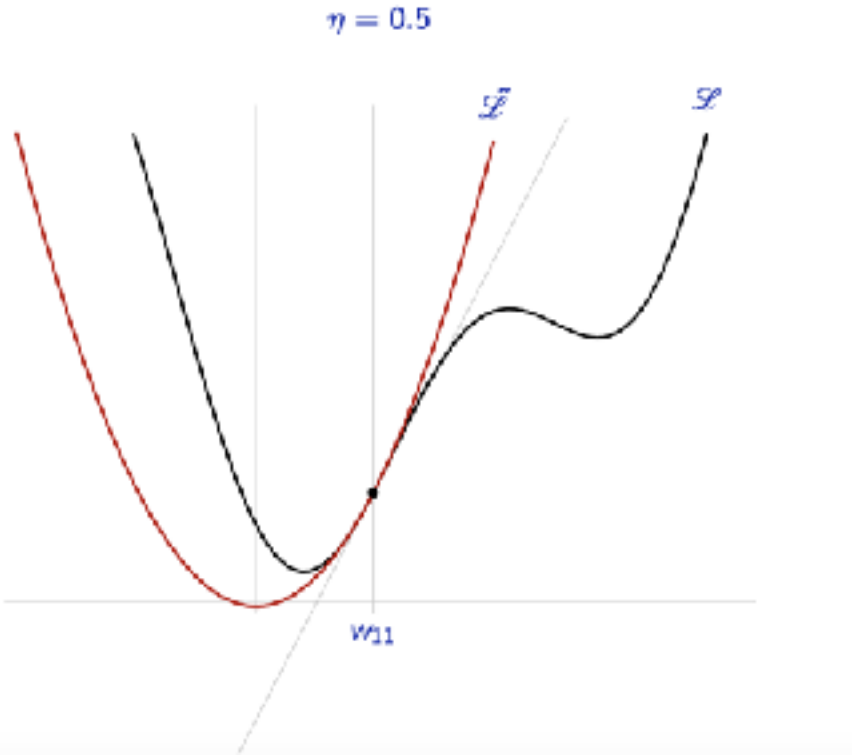
Local minima

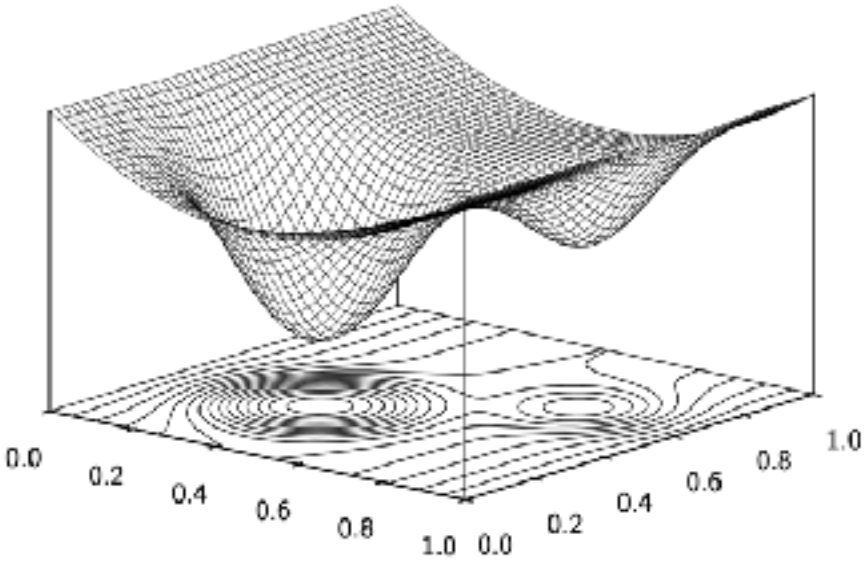


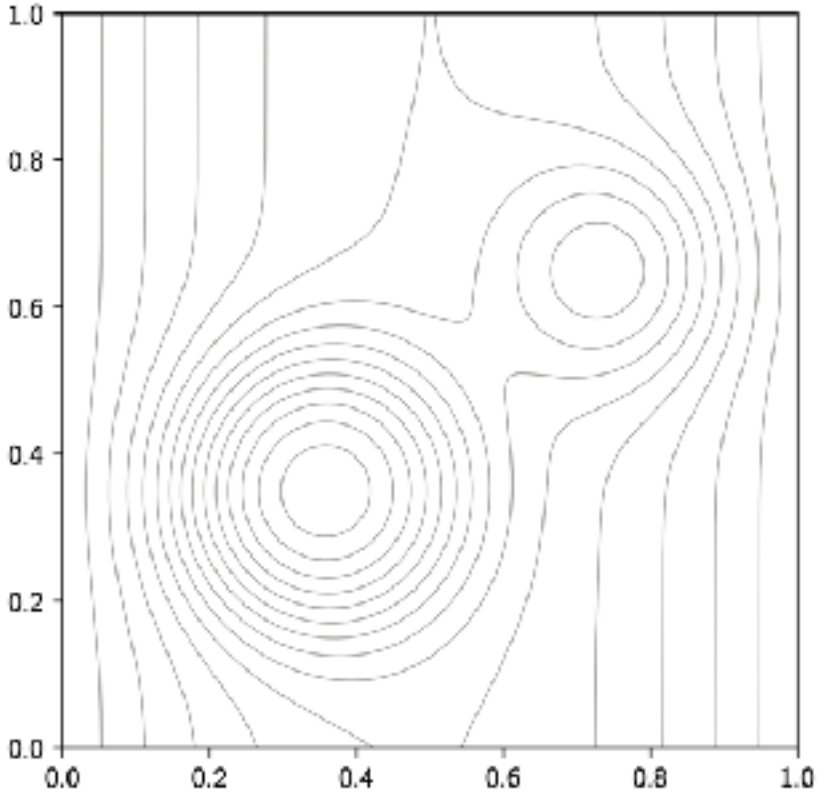


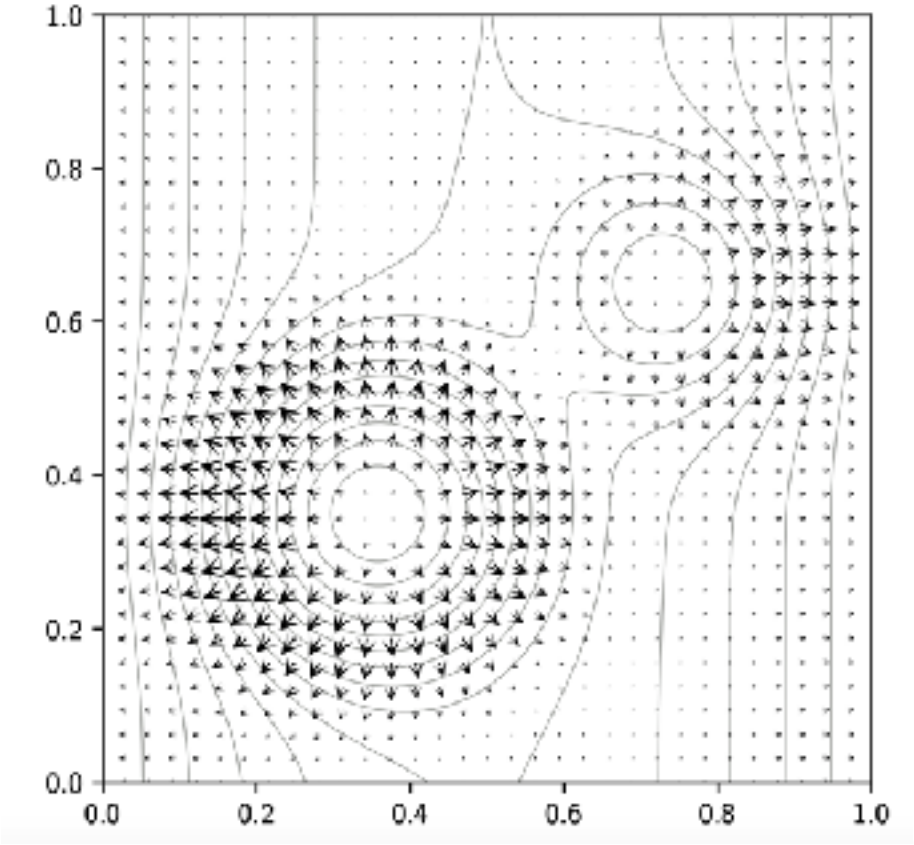
Big steps

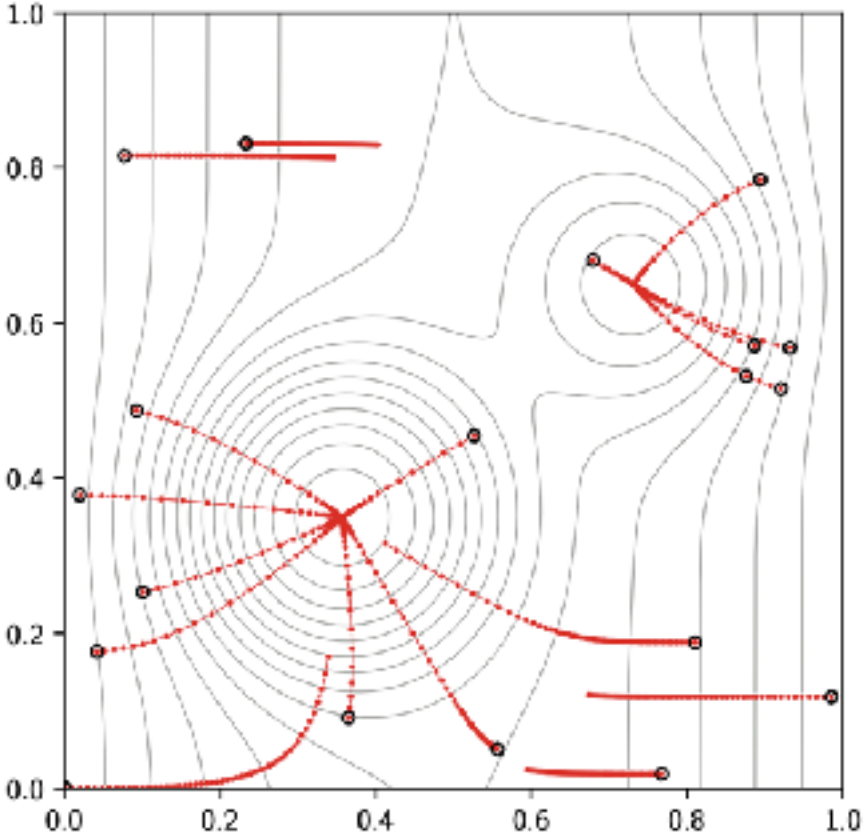

$\eta = 0.5$
$\tilde{\mathscr{L}}$
$\mathscr{L}$
w_{10}


$\eta = 0.5$
$\tilde{\mathscr{L}}$
$\mathscr{L}$
w_{11}


0.0
0.2
0.4
0.6
0.8
1.0
0.0
0.2
0.4
0.6
0.8
1.0


1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0


1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0


1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0

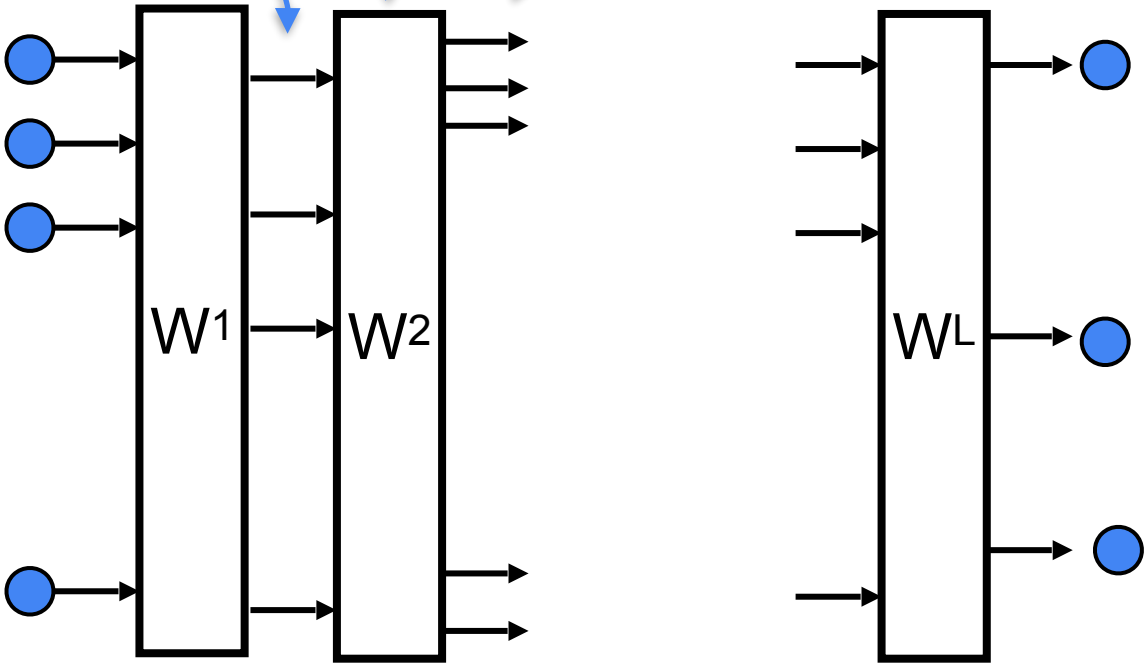
Backpropagation

Learning by error *backpropagation*

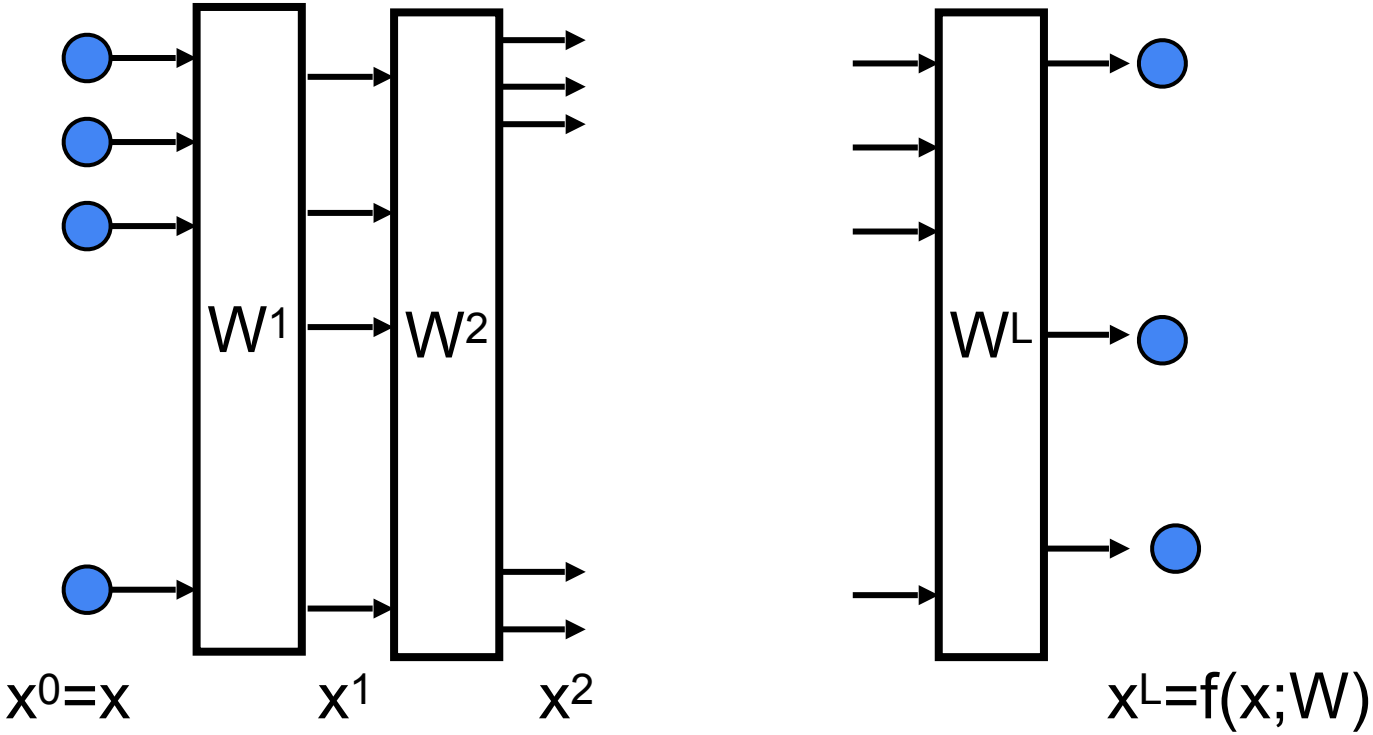
Full lecture by Yaser Abu-Mostafa here:

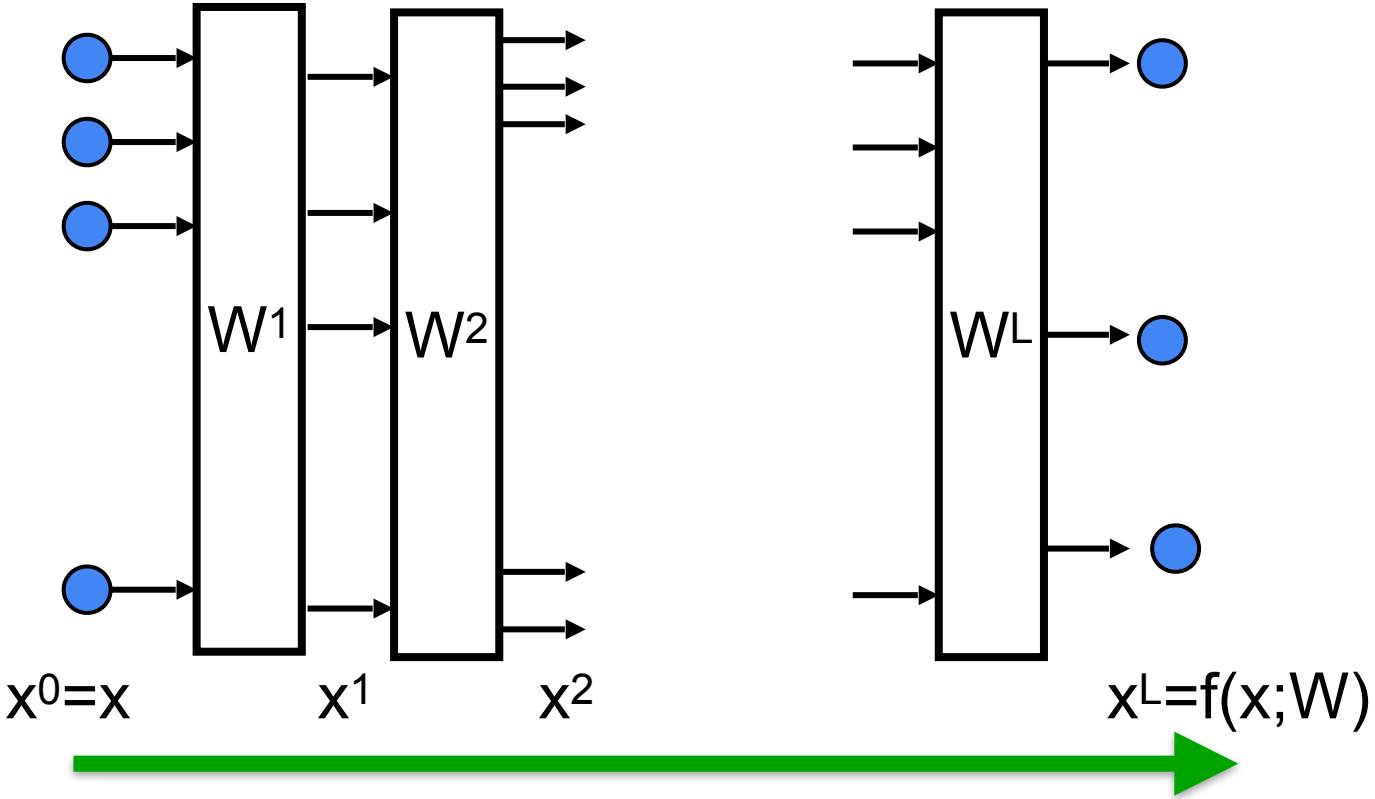
https://home.work.caltech.edu/library/106.html

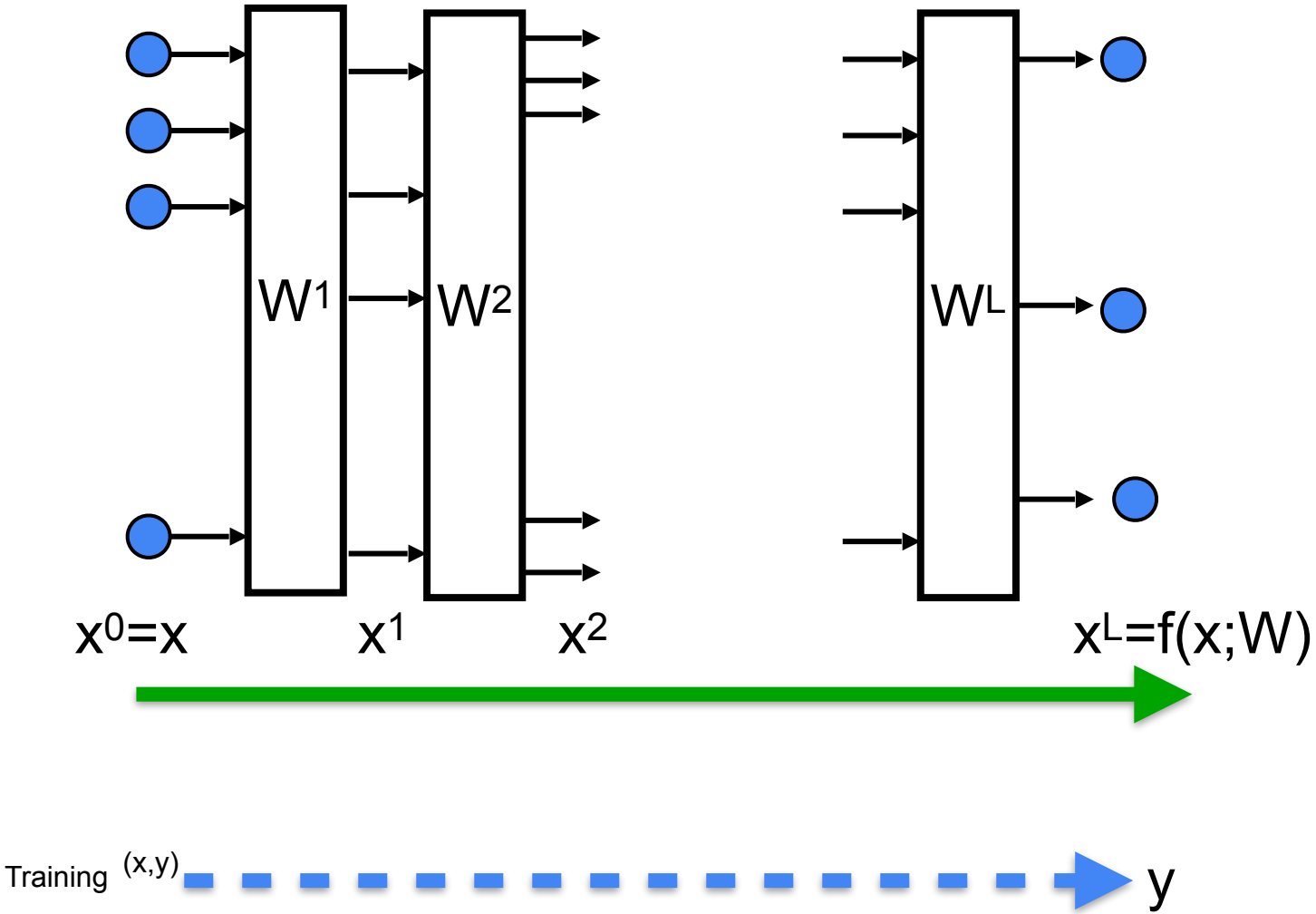
$$x^{(l-1)} \rightarrow W^{(l)} x^{(l-1)} = s^{(l)} \rightarrow h(s^{(l)}) = x^{(l)}$$

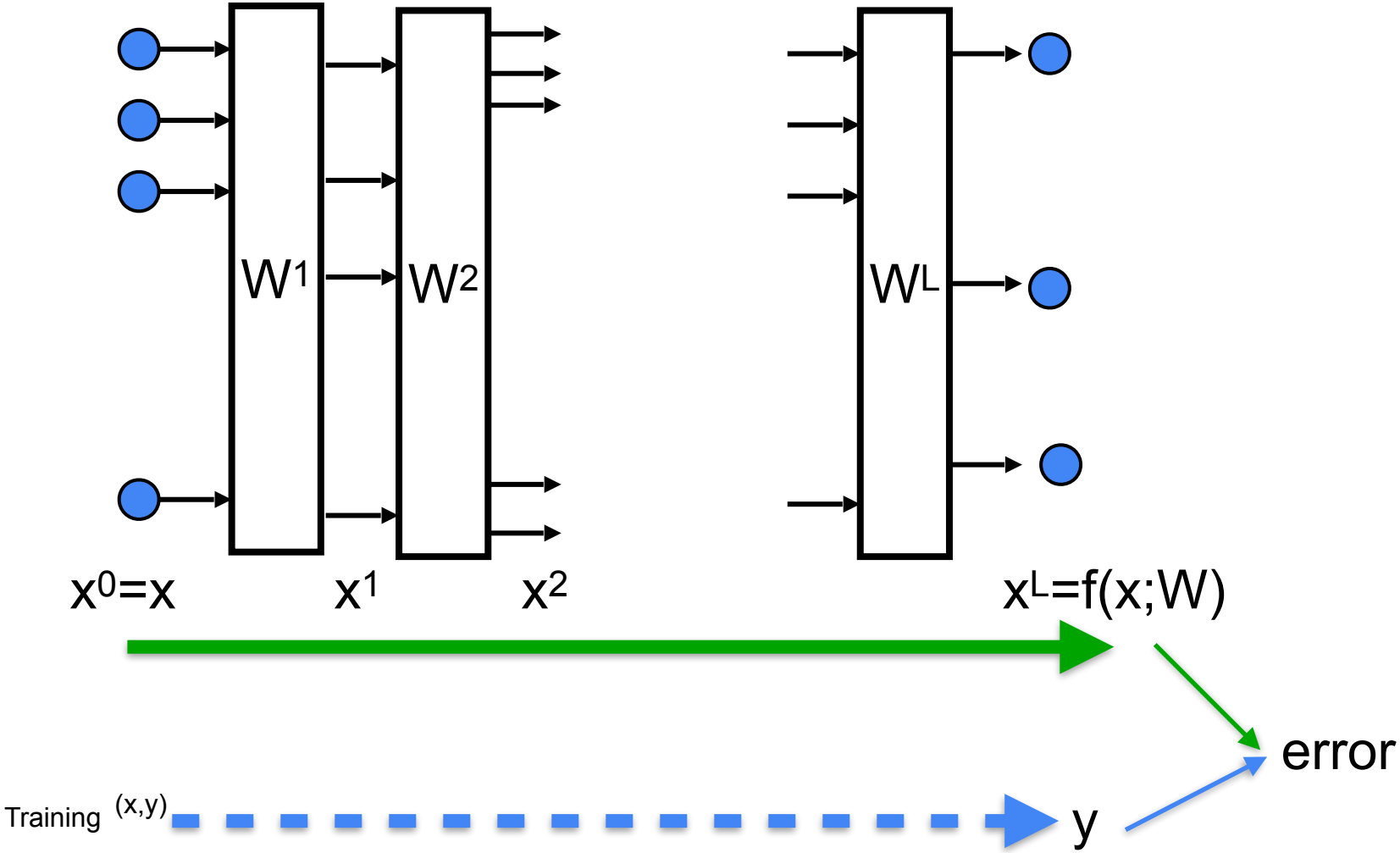


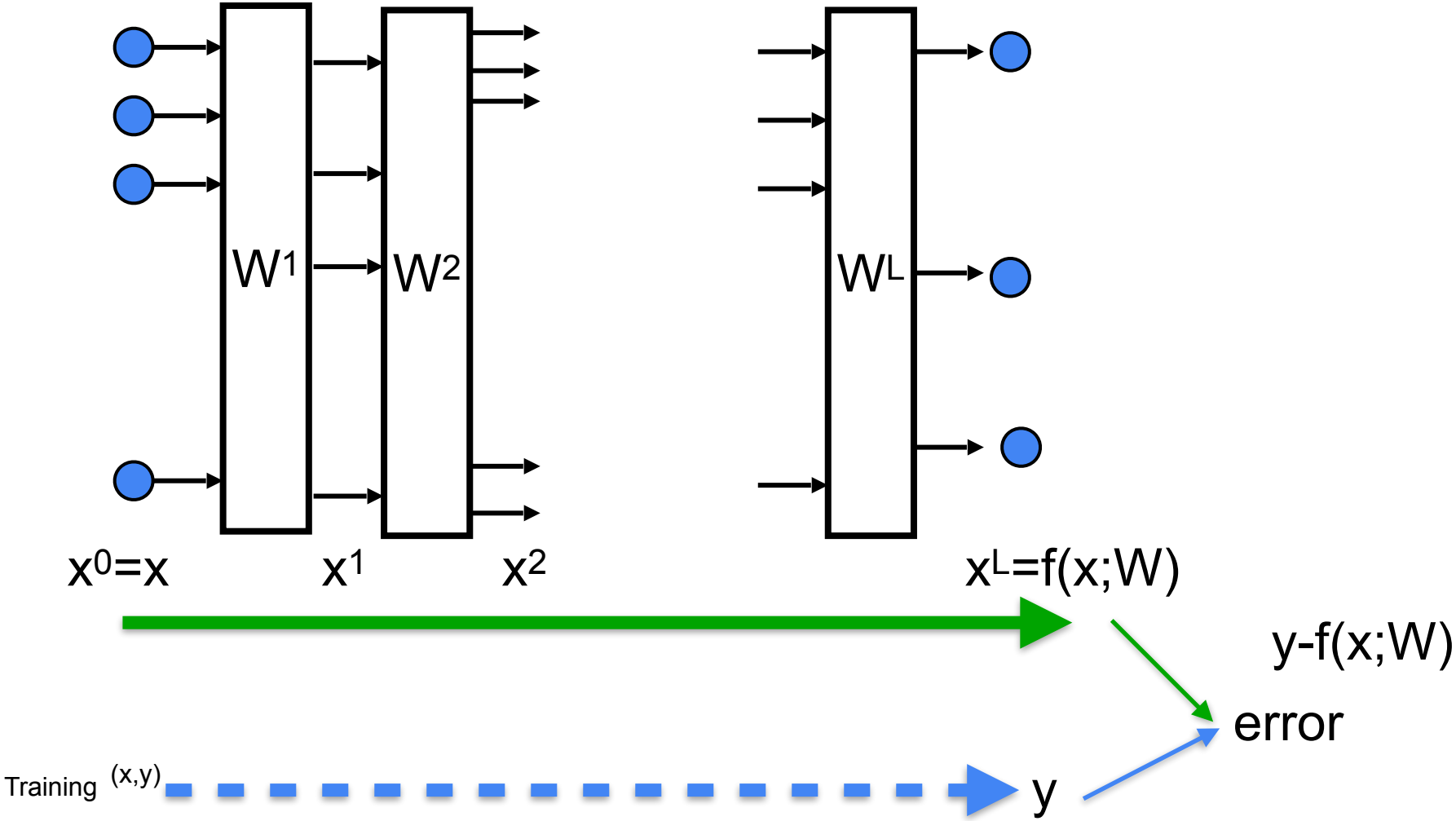
x^0 x^1 x^2 x^L

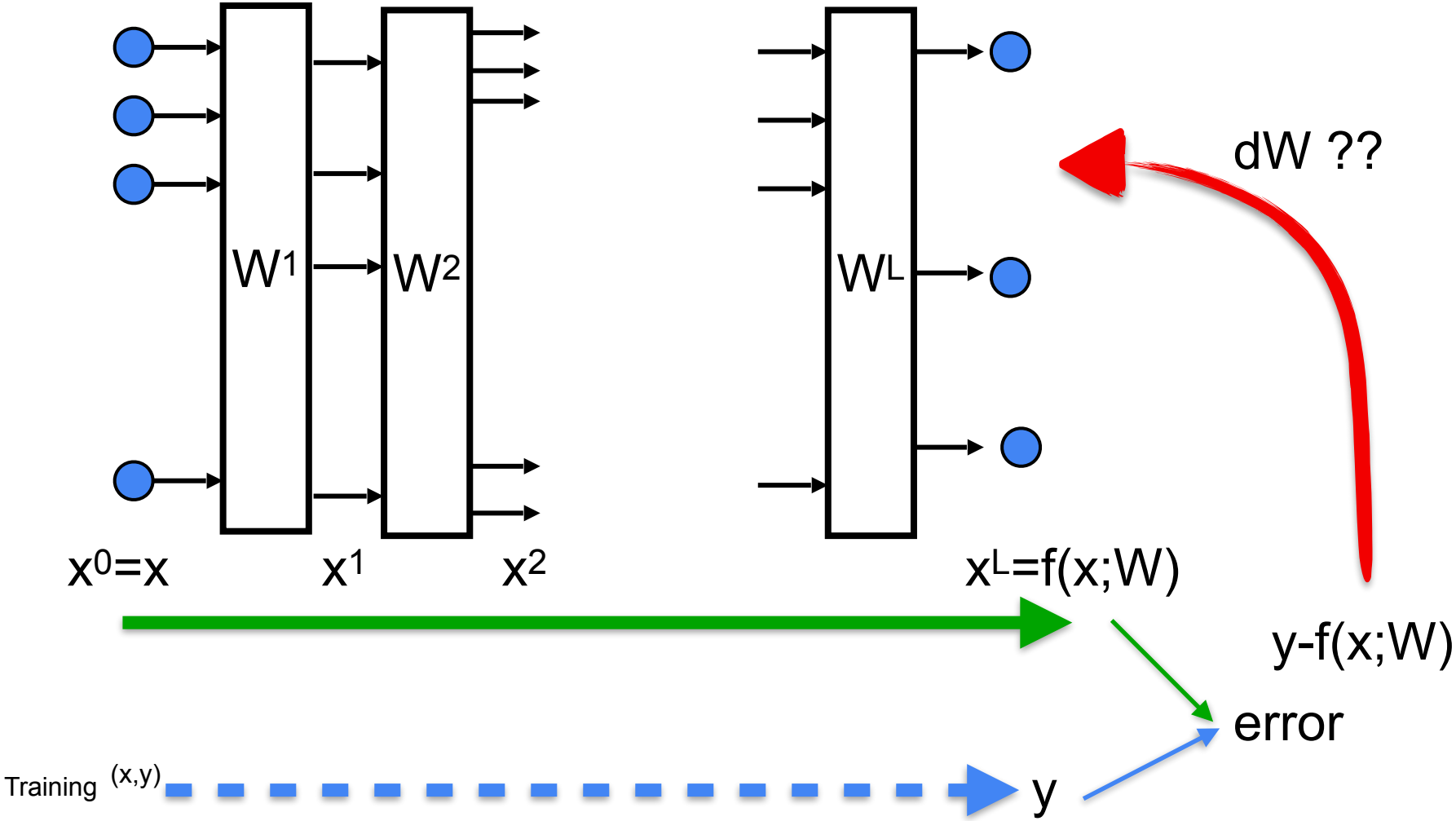

W1
W2
WL
x0=x
x1
x2
xL=f(x;W)


W1
W2
WL
x0=x
x1
x2
xL=f(x;W)


W1
W2
WL
x0=x
x1
x2
xL=f(x;W)
Training (x,y)
y


W1
W2
WL
x0=x
x1
x2
xL=f(x;W)
error
Training (x,y)
y


W1
W2
WL
x0=x
x1
x2
xL=f(x;W)
y-f(x;W)
error
Training (x,y)
y


W1
W2
WL
dW ??
x0=x
x1
x2
xL=f(x;W)
y-f(x;W)
error
Training (x,y)
y

Idea 0: Minimize a loss function

$$e_n = l(y_n, f(x_n; W))$$

$$\mathcal{L}(W) = \sum_{n=1}^{N} e_n$$

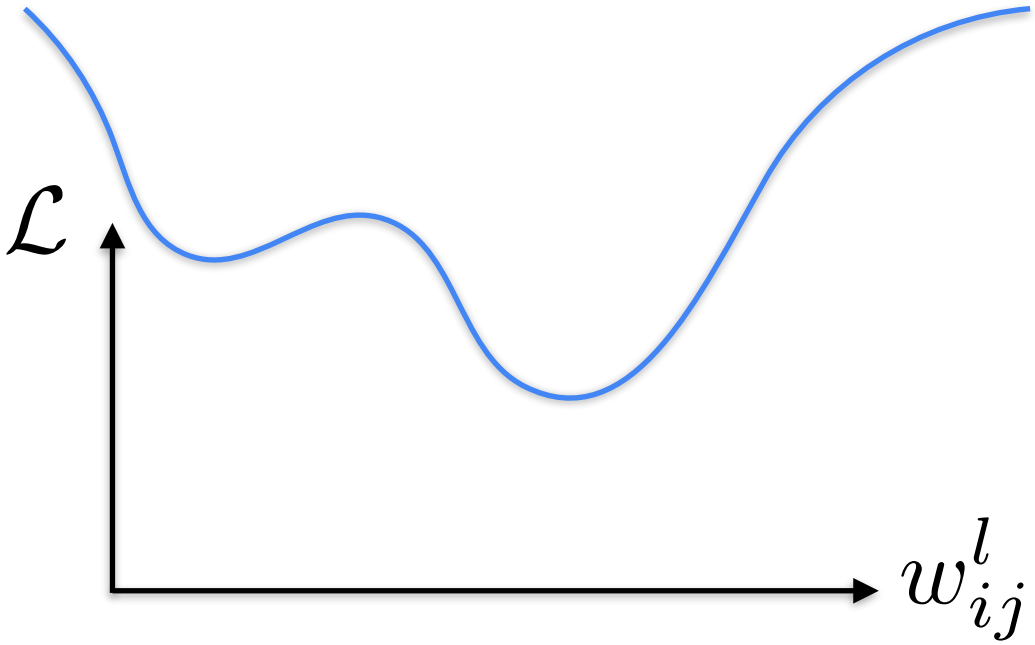
e.g. RMS loss

$$e_n = \|y_n - f(x_n; W)\|^2$$

Idea 1: Gradient descent

$$\mathcal{L}(W) = \sum_{n=1}^{N} e_n$$

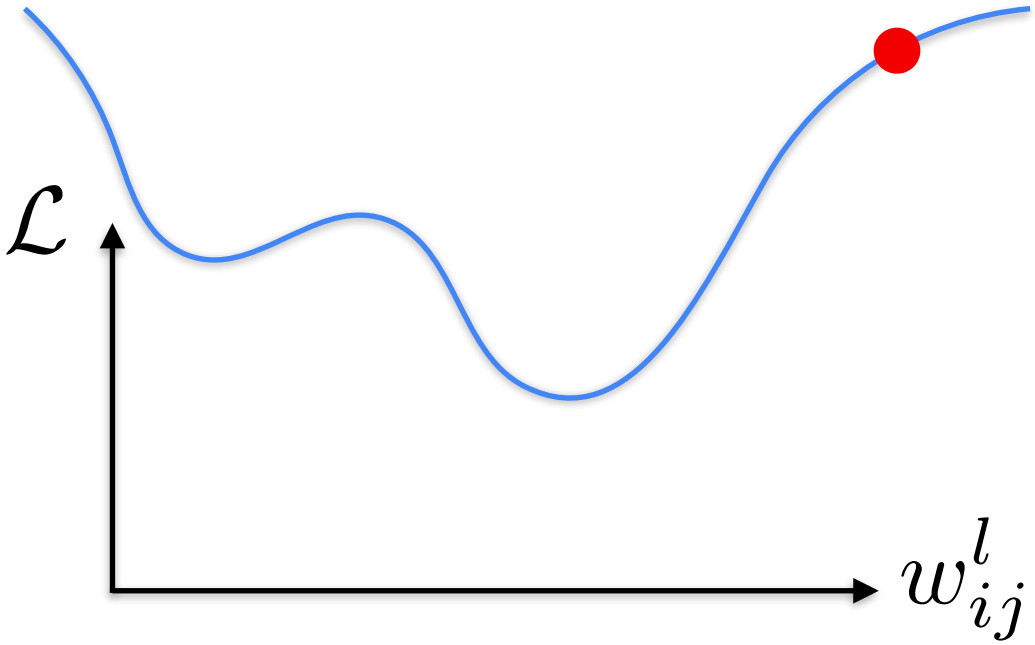
$$\nabla \mathcal{L}_{(W)}$$



Idea 1: Gradient descent

$$\mathcal{L}(W) = \sum_{n=1}^{N} e_n$$

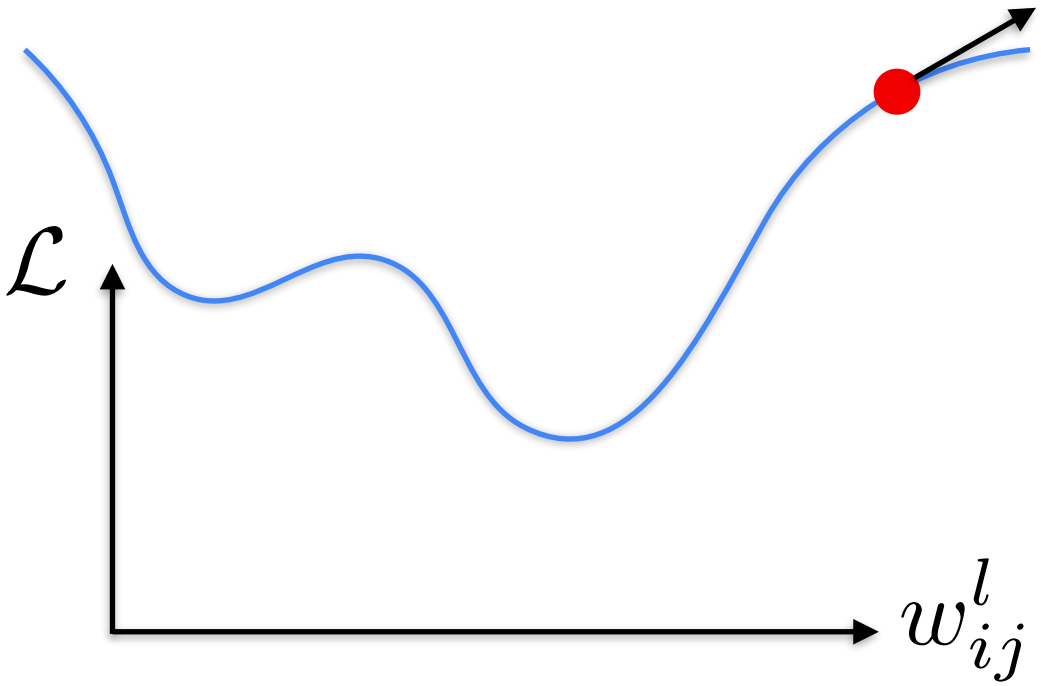
$$\nabla \mathcal{L}_{(W)}$$



Idea 1: Gradient descent

$$\mathcal{L}(W) = \sum_{n=1}^{N} e_n$$

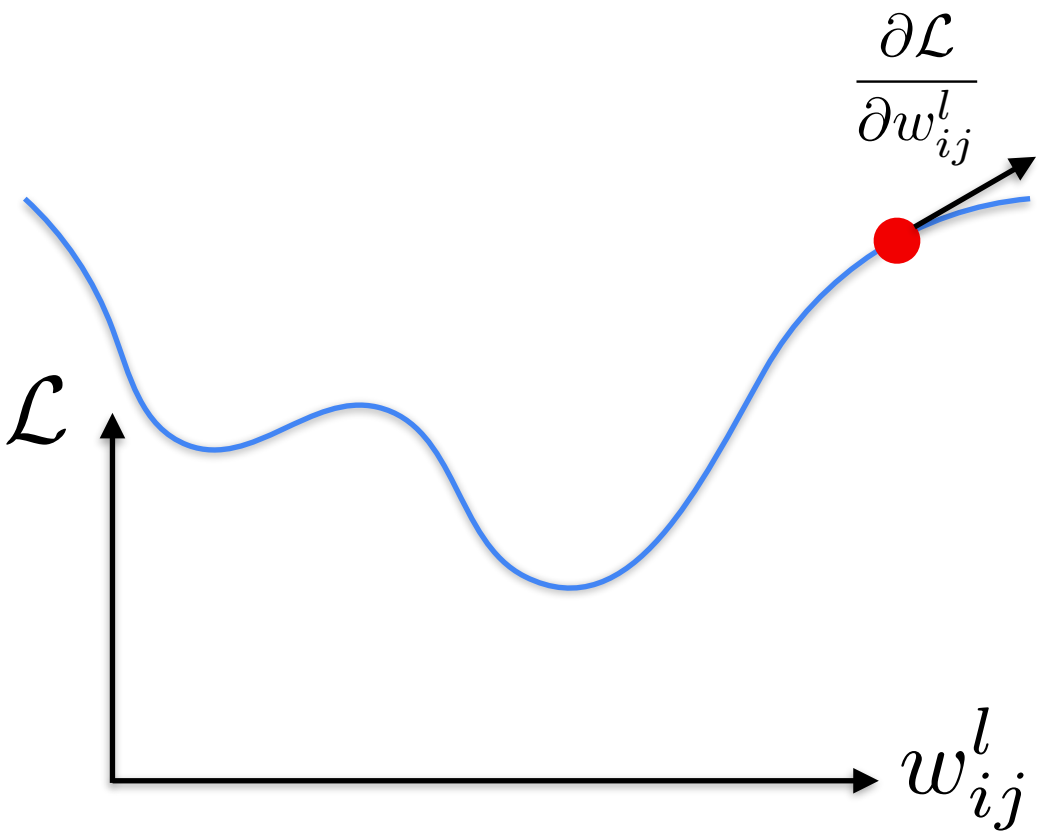
$$\nabla \mathcal{L}_{(W)}$$



Idea 1: Gradient descent

$$\mathcal{L}(W) = \sum_{n=1}^{N} e_n$$

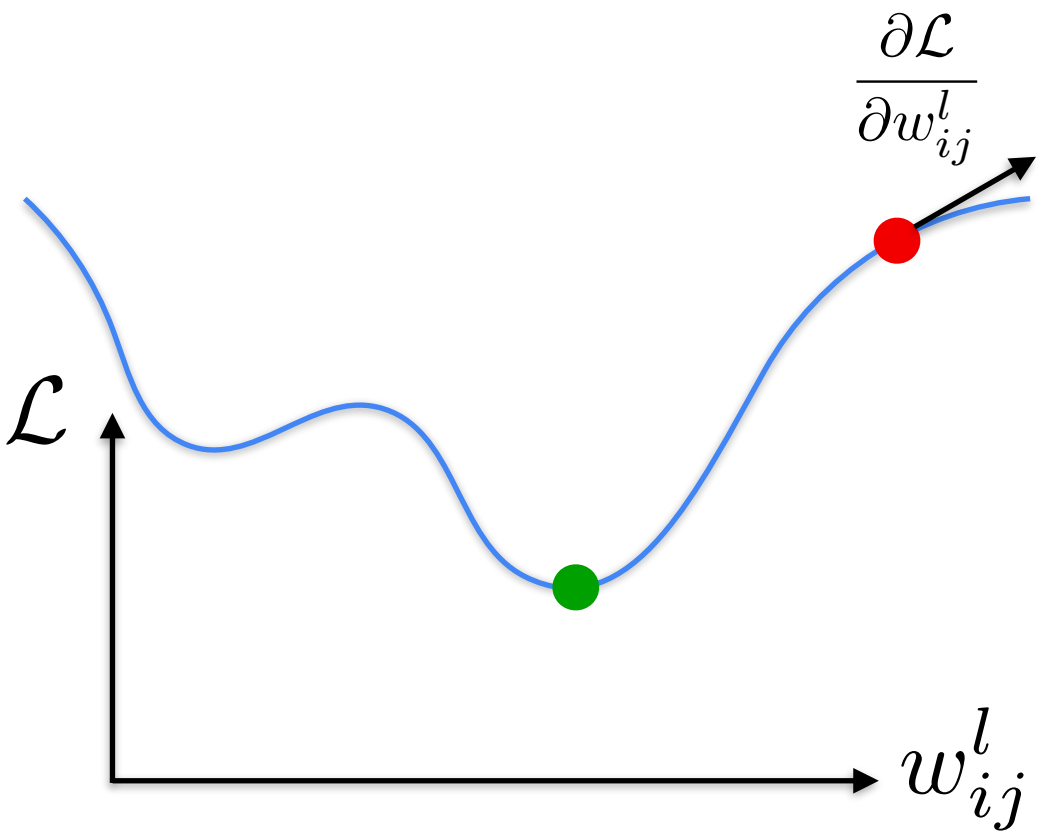
$$\nabla \mathcal{L}_{(W)}$$



Idea 1: Gradient descent

$$\mathcal{L}(W) = \sum_{n=1}^{N} e_n$$

$$\nabla \mathcal{L}_{(W)}$$



Idea 2: *Stochastic* gradient descent (SGD)

$$\nabla \mathcal{L}_{(W)}$$

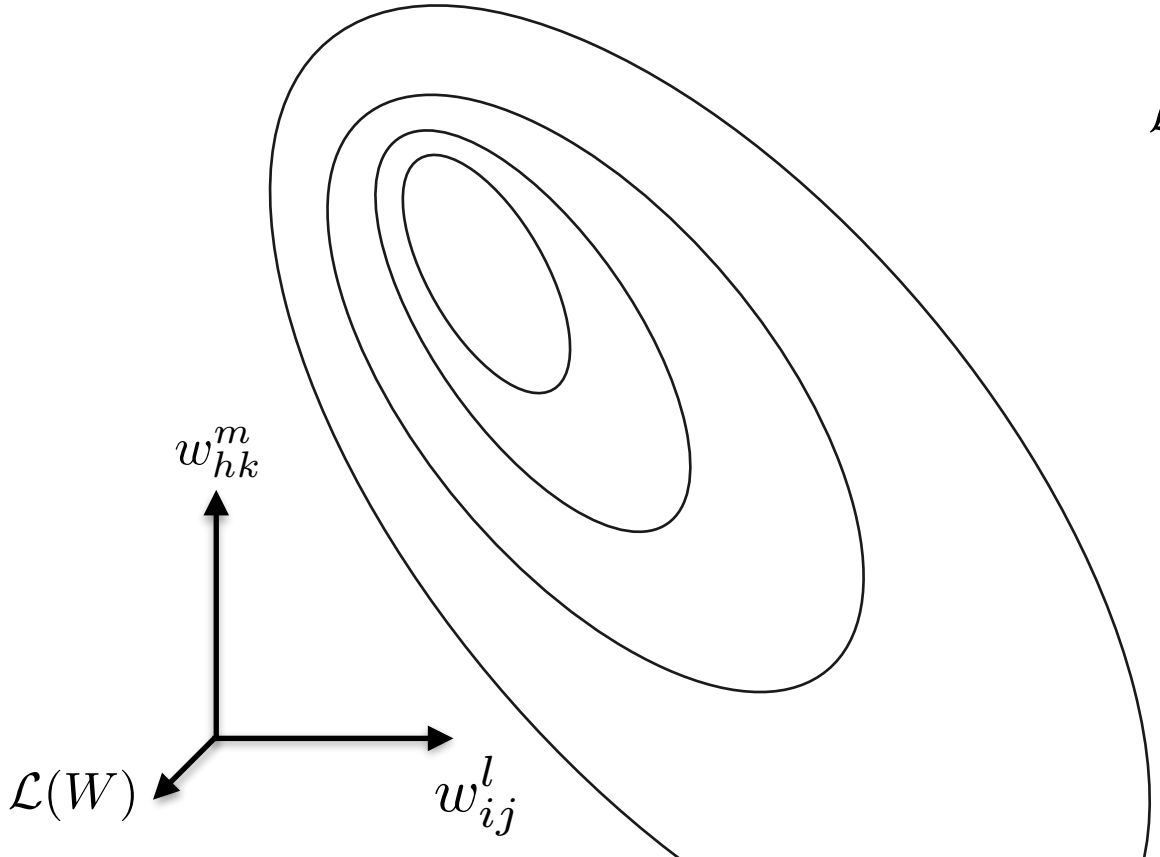
$$\mathcal{L}(W) = \sum_{n=1}^{N} e_n$$

(x_1, y_1)

$\cdots$

(x_N, y_N)

Training data



Idea 2: *Stochastic* gradient descent (SGD)

$$\nabla \mathcal{L}_{(W)}$$

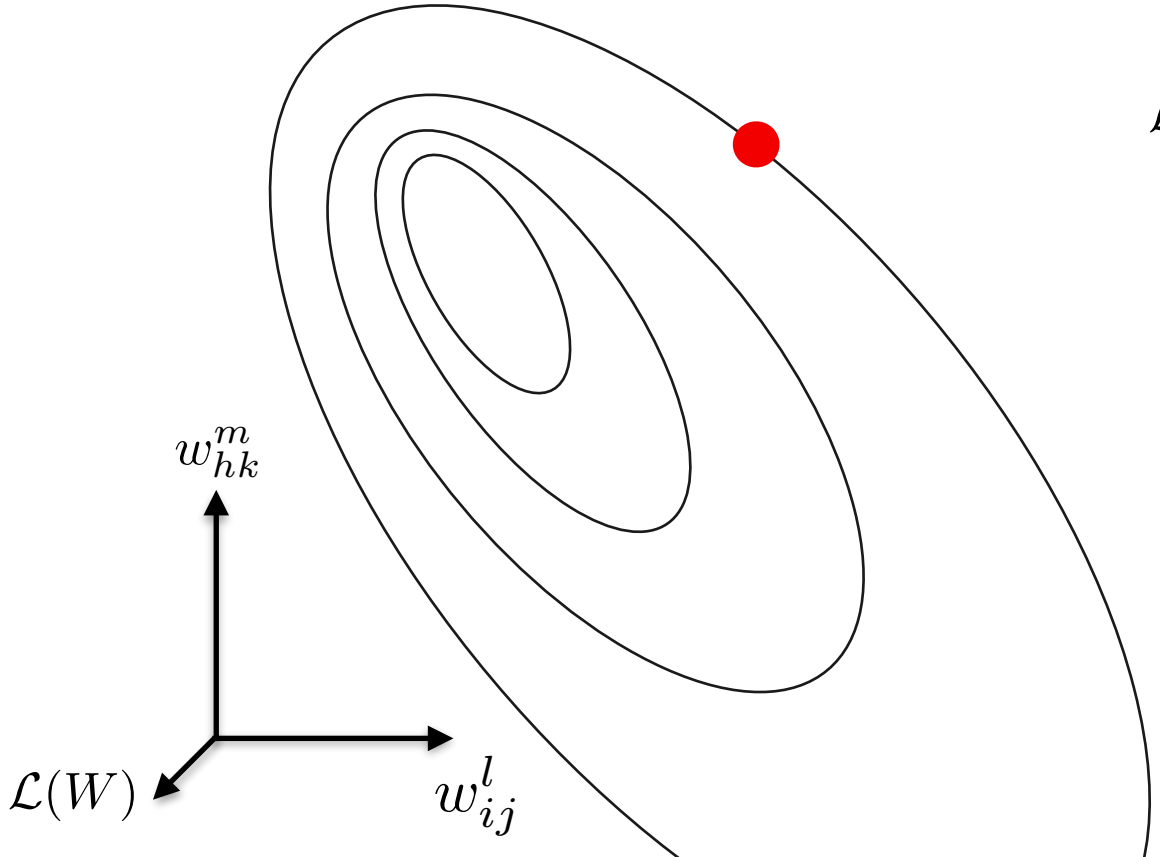
$$\mathcal{L}(W) = \sum_{n=1}^{N} e_n$$

(x_1, y_1)

$\cdots$

(x_N, y_N)

Training data



Idea 2: *Stochastic* gradient descent (SGD)

$$\nabla \mathcal{L}_{(W)}$$

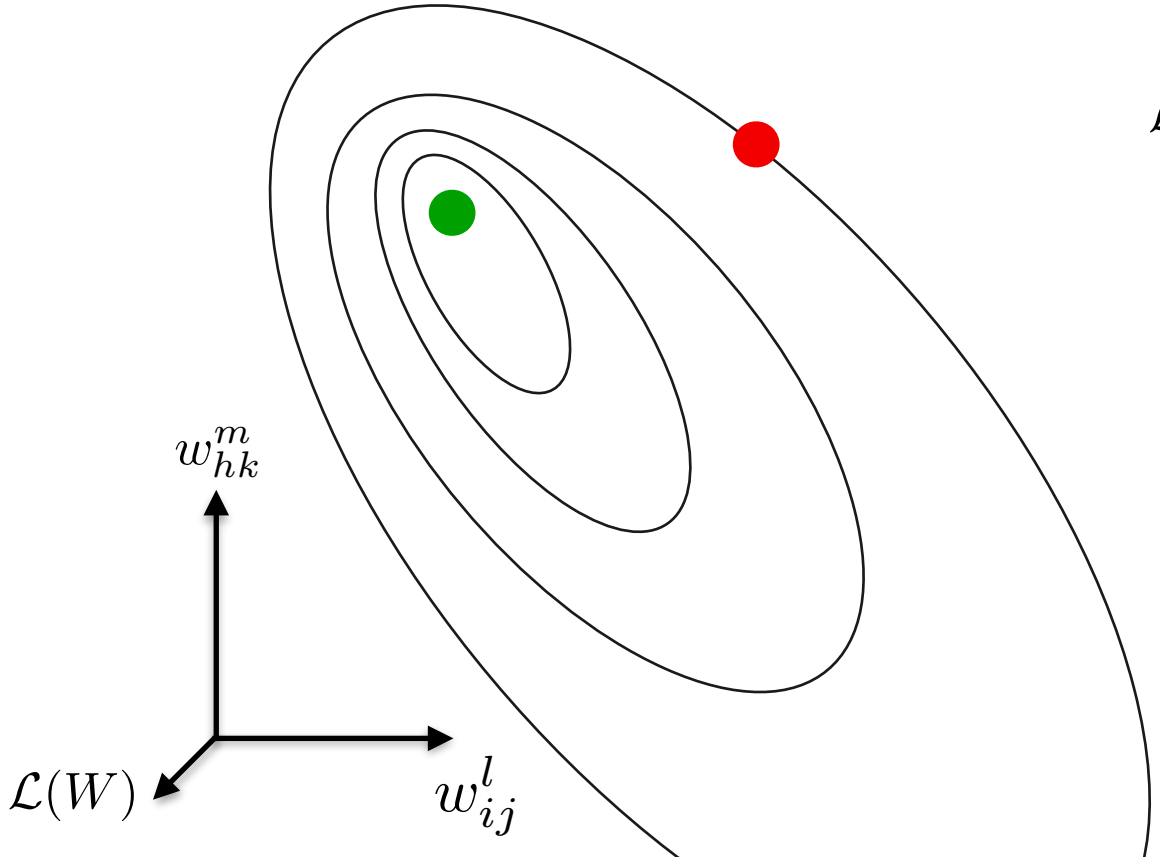
$$\mathcal{L}(W) = \sum_{n=1}^{N} e_n$$

(x_1, y_1)

$\cdots$

(x_N, y_N)

Training data



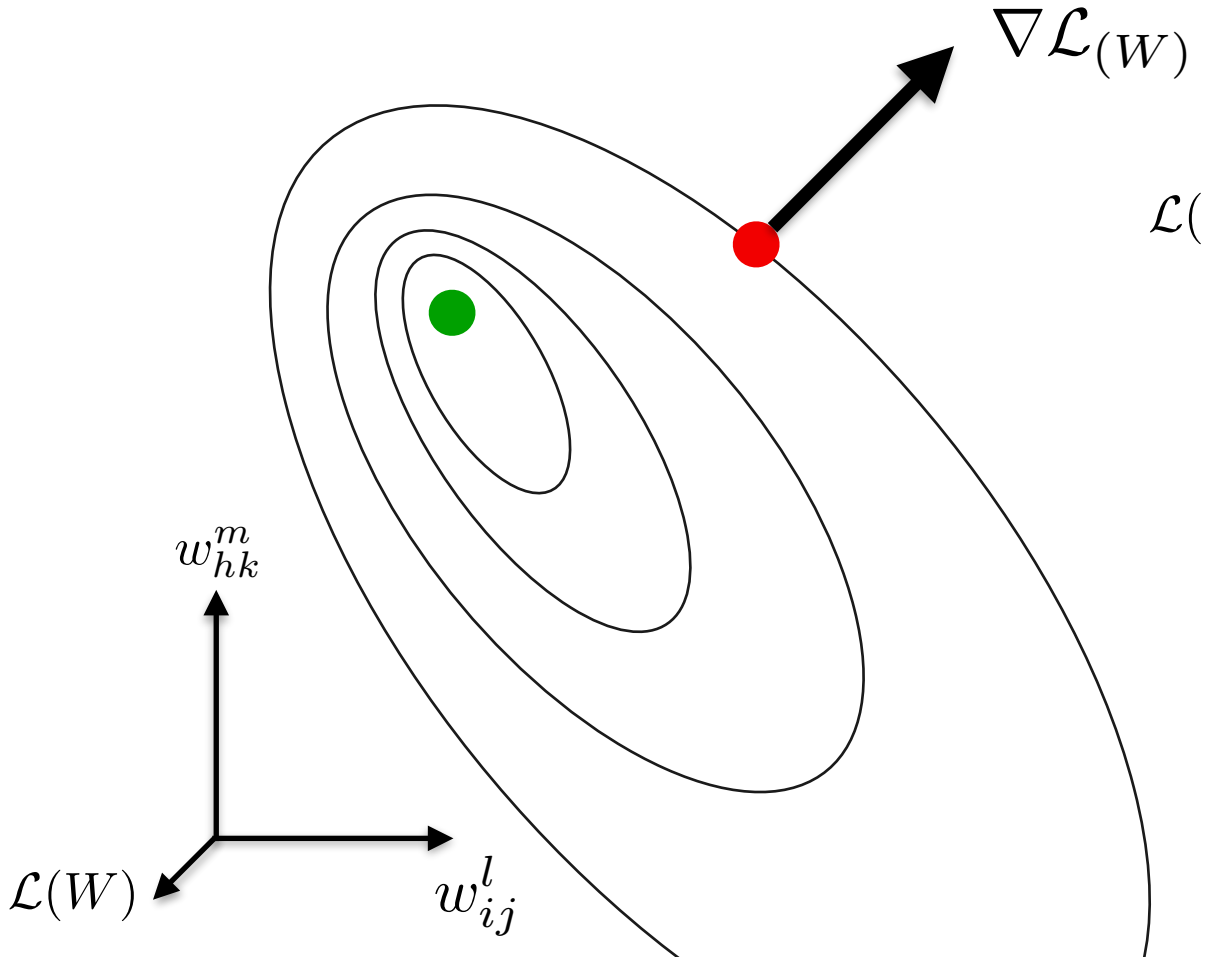
Idea 2: *Stochastic* gradient descent (SGD)

(x_1, y_1)

$\cdots$

(x_N, y_N)

Training data



$$\mathcal{L}(W) = \sum_{n=1}^{N} e_n$$

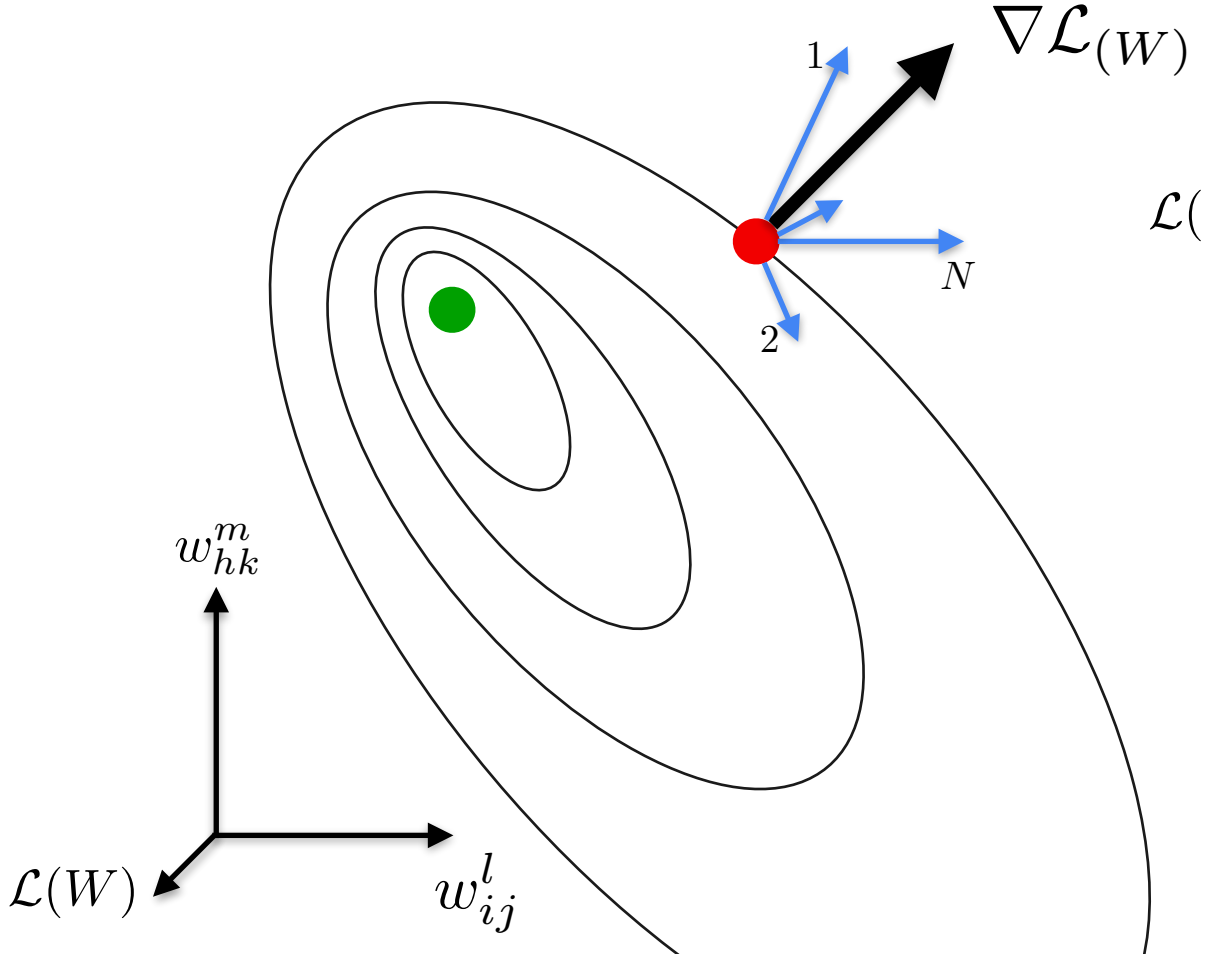
Idea 2: *Stochastic* gradient descent (SGD)

(x_1, y_1)

$\cdots$

(x_N, y_N)

Training data



$$\mathcal{L}(W) = \sum_{n=1}^{N} e_n$$

All the weights $\mathbf{w} = \{w_{ij}^{(l)}\}$ determine $h(\mathbf{x})$

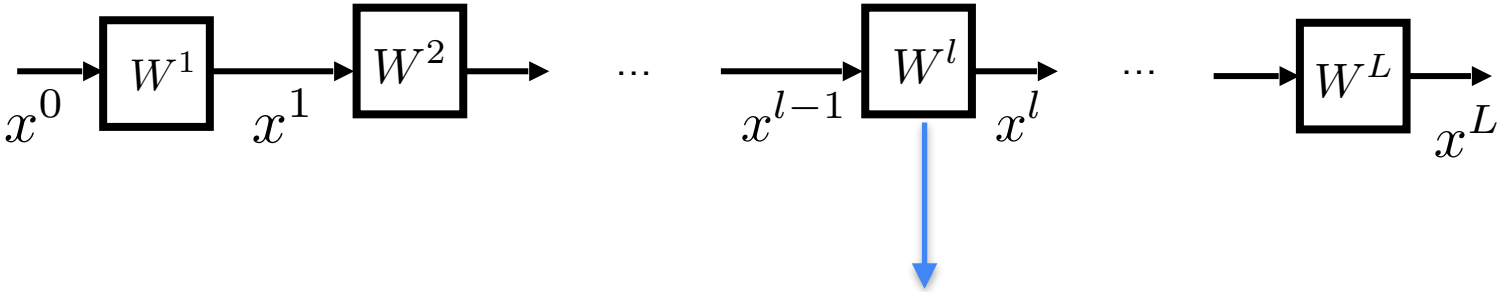
Error on example $(\mathbf{x}_n, y_n)$ is

$$\mathbf{e}\left(h(\mathbf{x}_n), y_n\right) = \mathbf{e}(\mathbf{w})$$

To implement SGD, we need the gradient

$$\nabla \mathbf{e}(\mathbf{w}): \quad \frac{\partial\ \mathrm{e}(\mathbf{w})}{\partial\ w_{ij}^{(l)}} \quad \text{for all } i, j, l$$

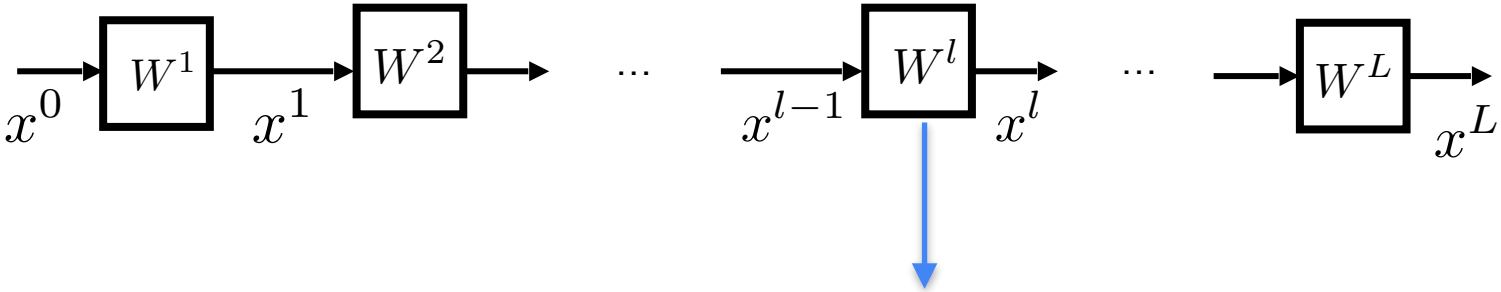
Idea 3: Chain rule



$$x_i^l = h(s_i) = h(\sum_j w_{ij}^l x^{l-1})$$

Idea 3: Chain rule

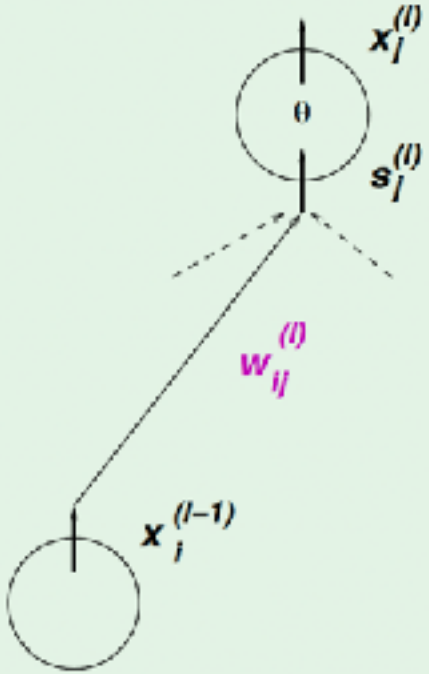
$$x^L = h(W^L\, x^{L-1}) = h(W^L\, h(W^{L-1} x^{L-2})) \ \ldots$$



$$x_i^l = h(s_i) = h(\sum_j w_{ij}^l x^{l-1})$$

Computing $\dfrac{\partial\ \mathrm{e}(\mathbf{w})}{\partial\ w_{ij}^{(l)}}$

We can evaluate $\dfrac{\partial\ \mathrm{e}(\mathbf{w})}{\partial\ w_{ij}^{(l)}}$ one by one: analytically or numerically



A trick for efficient computation:

$$\frac{\partial\ \mathrm{e}(\mathbf{w})}{\partial\ w_{ij}^{(l)}} = \frac{\partial\ \mathrm{e}(\mathbf{w})}{\partial\ s_j^{(l)}} \times \frac{\partial\ s_j^{(l)}}{\partial\ w_{ij}^{(l)}}$$

We have $\dfrac{\partial\ s_j^{(l)}}{\partial\ w_{ij}^{(l)}} = x_i^{(l-1)}$ We only need: $\dfrac{\partial\ \mathrm{e}(\mathbf{w})}{\partial\ s_j^{(l)}} = \delta_j^{(l)}$

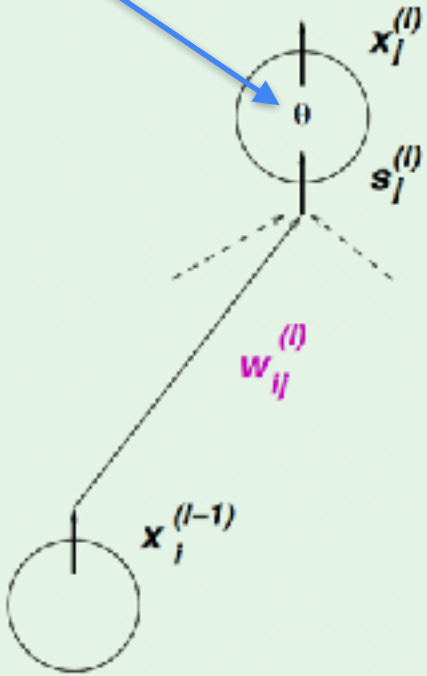
Computing $\dfrac{\partial\ \mathrm{e}(\mathbf{w})}{\partial\ w_{ij}^{(l)}}$

Change of notation:
This was h(.)

We can evaluate $\dfrac{\partial\ \mathrm{e}(\mathbf{w})}{\partial\ w_{ij}^{(l)}}$ one by one: analytically or numerically

A trick for efficient computation:

$$\frac{\partial\ \mathrm{e}(\mathbf{w})}{\partial\ w_{ij}^{(l)}} = \frac{\partial\ \mathrm{e}(\mathbf{w})}{\partial\ s_j^{(l)}} \times \frac{\partial\ s_j^{(l)}}{\partial\ w_{ij}^{(l)}}$$

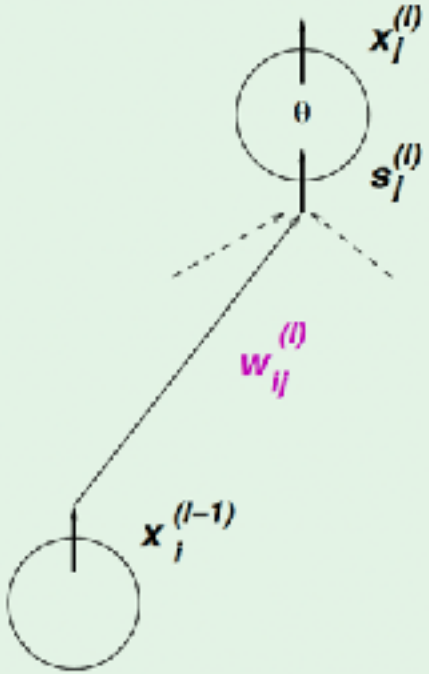


We have $\dfrac{\partial\ s_j^{(l)}}{\partial\ w_{ij}^{(l)}} = x_i^{(l-1)}$

We only need: $\dfrac{\partial\ \mathrm{e}(\mathbf{w})}{\partial\ s_j^{(l)}} = \delta_j^{(l)}$

Computing $\dfrac{\partial\ \mathrm{e}(\mathbf{w})}{\partial\ w_{ij}^{(l)}}$

We can evaluate $\dfrac{\partial\ \mathrm{e}(\mathbf{w})}{\partial\ w_{ij}^{(l)}}$ one by one: analytically or numerically



A trick for efficient computation:

$$\frac{\partial\ \mathrm{e}(\mathbf{w})}{\partial\ w_{ij}^{(l)}} = \frac{\partial\ \mathrm{e}(\mathbf{w})}{\partial\ s_j^{(l)}} \times \frac{\partial\ s_j^{(l)}}{\partial\ w_{ij}^{(l)}}$$

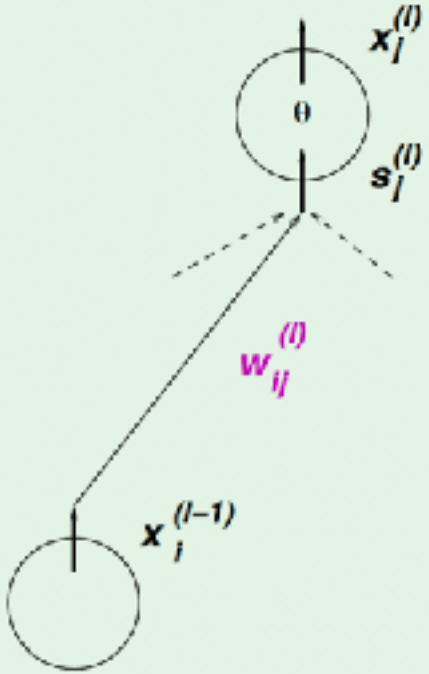
We have $\dfrac{\partial\ s_j^{(l)}}{\partial\ w_{ij}^{(l)}} = x_i^{(l-1)}$ We only need: $\dfrac{\partial\ \mathrm{e}(\mathbf{w})}{\partial\ s_j^{(l)}} = \delta_j^{(l)}$

Computing $\dfrac{\partial\ \mathrm{e}(\mathbf{w})}{\partial\ w_{ij}^{(l)}}$

We can evaluate $\dfrac{\partial\ \mathrm{e}(\mathbf{w})}{\partial\ w_{ij}^{(l)}}$ one by one: analytically or numerically

A trick for efficient computation:

$$\frac{\partial\ \mathrm{e}(\mathbf{w})}{\partial\ w_{ij}^{(l)}} = \frac{\partial\ \mathrm{e}(\mathbf{w})}{\partial\ s_j^{(l)}} \times \frac{\partial\ s_j^{(l)}}{\partial\ w_{ij}^{(l)}}$$



We have $\dfrac{\partial\ s_j^{(l)}}{\partial\ w_{ij}^{(l)}} = x_i^{(l-1)}$

We only need: $\dfrac{\partial\ \mathrm{e}(\mathbf{w})}{\partial\ s_j^{(l)}} = \delta_j^{(l)}$

Let's compute it!

Start with δ for the final layer

$$\delta_j^{(l)} = \frac{\partial\, \mathrm{e}(\mathbf{w})}{\partial\, s_j^{(l)}}$$

For the final layer $l = L$ and $j = 1$:

$$\delta_1^{(L)} = \frac{\partial\, \mathrm{e}(\mathbf{w})}{\partial\, s_1^{(L)}}$$

$$\mathrm{e}(\mathbf{w}) = (\, x_1^{(L)} - y_n)^2$$

$$x_1^{(L)} = \theta(s_1^{(L)})$$

$$\theta'(s) = 1 - \theta^2(s)$$ for the tanh

Start with δ for the final layer

$$\delta_j^{(l)} = \frac{\partial\, \mathrm{e}(\mathbf{w})}{\partial\, s_j^{(l)}}$$

For the final layer $l = L$ and $j = 1$:

$$\delta_1^{(L)} = \frac{\partial\, \mathrm{e}(\mathbf{w})}{\partial\, s_1^{(L)}}$$

$$\mathrm{e}(\mathbf{w}) = (\, x_1^{(L)} - y_n)^2$$

$$\delta_1^{(L)} = 2(x_1^{(L)} - y_n)\ \theta'(s_j^{(L)})$$

$$x_1^{(L)} = \theta(s_1^{(L)})$$

$$\theta'(s) = 1 - \theta^2(s)$$ for the tanh

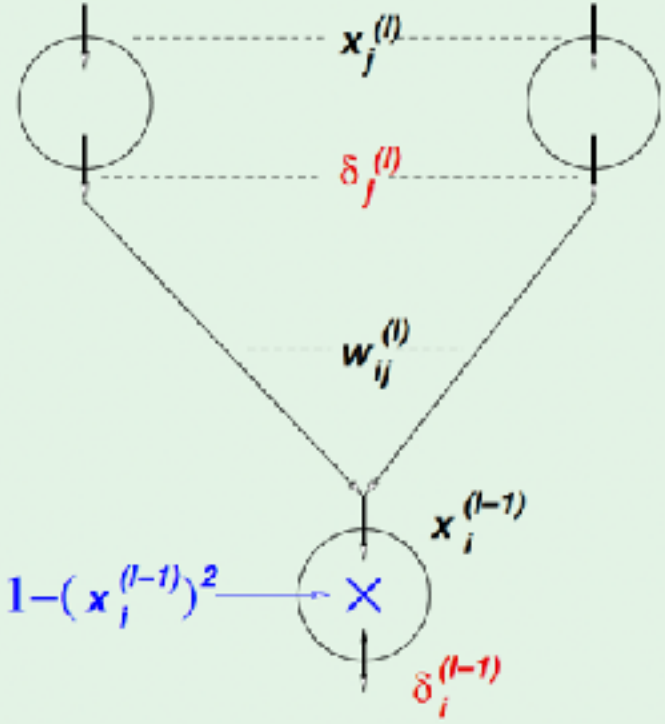
Back propagation of δ

$$\delta_i^{(l-1)} = \frac{\partial\, \mathrm{e}(\mathbf{w})}{\partial\, s_i^{(l-1)}}$$

$$= \sum_{j=1}^{d^{(l)}} \frac{\partial\, \mathrm{e}(\mathbf{w})}{\partial\, s_j^{(l)}} \times \frac{\partial\, s_j^{(l)}}{\partial\, x_i^{(l-1)}} \times \frac{\partial\, x_i^{(l-1)}}{\partial\, s_i^{(l-1)}}$$

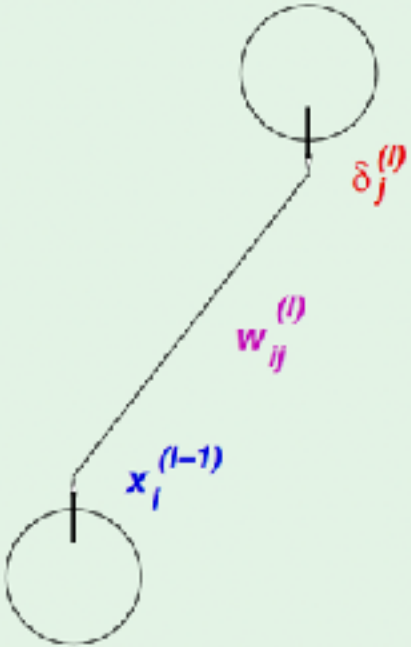
$$= \sum_{j=1}^{d^{(l)}} \delta_j^{(l)} \times w_{ij}^{(l)} \times \theta'(s_i^{(l-1)})$$

$$\delta_i^{(l-1)} = (1 - (x_i^{(l-1)})^2) \sum_{j=1}^{d^{(l)}} w_{ij}^{(l)}\, \delta_j^{(l)}$$



Backpropagation algorithm

1. Initialize all weights $w_{ij}^{(l)}$ at random
2. **for** $t = 0, 1, 2, \ldots$ **do**
3. Pick $n \in \{1, 2, \cdots, N\}$
4. *Forward:* Compute all $x_j^{(l)}$
5. *Backward:* Compute all $\delta_j^{(l)}$
6. Update the weights: $w_{ij}^{(l)} \leftarrow w_{ij}^{(l)} - \eta\, x_i^{(l-1)} \delta_j^{(l)}$
7. Iterate to the next step until it is time to stop
8. Return the final weights $w_{ij}^{(l)}$



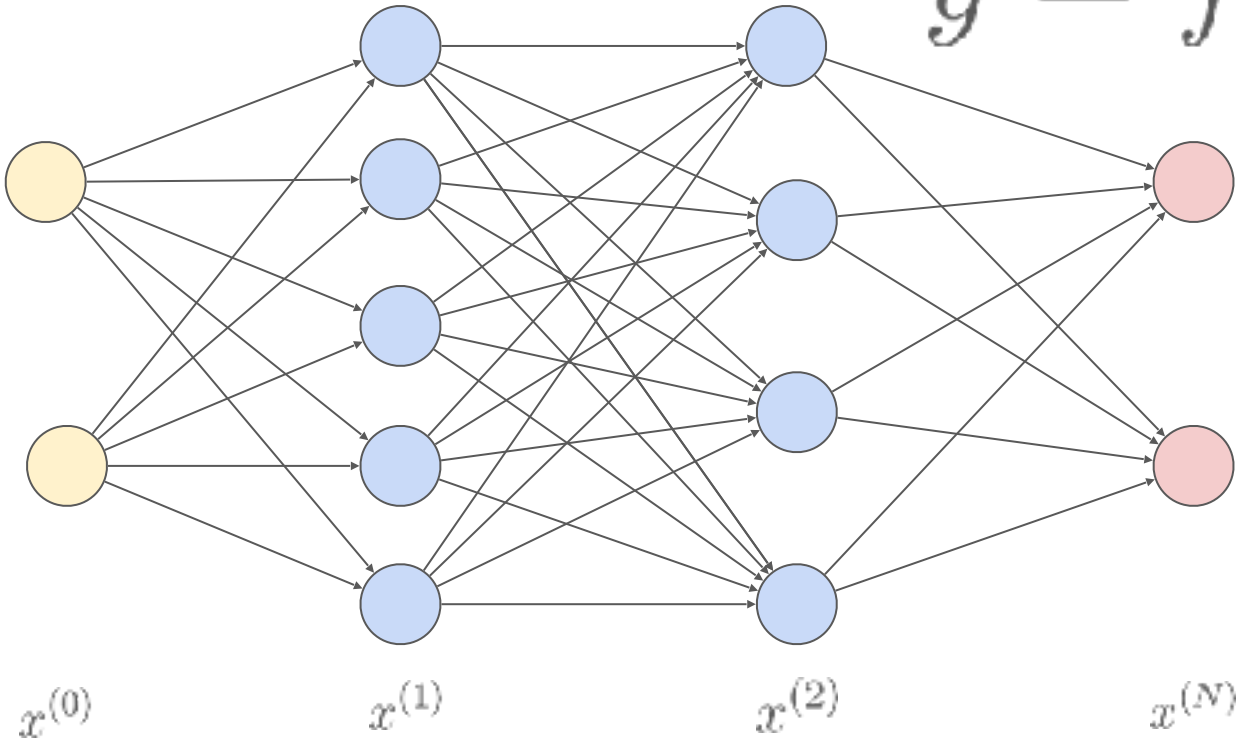
Recap

- Defined neural networks
- Loss functions for classification and regression
- Gradient descent
- Backpropagation

Neural net

$$x_i^{(n)} = \sigma\left(\sum_j w_{ij}^{(n)} x_j^{(n-1)} + b_i^{(n)} \right)$$

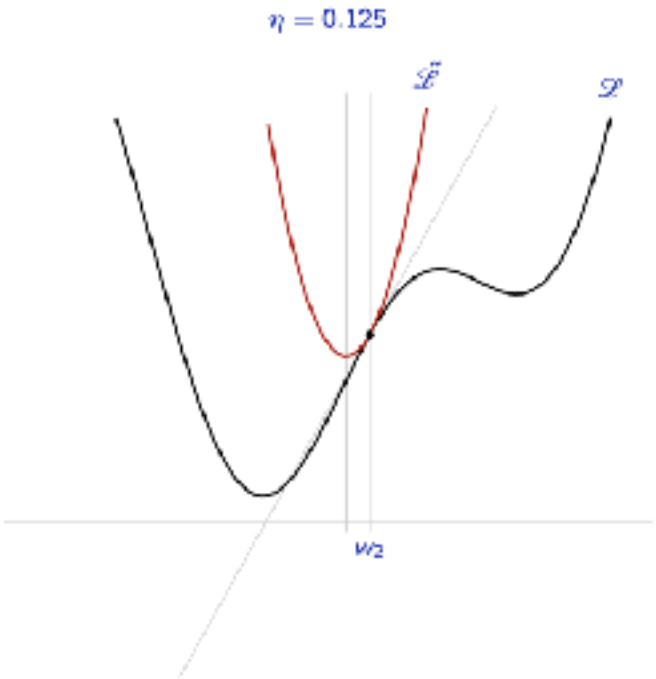
$$y = f(x, w)$$



Recap: loss functions and gradient descent

$$\mathcal{L}(w) = \frac{1}{K} \sum_{(\bar{x},\bar{y}) \in \mathcal{D}} \mathcal{L}(w, \bar{x}, \bar{y})$$

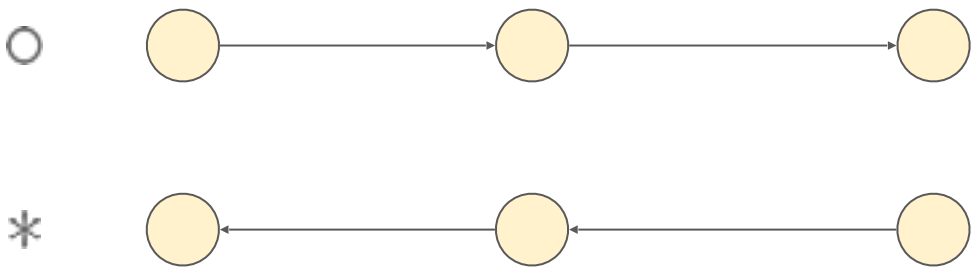
$$w_{t+1} = w_t - \eta \nabla \mathcal{L}(w_t)$$



Recap: backpropagation

$$\boxed{y = f(x, w)}$$

$$x^{(N-2)} \xrightarrow{w^{(N-1)}, b^{(N-1)}} s^{(N-1)} \xrightarrow{\sigma} x^{(N-1)} \xrightarrow{w^{(N)}, b^{(N)}} s^{(N)} \xrightarrow{\sigma} x^{(N)}$$



Recap: backpropagation

$$\frac{\partial x^{(N)}}{dw_{i,j}^{(N)}}(\bar{x}) = \boxed{\sigma'(s^{(N)}(\bar{x}))} * x_j^{(N-1)}(\bar{x})$$

$$\frac{\partial x^{(N)}}{db_i^{(N)}}(\bar{x}) = \boxed{\sigma'(s^{(N)}(\bar{x}))}$$

$$x^{(N-2)} \xrightarrow{w^{(N-1)},b^{(N-1)}} s^{(N-1)} \xrightarrow{\sigma} x^{(N-1)} \xrightarrow{w^{(N)},b^{(N)}} s^{(N)} \xrightarrow{\sigma} x^{(N)}$$

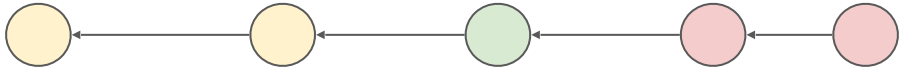
$$\frac{\partial x^{(N)}}{dw_{i,j}^{(N-1)}}(\bar{x}) = \boxed{\sigma'(s^{(N)}(\bar{x}))} * \sum_j w_{ij}^{(N)} \sigma'(s^{(N-1)}(\bar{x})) * x_j^{(N-2)}(\bar{x})$$

$$\frac{\partial x^{(N)}}{db_i^{(N-1)}}(\bar{x}) = \sum_j w_{ij}^{(N)} \sigma'(s^{(N-1)}(\bar{x})) * x_j^{(N-2)}(\bar{x})$$

$$\frac{\partial x^{(N)}}{dw_{i,j}^{(N)}}(\bar{x}) = \boxed{\sigma'(s^{(N)}(\bar{x}))} * \boxed{x_j^{(N-1)}(\bar{x})}$$

$$\frac{\partial x^{(N)}}{db_i^{(N)}}(\bar{x}) = \boxed{\sigma'(s^{(N)}(\bar{x}))}$$

$$x^{(N-2)} \xrightarrow{w^{(N-1)},b^{(N-1)}} s^{(N-1)} \xrightarrow{\sigma} x^{(N-1)} \xrightarrow{w^{(N)},b^{(N)}} s^{(N)} \xrightarrow{\sigma} x^{(N)}$$



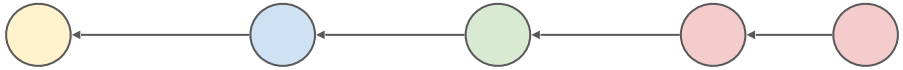
$$\frac{\partial x^{(N)}}{dw_{i,j}^{(N-1)}}(\bar{x}) = \boxed{\sigma'(s^{(N)}(\bar{x}))} * \sum_j \boxed{w_{ij}^{(N)}} \sigma'(s^{(N-1)}(\bar{x})) * x_j^{(N-2)}(\bar{x})$$

$$\frac{\partial x^{(N)}}{db_i^{(N-1)}}(\bar{x}) = \sum_j \boxed{w_{ij}^{(N)}} \sigma'(s^{(N-1)}(\bar{x})) * x_j^{(N-2)}(\bar{x})$$

$$\frac{\partial x^{(N)}}{dw_{i,j}^{(N)}}(\bar{x}) = \boxed{\sigma'(s^{(N)}(\bar{x}))} * \boxed{x_j^{(N-1)}(\bar{x})}$$

$$\frac{\partial x^{(N)}}{db_i^{(N)}}(\bar{x}) = \boxed{\sigma'(s^{(N)}(\bar{x}))}$$

$$x^{(N-2)} \xrightarrow{w^{(N-1)},b^{(N-1)}} s^{(N-1)} \xrightarrow{\sigma} x^{(N-1)} \xrightarrow{w^{(N)},b^{(N)}} s^{(N)} \xrightarrow{\sigma} x^{(N)}$$



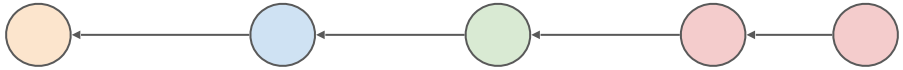
$$\frac{\partial x^{(N)}}{dw_{i,j}^{(N-1)}}(\bar{x}) = \boxed{\sigma'(s^{(N)}(\bar{x}))} * \sum_j \boxed{w_{ij}^{(N)}}\boxed{\sigma'(s^{(N-1)}(\bar{x}))} * x_j^{(N-2)}(\bar{x})$$

$$\frac{\partial x^{(N)}}{db_i^{(N-1)}}(\bar{x}) = \sum_j \boxed{w_{ij}^{(N)}}\boxed{\sigma'(s^{(N-1)}(\bar{x}))} * x_j^{(N-2)}(\bar{x})$$

$$\frac{\partial x^{(N)}}{dw_{i,j}^{(N)}}(\bar{x}) = \boxed{\sigma'(s^{(N)}(\bar{x}))} * \boxed{x_j^{(N-1)}(\bar{x})}$$

$$\frac{\partial x^{(N)}}{db_i^{(N)}}(\bar{x}) = \boxed{\sigma'(s^{(N)}(\bar{x}))}$$

$$x^{(N-2)} \xrightarrow{w^{(N-1)},b^{(N-1)}} s^{(N-1)} \xrightarrow{\sigma} x^{(N-1)} \xrightarrow{w^{(N)},b^{(N)}} s^{(N)} \xrightarrow{\sigma} x^{(N)}$$



$$\frac{\partial x^{(N)}}{dw_{i,j}^{(N-1)}}(\bar{x}) = \boxed{\sigma'(s^{(N)}(\bar{x}))} * \sum_j \boxed{w_{ij}^{(N)}}\boxed{\sigma'(s^{(N-1)}(\bar{x}))} * \boxed{x_j^{(N-2)}(\bar{x})}$$

$$\frac{\partial x^{(N)}}{db_i^{(N-1)}}(\bar{x}) = \sum_j \boxed{w_{ij}^{(N)}}\boxed{\sigma'(s^{(N-1)}(\bar{x}))} * \boxed{x_j^{(N-2)}(\bar{x})}$$

Gradient descent

$$w_{t+1} = w_t - \eta \nabla \mathcal{L}(w_t)$$

$$\nabla \mathcal{L}(w) = \sum_{(\bar{x},\bar{y}) \in \mathcal{D}} \frac{1}{K} \nabla \mathcal{L}(w, \bar{x}, \bar{y})$$

$$w_{t+1} = w_t - \eta \nabla \mathcal{L}(w_t)$$

$$\nabla \mathcal{L}(w) = \sum_{(\bar{x},\bar{y}) \in \mathcal{D}} \frac{1}{K} \nabla \mathcal{L}(w, \bar{x}, \bar{y})$$

```
def gradient_descent(gradient_func, initial_w, data,
                     learning_rate, tolerance, max_iterations):
  w = initial_w
  K = len(data)
  for i in range(max_iterations):
    gradient = 0
    for d in data:
      gradient += gradient_func(d, w)
    new_w = w - (1/K) * learning_rate * gradient
    if abs(new_w - w) < tolerance:
      return new_w
    w = new_w
 return w
```

What can go wrong?

$$w_{t+1} = w_t - \eta \nabla \mathcal{L}(w_t)$$

$$\nabla \mathcal{L}(w) = \sum_{(\bar{x},\bar{y}) \in \mathcal{D}} \frac{1}{K} \nabla \mathcal{L}(w, \bar{x}, \bar{y})$$

What can go wrong?

$$w_{t+1} = w_t - \eta \nabla \mathcal{L}(w_t)$$

$$\nabla \mathcal{L}(w) = \sum_{(\bar{x},\bar{y}) \in \mathcal{D}} \frac{1}{K} \nabla \mathcal{L}(w, \bar{x}, \bar{y})$$

What can go wrong?

No convergence

$$w_{t+1} = w_t - \eta \nabla \mathcal{L}(w_t)$$

$$\nabla \mathcal{L}(w) = \sum_{(\bar{x},\bar{y}) \in \mathcal{D}} \frac{1}{K} \nabla \mathcal{L}(w, \bar{x}, \bar{y})$$

What can go wrong?

No convergence

Local minimum

$$w_{t+1} = w_t - \eta \nabla \mathcal{L}(w_t)$$

$$\nabla \mathcal{L}(w) = \sum_{(\bar{x},\bar{y}) \in \mathcal{D}} \frac{1}{K} \nabla \mathcal{L}(w, \bar{x}, \bar{y})$$

What can go wrong?

No convergence

Local minimum

Poor generalization

$$w_{t+1} = w_t - \eta \nabla \mathcal{L}(w_t)$$

$$\nabla \mathcal{L}(w) = \sum_{(\bar{x},\bar{y}) \in \mathcal{D}} \frac{1}{K} \nabla \mathcal{L}(w, \bar{x}, \bar{y})$$

What can go wrong?

$$w_{t+1} = w_t - \eta \nabla \mathcal{L}(w_t)$$

$$\nabla \mathcal{L}(w) = \sum_{(\bar{x},\bar{y}) \in \mathcal{D}} \frac{1}{K} \nabla \mathcal{L}(w, \bar{x}, \bar{y})$$

No convergence

Local minimum

Poor generalization

High loss at convergence

What can go wrong?

$$w_{t+1} = w_t - \eta \nabla \mathcal{L}(w_t)$$

$$\nabla \mathcal{L}(w) = \sum_{(\bar{x},\bar{y}) \in \mathcal{D}} \frac{1}{K} \nabla \mathcal{L}(w, \bar{x}, \bar{y})$$

No convergence

Local minimum

Poor generalization

High loss at convergence

Slow convergence

What can go wrong?

$$w_{t+1} = w_t - \eta \nabla \mathcal{L}(w_t)$$

$$\nabla \mathcal{L}(w) = \sum_{(\bar{x},\bar{y}) \in \mathcal{D}} \frac{1}{K} \nabla \mathcal{L}(w, \bar{x}, \bar{y})$$

- No convergence
- Local minimum
- Poor generalization
- High loss at convergence
- Slow convergence
- Heavy computation

What can go wrong?

$$w_{t+1} = w_t - \eta \nabla \mathcal{L}(w_t)$$

$$\nabla \mathcal{L}(w) = \sum_{(\bar{x},\bar{y}) \in \mathcal{D}} \frac{1}{K} \nabla \mathcal{L}(w, \bar{x}, \bar{y})$$

- No convergence
- Local minimum
- Poor generalization
- High loss at convergence
- Slow convergence
- Heavy computation
- Vanishing gradient

What can go wrong?

$$w_{t+1} = w_t - \eta \nabla \mathcal{L}(w_t)$$

$$\nabla \mathcal{L}(w) = \sum_{(\bar{x},\bar{y}) \in \mathcal{D}} \frac{1}{K} \nabla \mathcal{L}(w, \bar{x}, \bar{y})$$

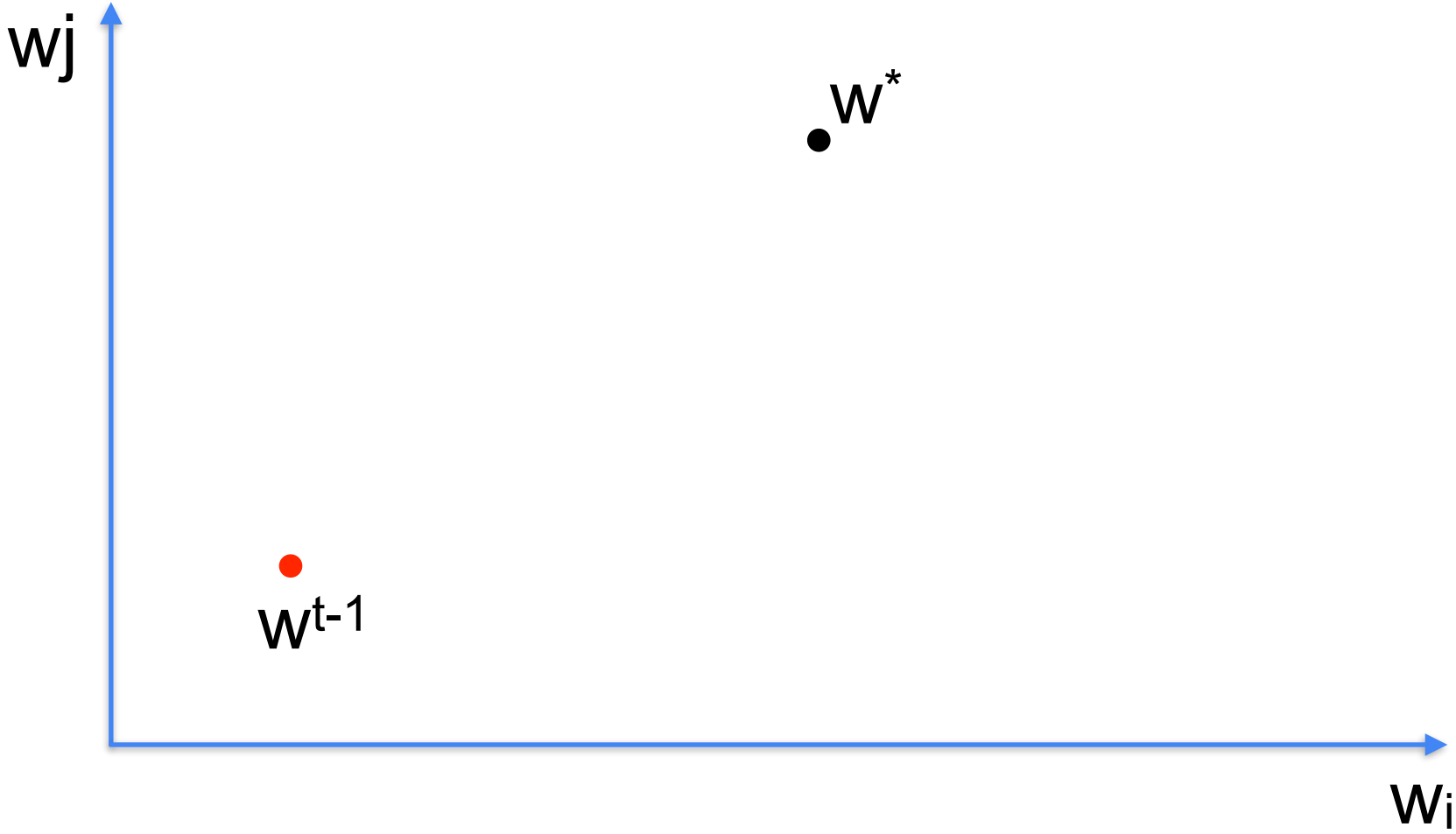
- No convergence
- Local minimum
- Poor generalization
- High loss at convergence
- Slow convergence
- Heavy computation
- Vanishing gradient
- Exploding gradient

Stochastic gradient descent

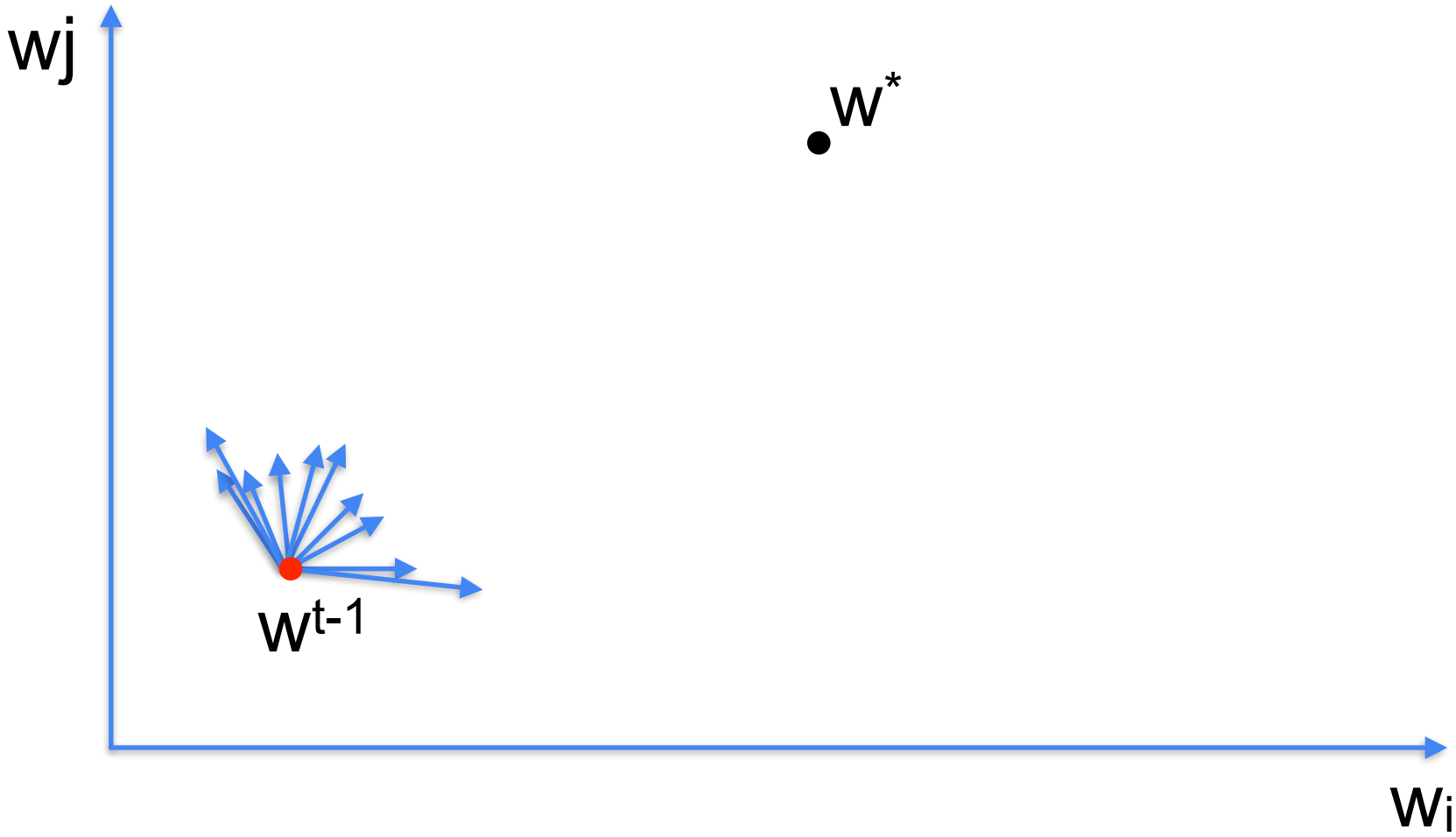
Gradient descent

Gradient descent

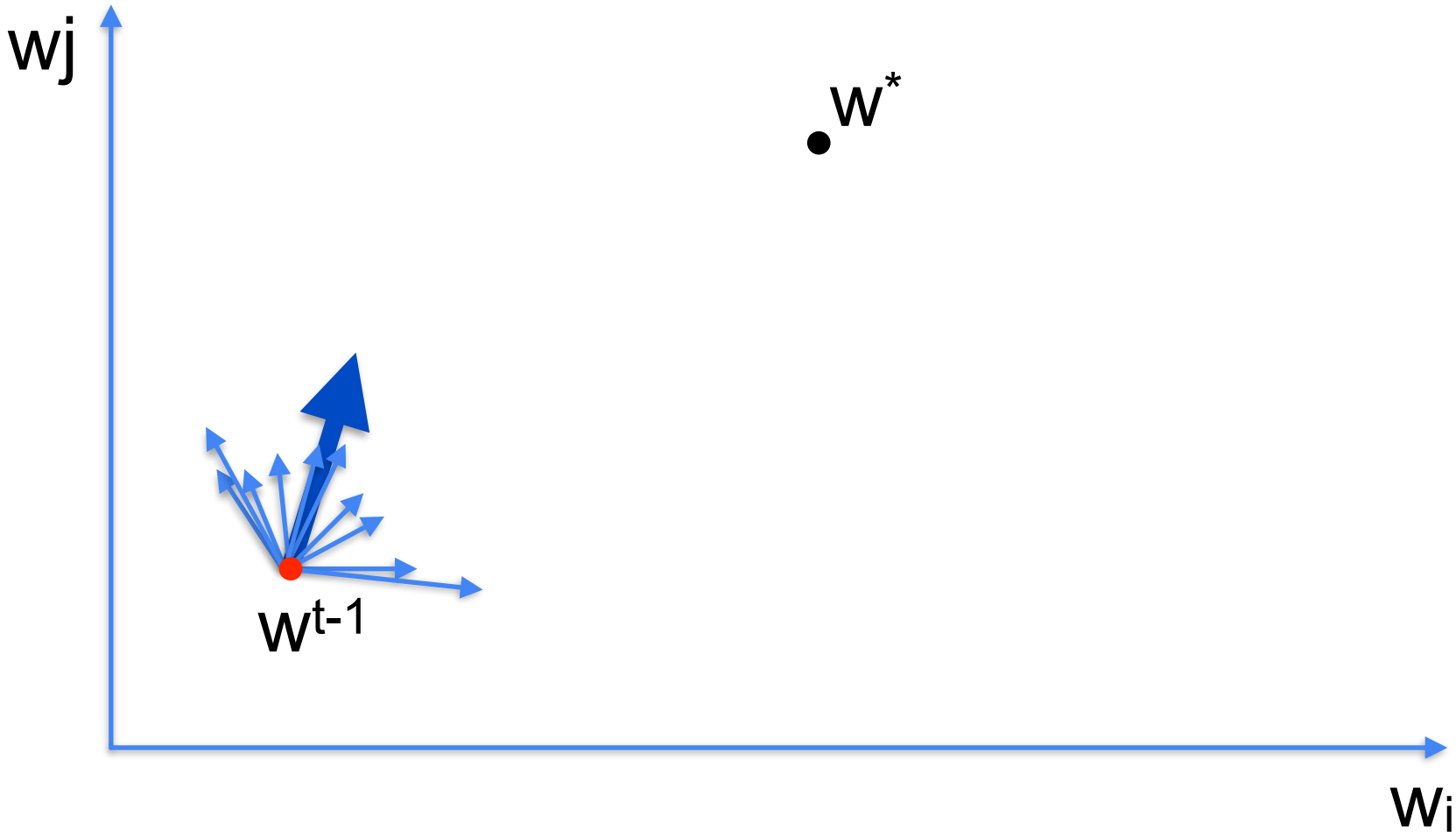
Gradient descent



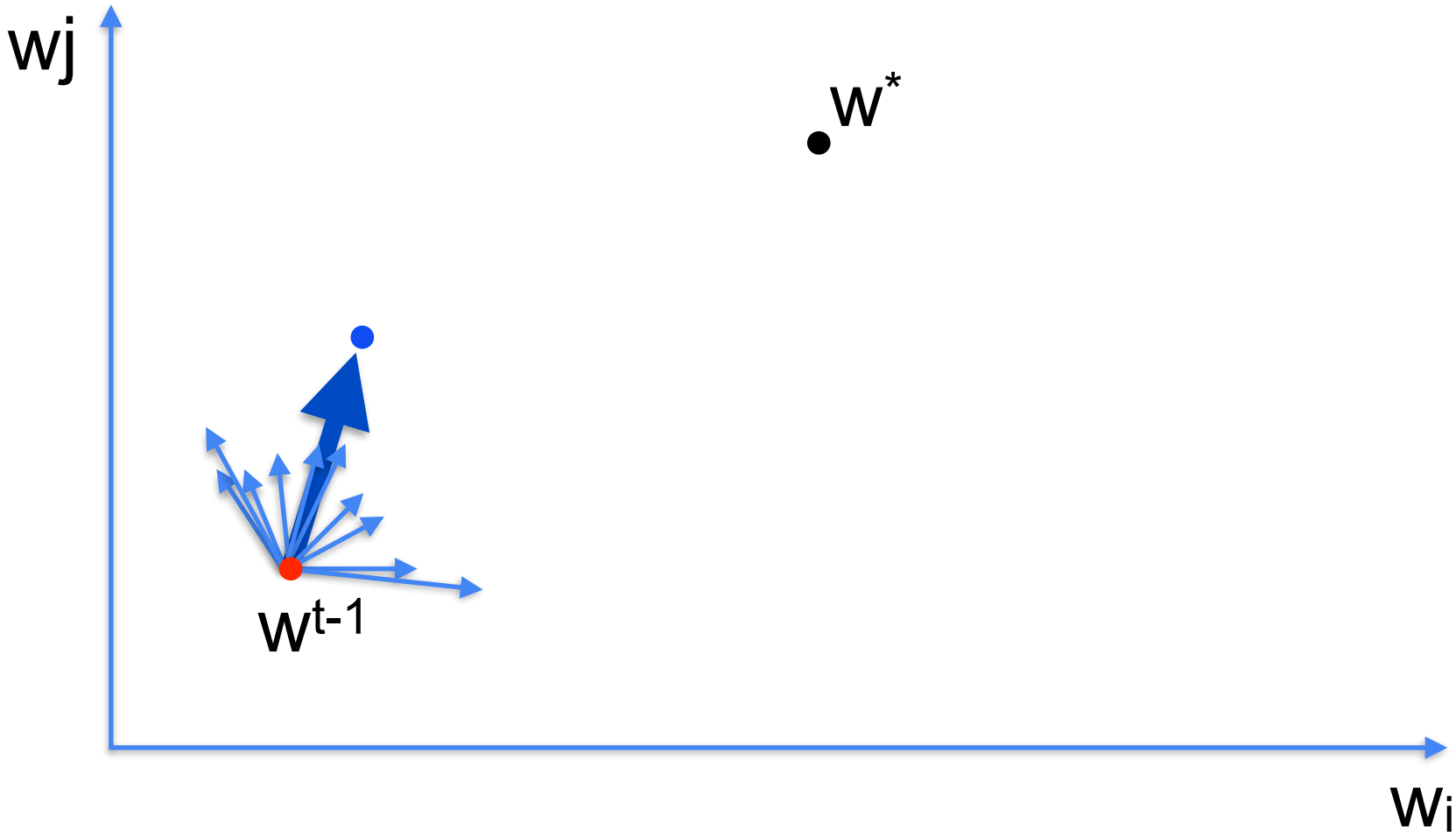
Gradient descent



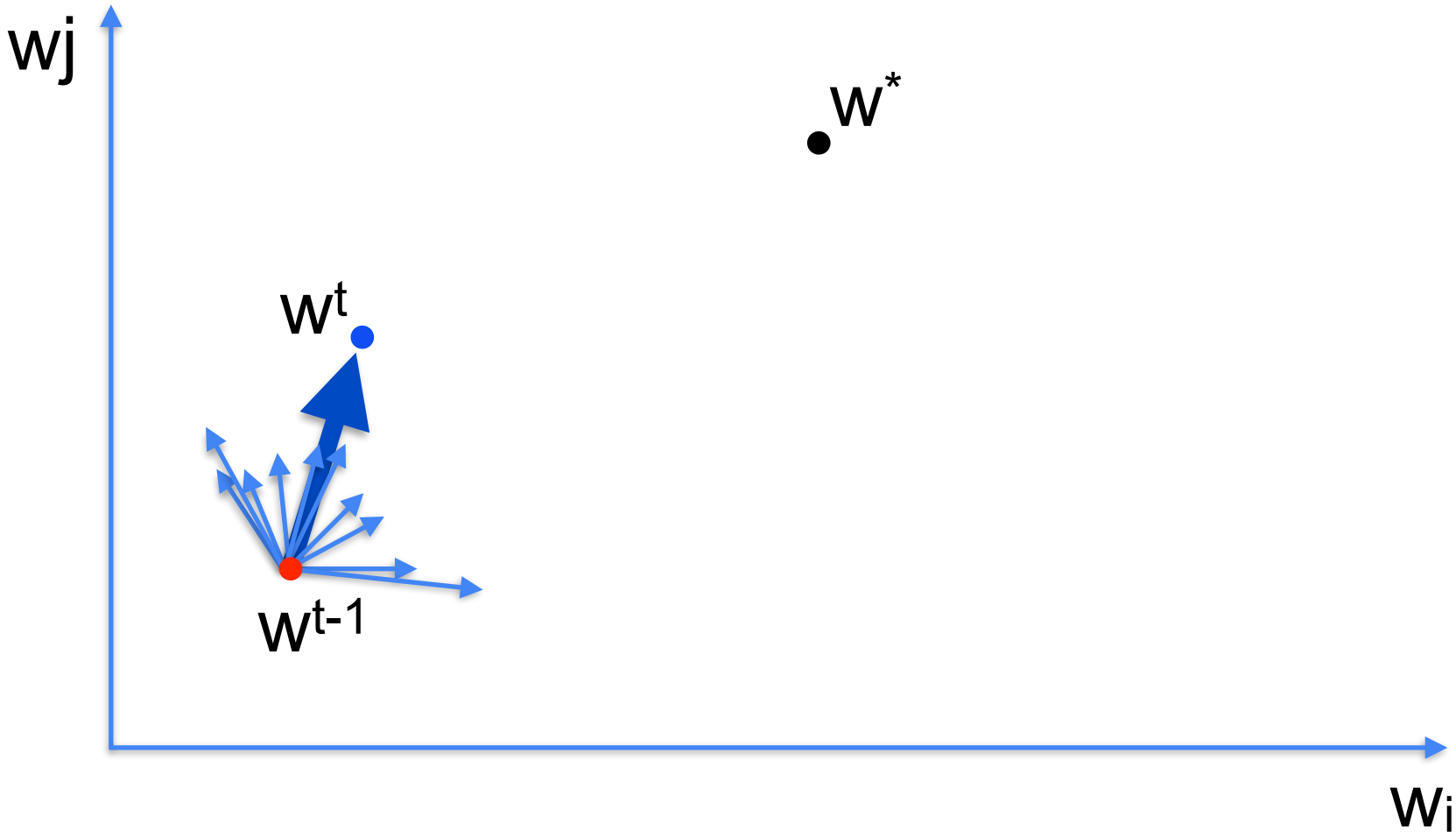
Gradient descent



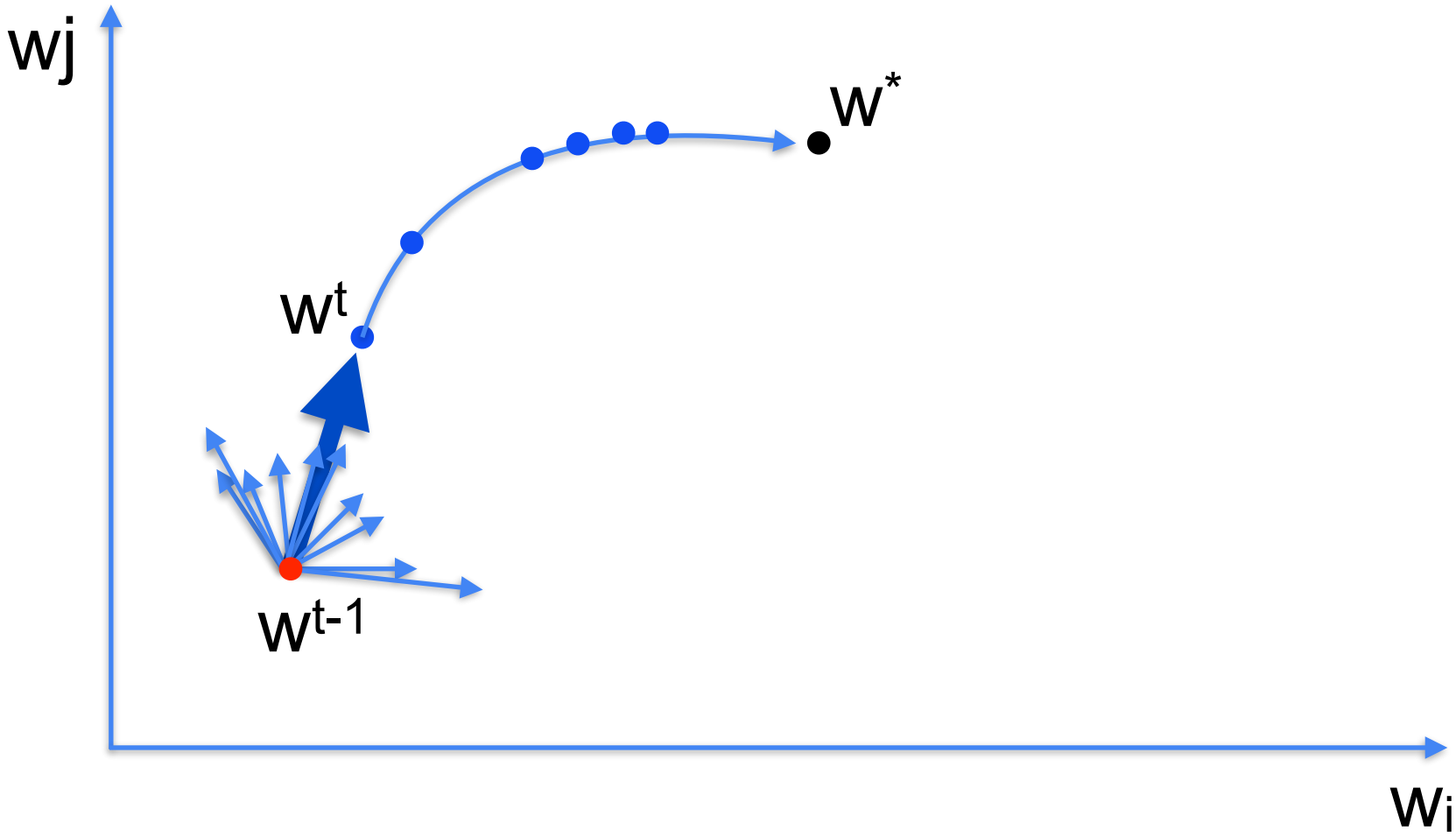
Gradient descent



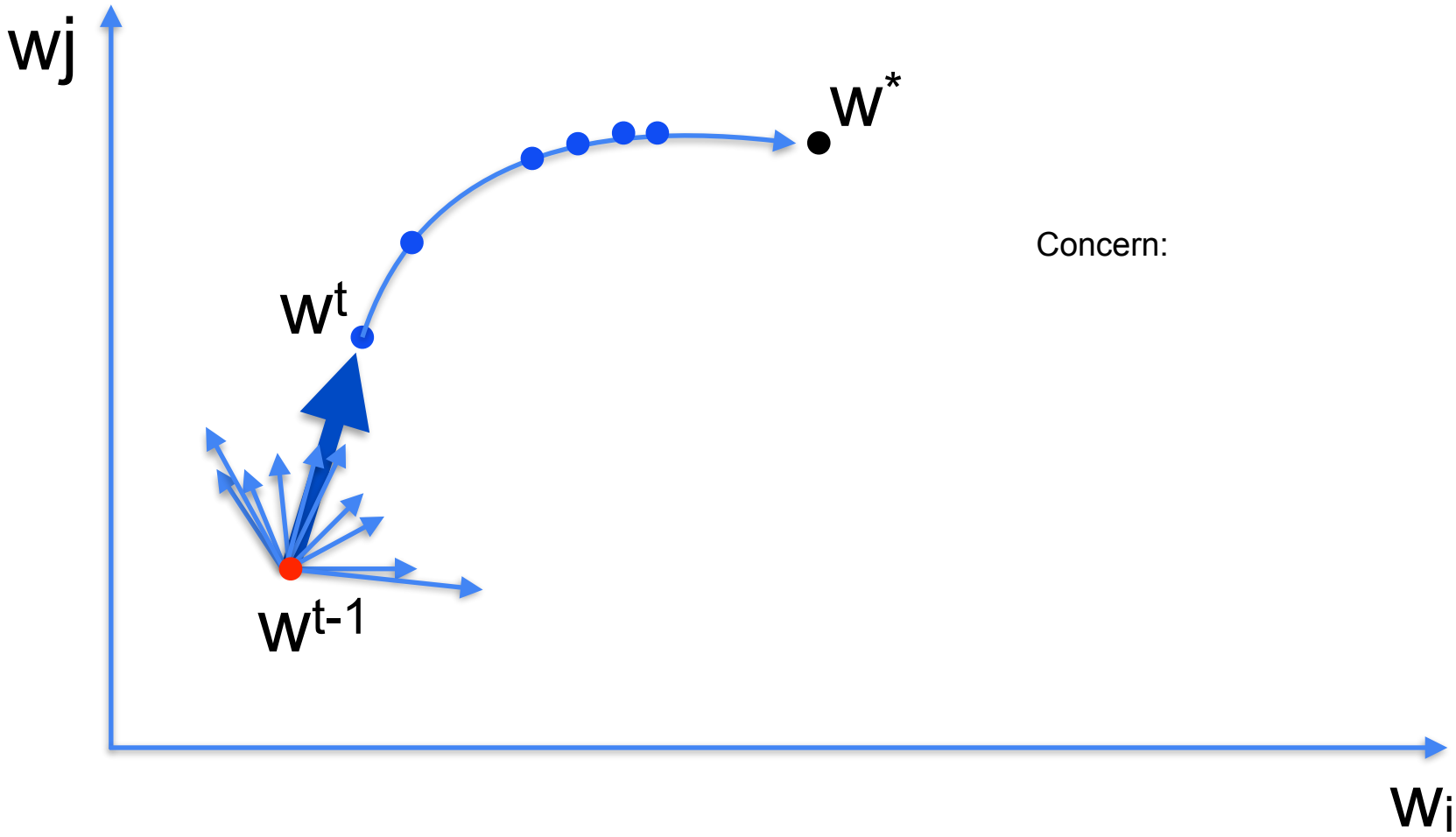
Gradient descent



Gradient descent

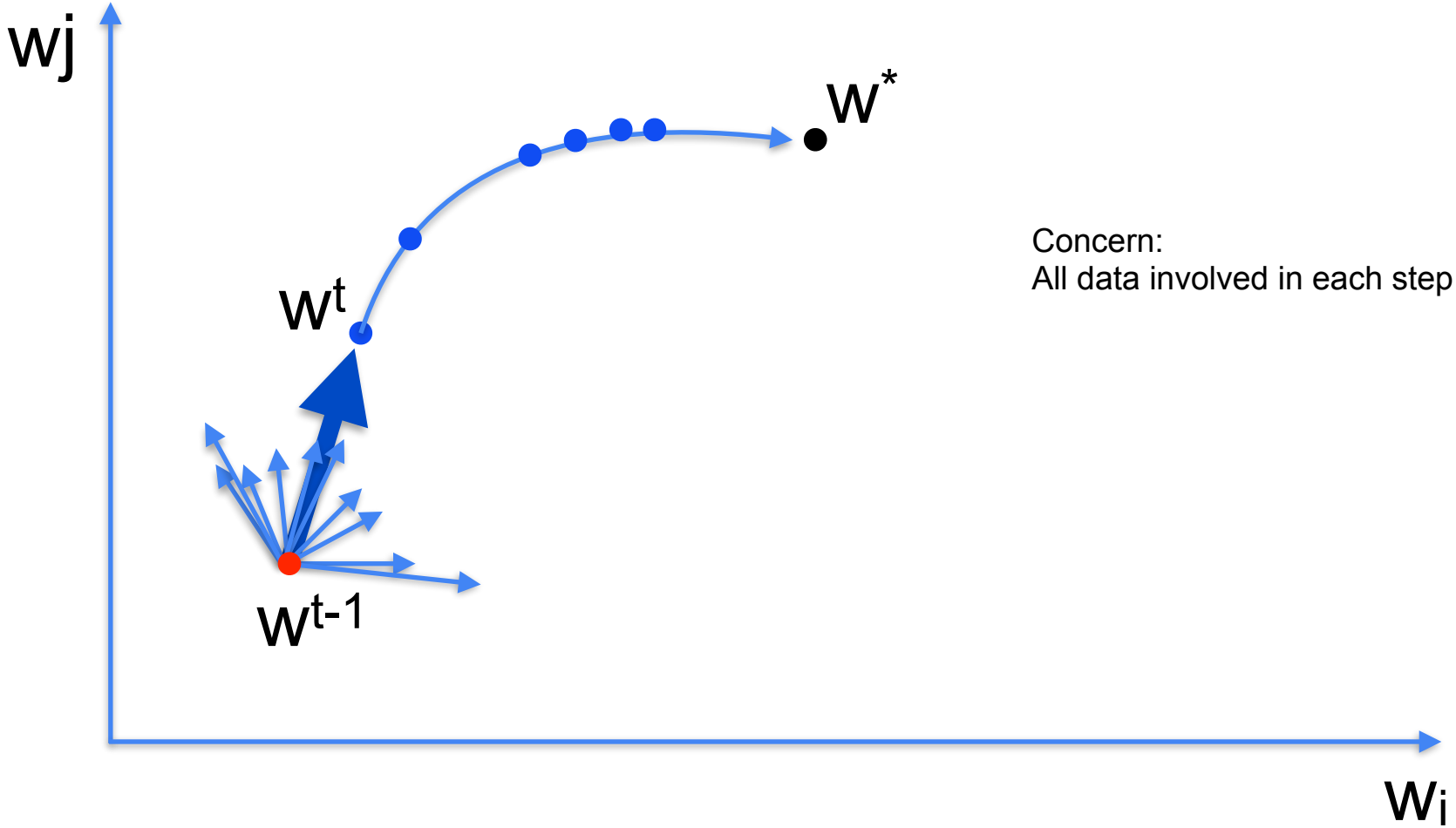


Gradient descent



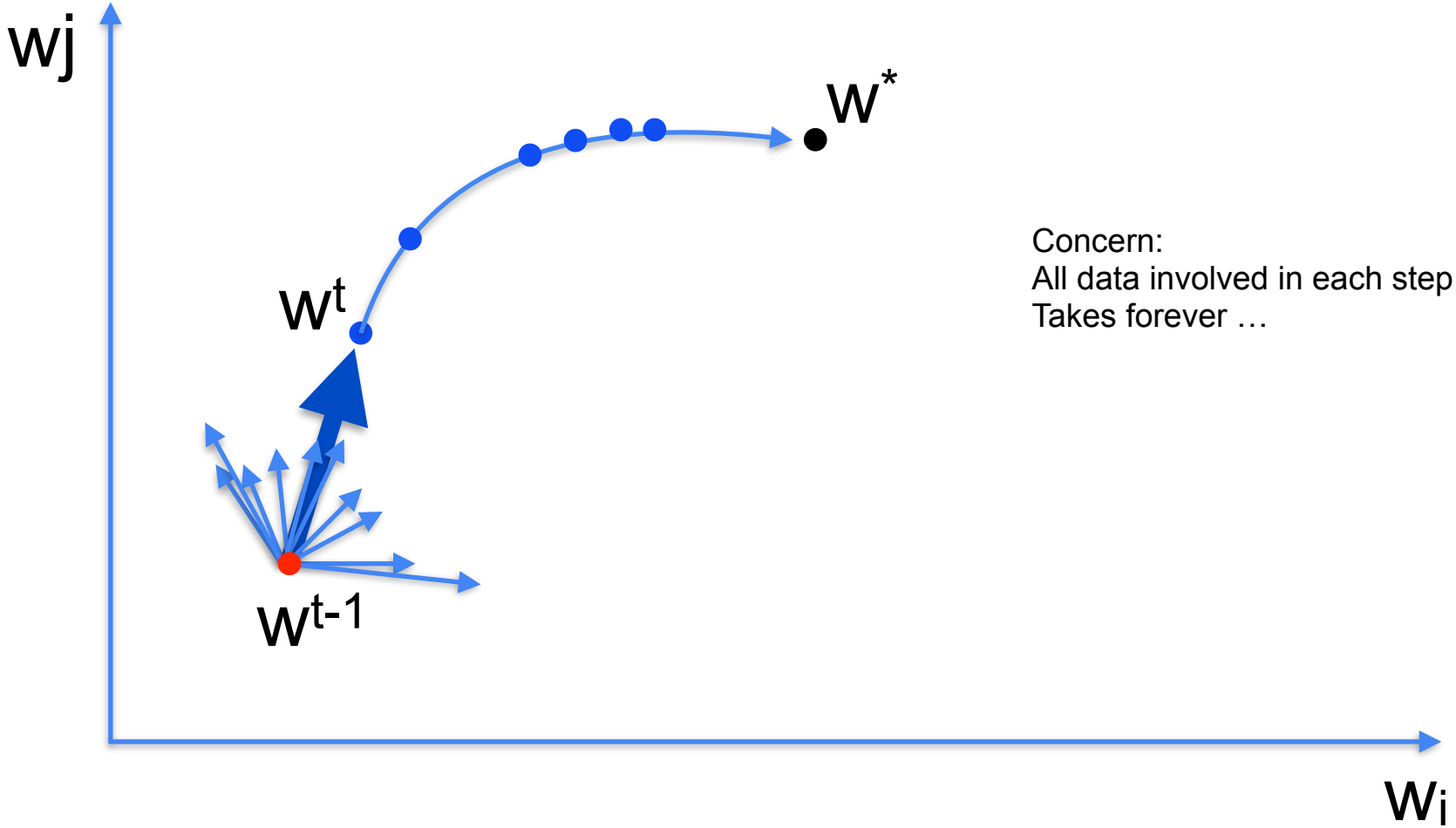
Concern:

Gradient descent



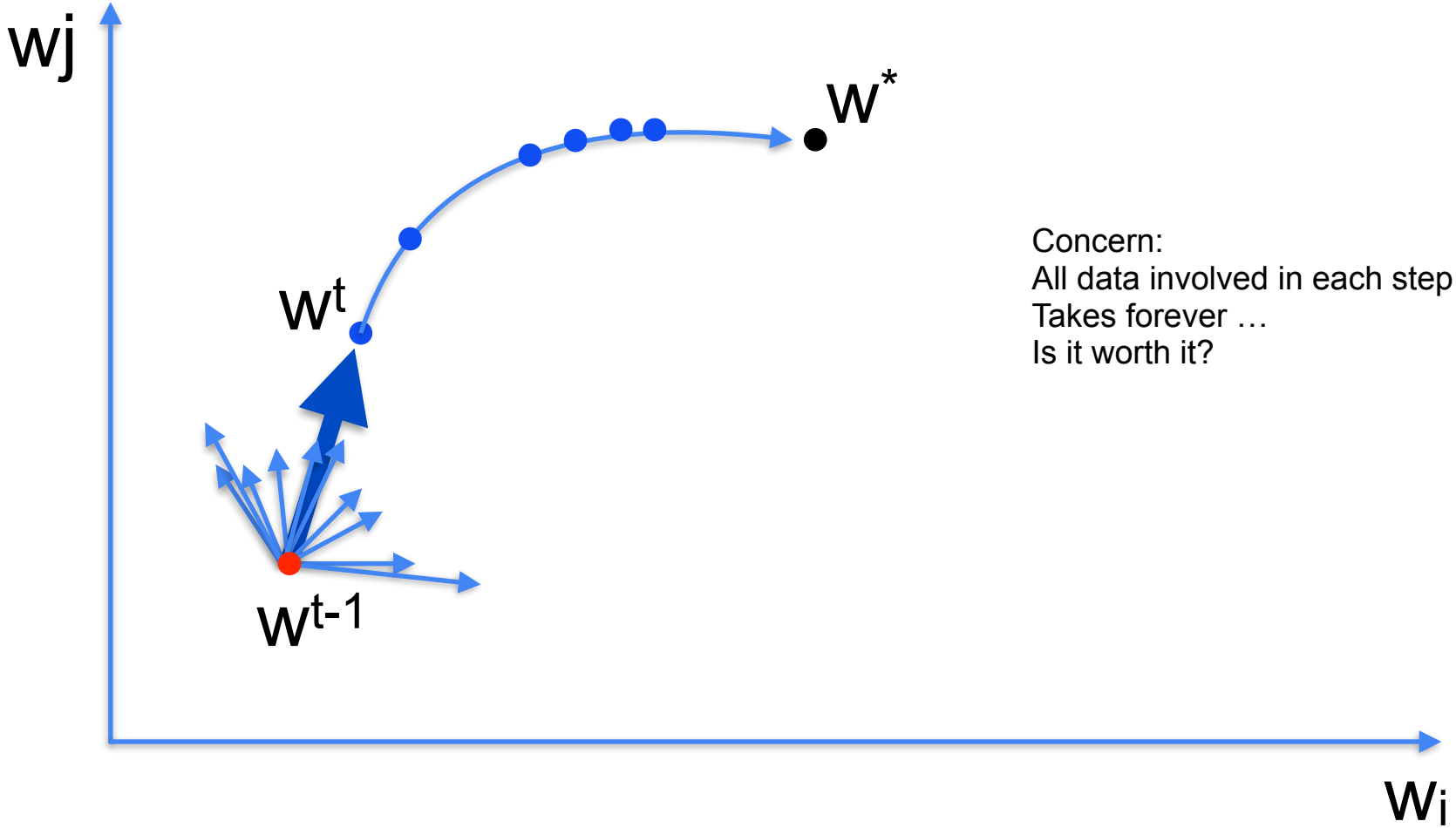
Concern:
All data involved in each step…

Gradient descent



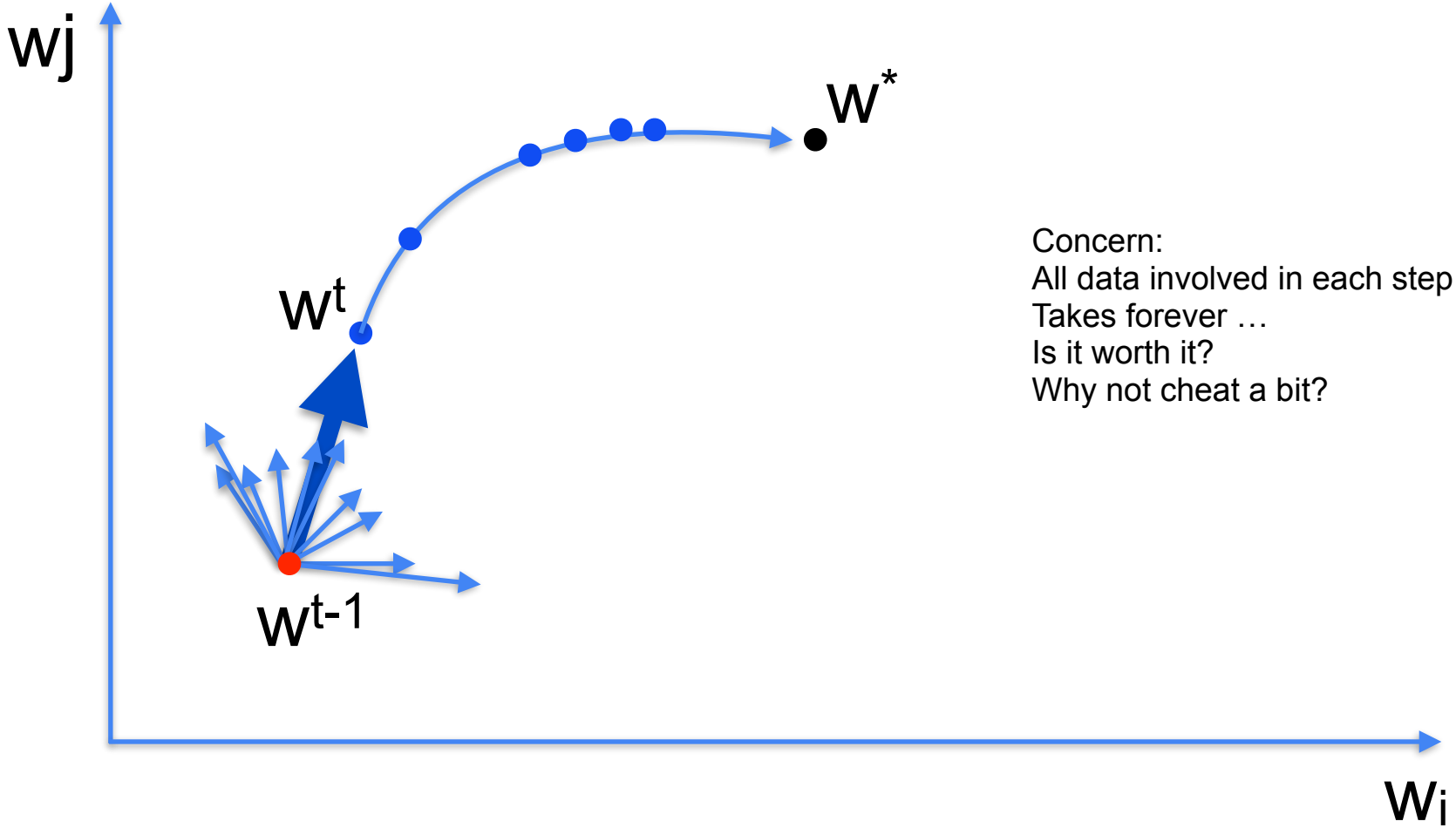
Concern:
All data involved in each step…
Takes forever …

Gradient descent



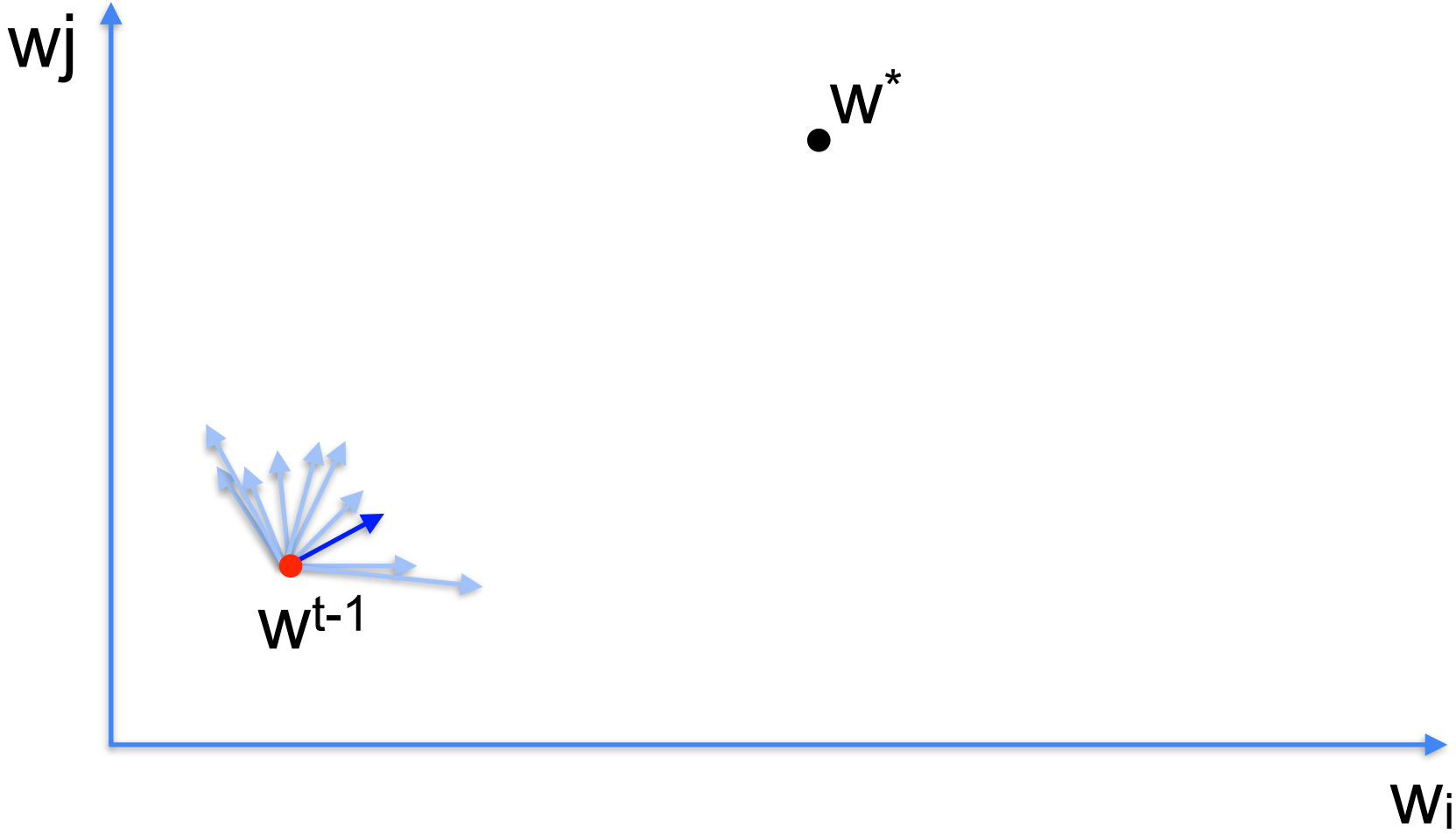
Concern:
All data involved in each step…
Takes forever …
Is it worth it?

Gradient descent

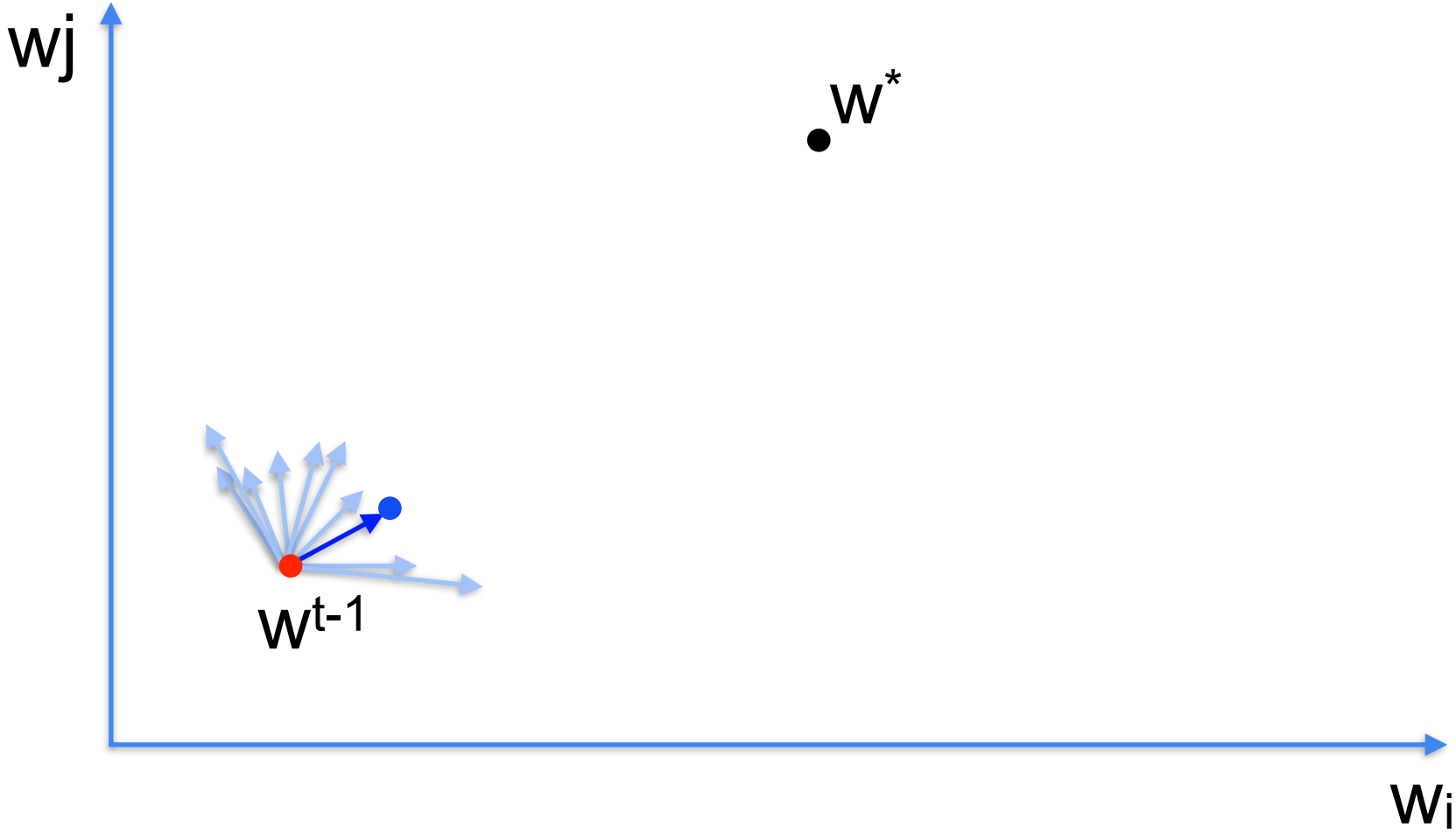


Concern:
All data involved in each step…
Takes forever …
Is it worth it?
Why not cheat a bit?

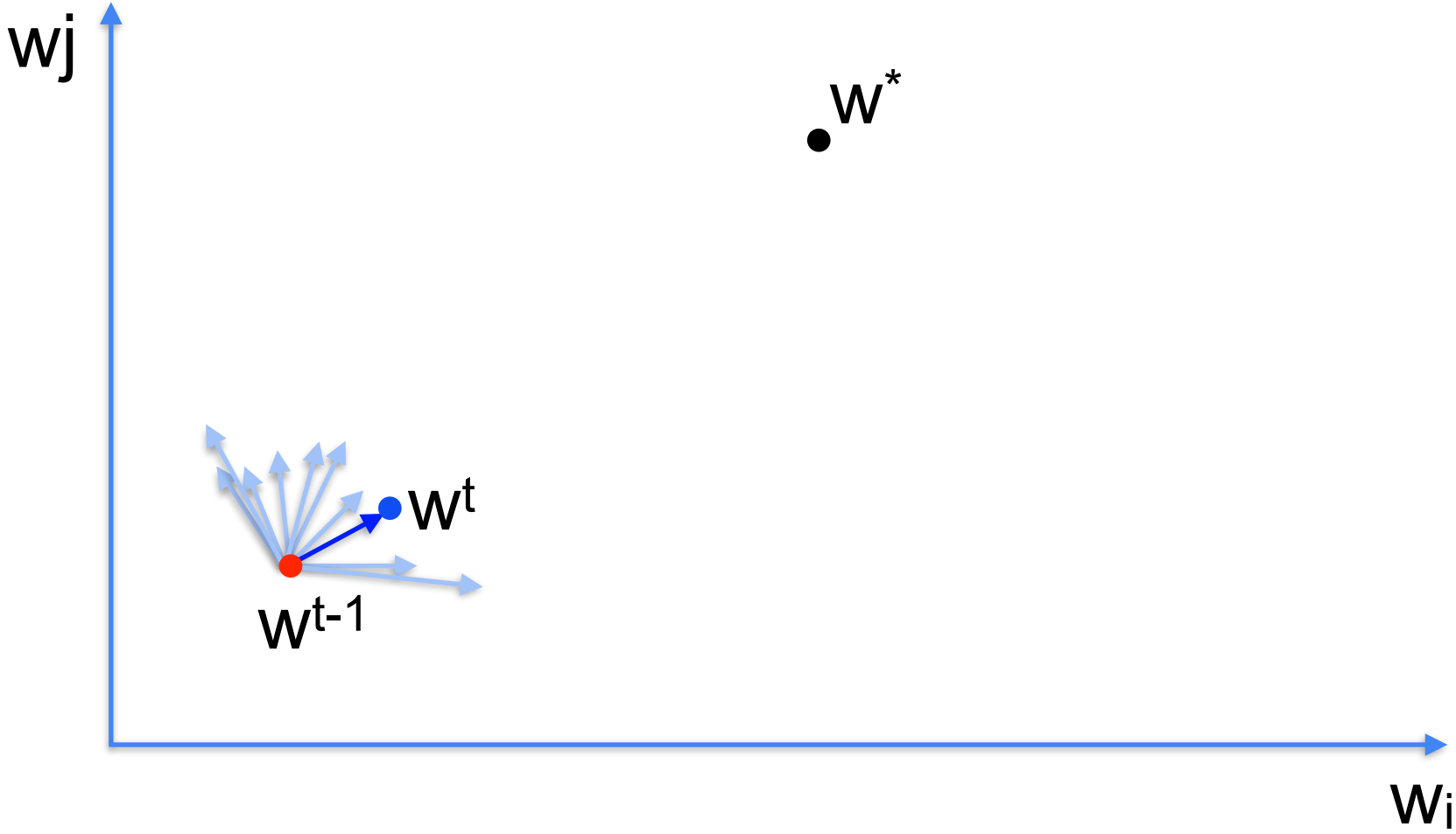
Stochastic gradient descent



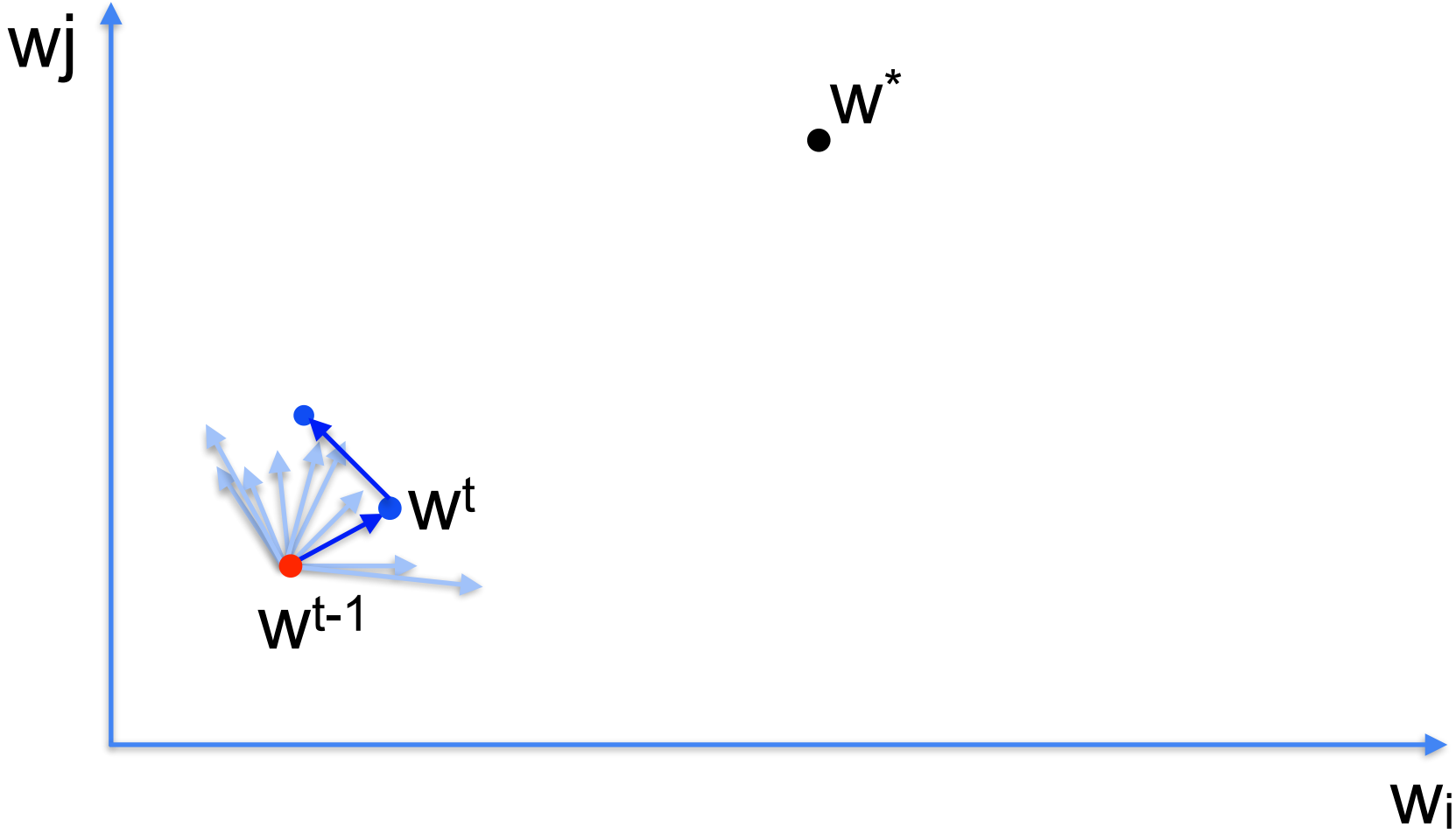
Stochastic gradient descent



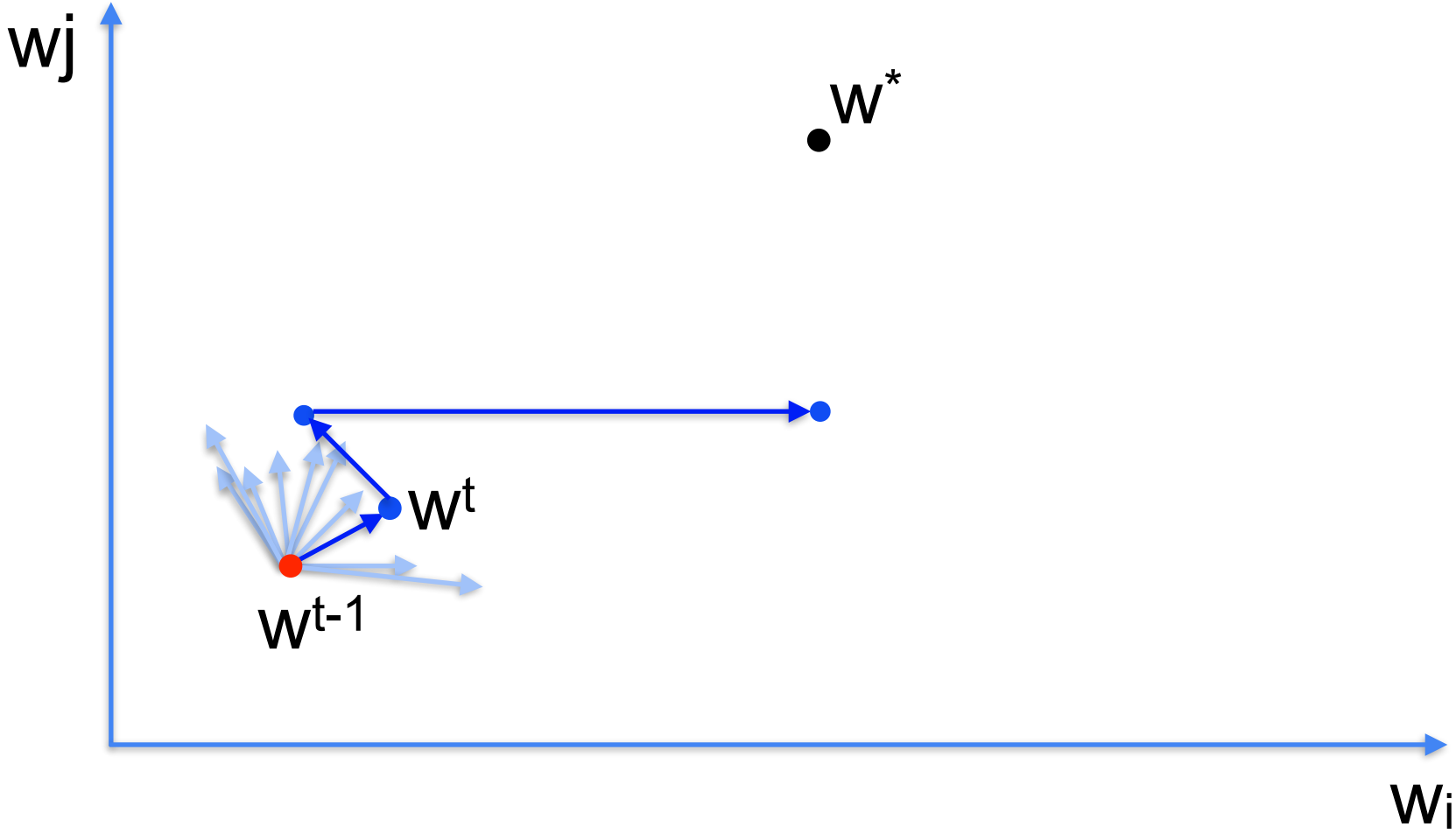
Stochastic gradient descent



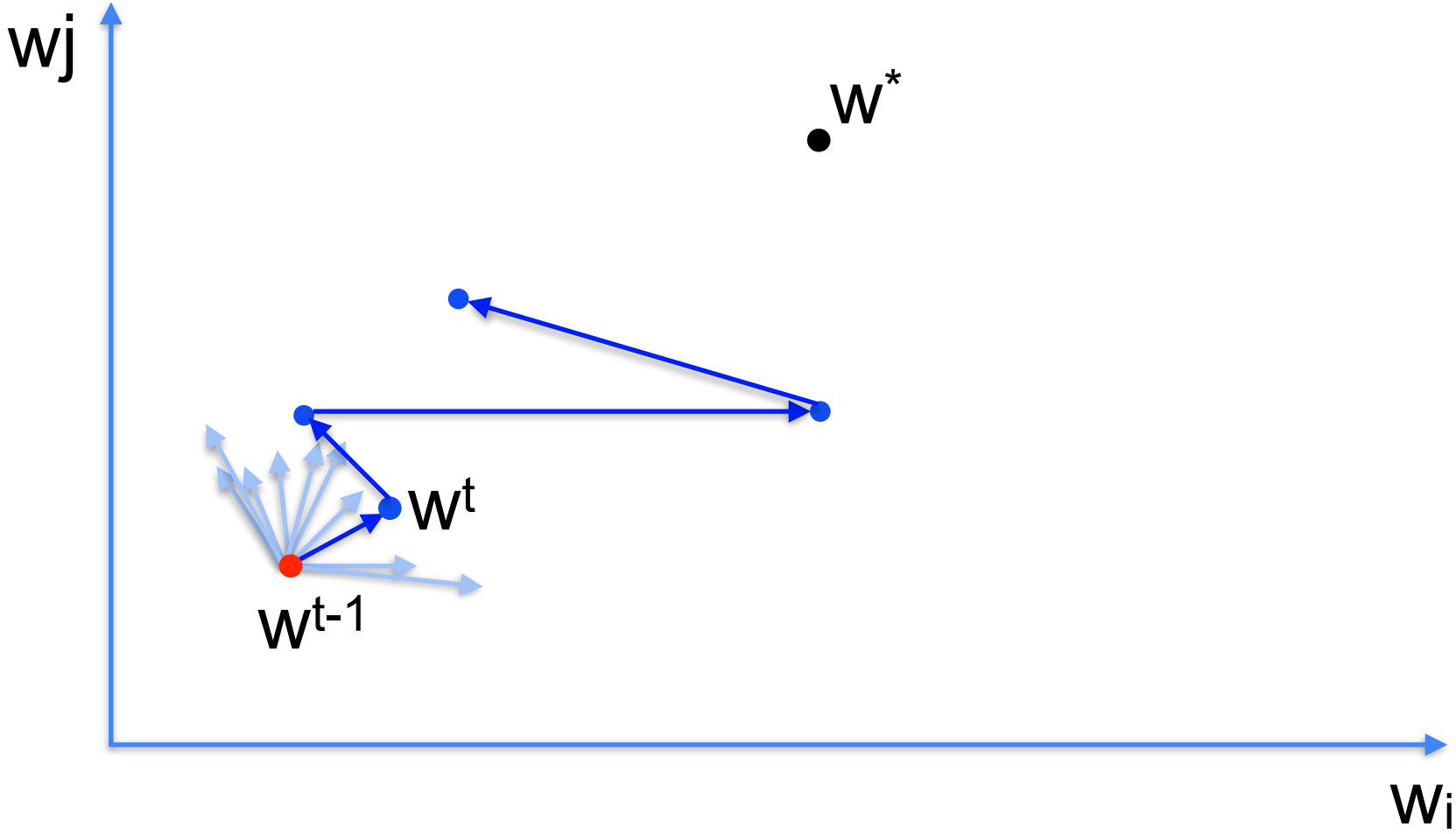
Stochastic gradient descent



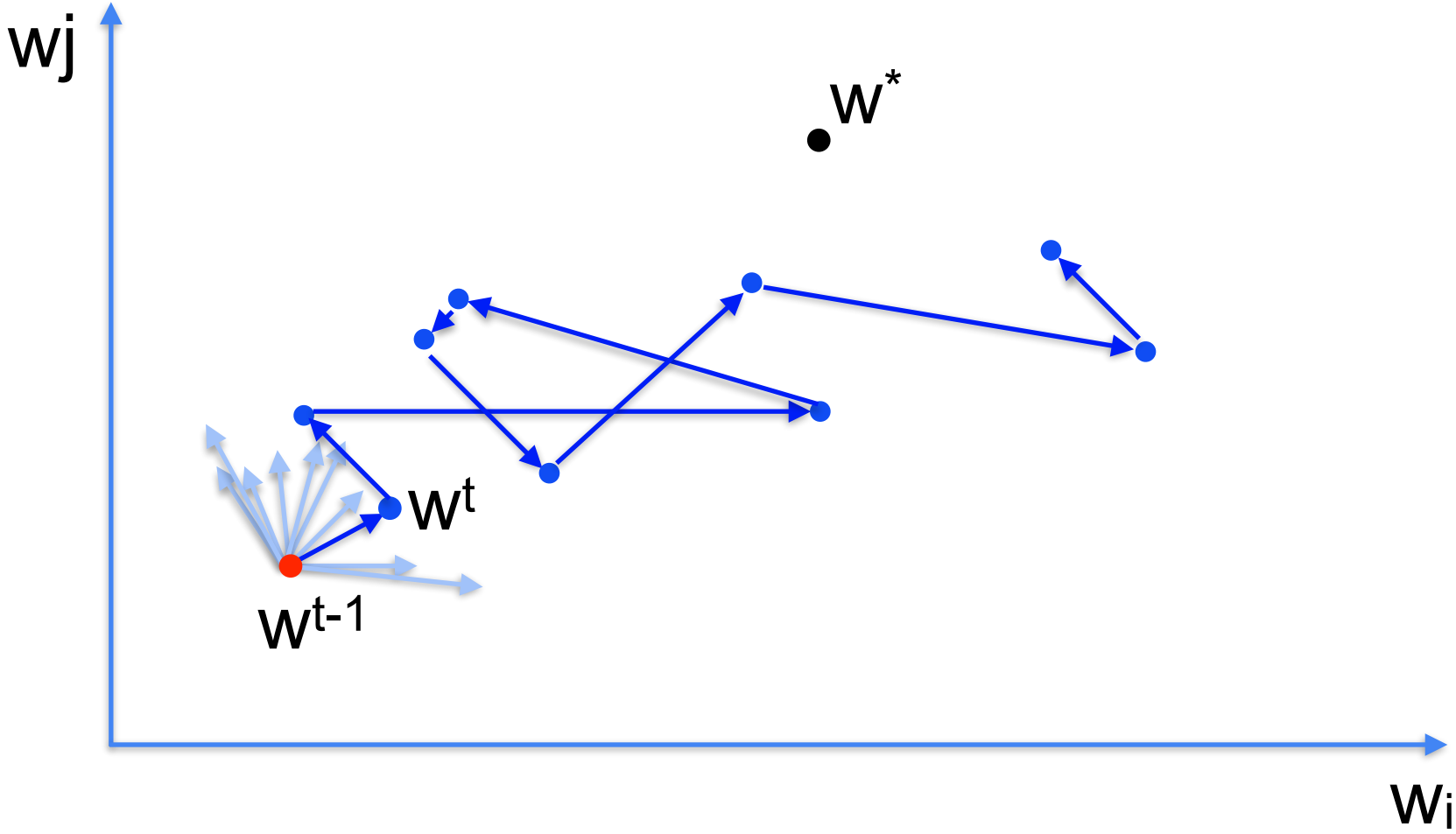
Stochastic gradient descent



Stochastic gradient descent



Stochastic gradient descent



Gradient descent

```
Input: dataset D = {(x(i), y(i))}, learning rate η, initial parameters w
repeat until convergence:
    compute gradient g = (1/N) * Σ_i ∇_w L(f(x(i), w), y(i))
    w ← w − η * g
return w
```

Gradient descent

```
Input: dataset D = {(x(i), y(i))}, learning rate η, initial parameters w
repeat until convergence:
    compute gradient g = (1/N) * Σ_i ∇_w L(f(x(i), w), y(i))
    w ← w − η * g
return w
```

Stochastic gradient descent (SGD)

```
Input: dataset D, learning rate η, initial parameters w
repeat until convergence:
    shuffle D
    for each (x(i), y(i)) in D:
        compute gradient g = ∇_w L(f(x(i), w) , y(i))
        w ← w − η * g
return w
```

Gradient descent

```
Input: dataset D = {(x(i), y(i))}, learning rate η, initial parameters w
repeat until convergence:
    compute gradient g = (1/N) * Σ_i ∇_w L(f(x(i), w), y(i))
    w ← w − η * g
return w
```

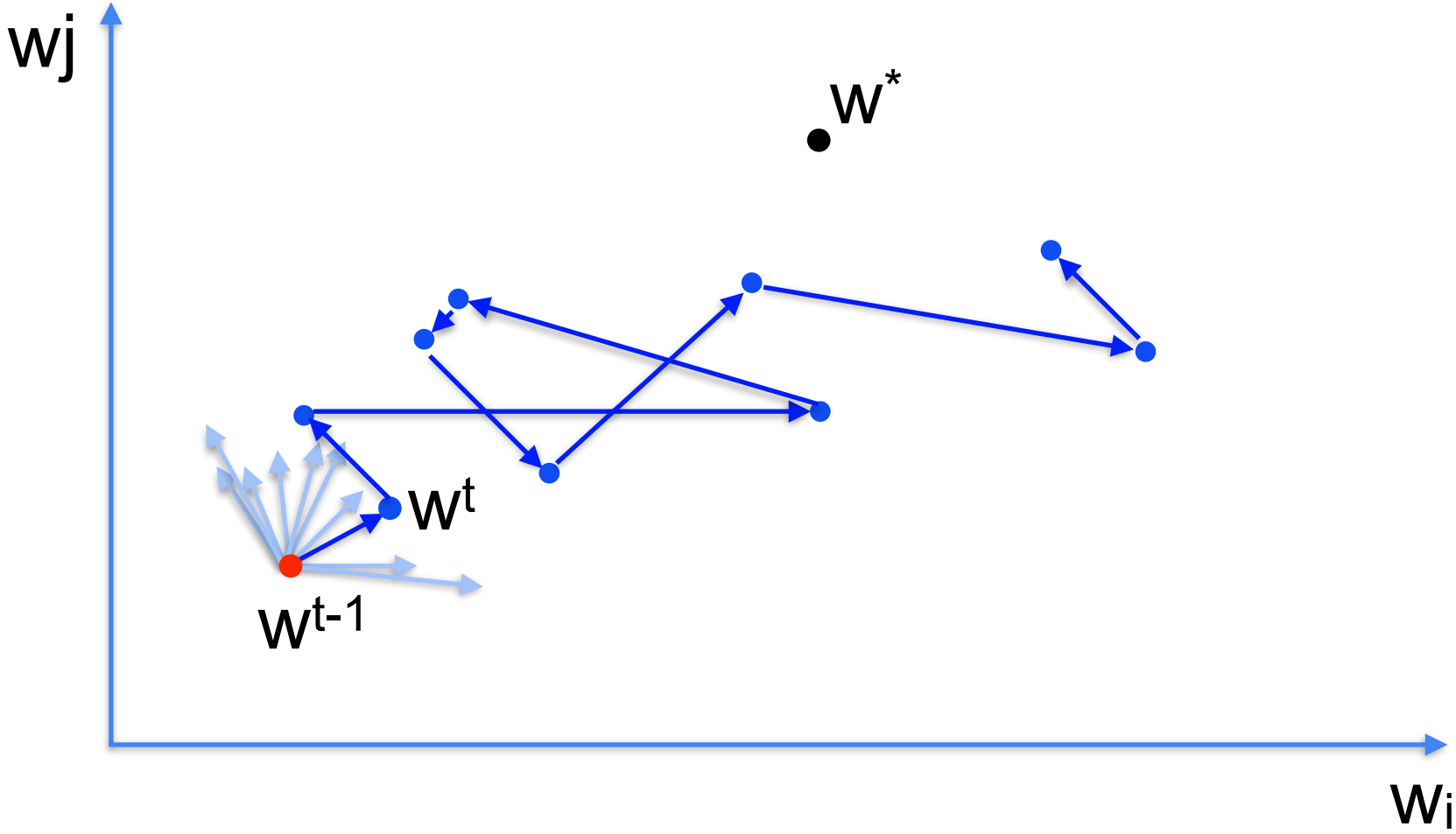
Stochastic gradient descent (SGD)

```
Input: dataset D, learning rate η, initial parameters w
repeat until convergence:
    shuffle D
    for each (x(i), y(i)) in D:
        compute gradient g = ∇_w L(f(x(i), w) , y(i))
        w ← w − η * g
return w
```

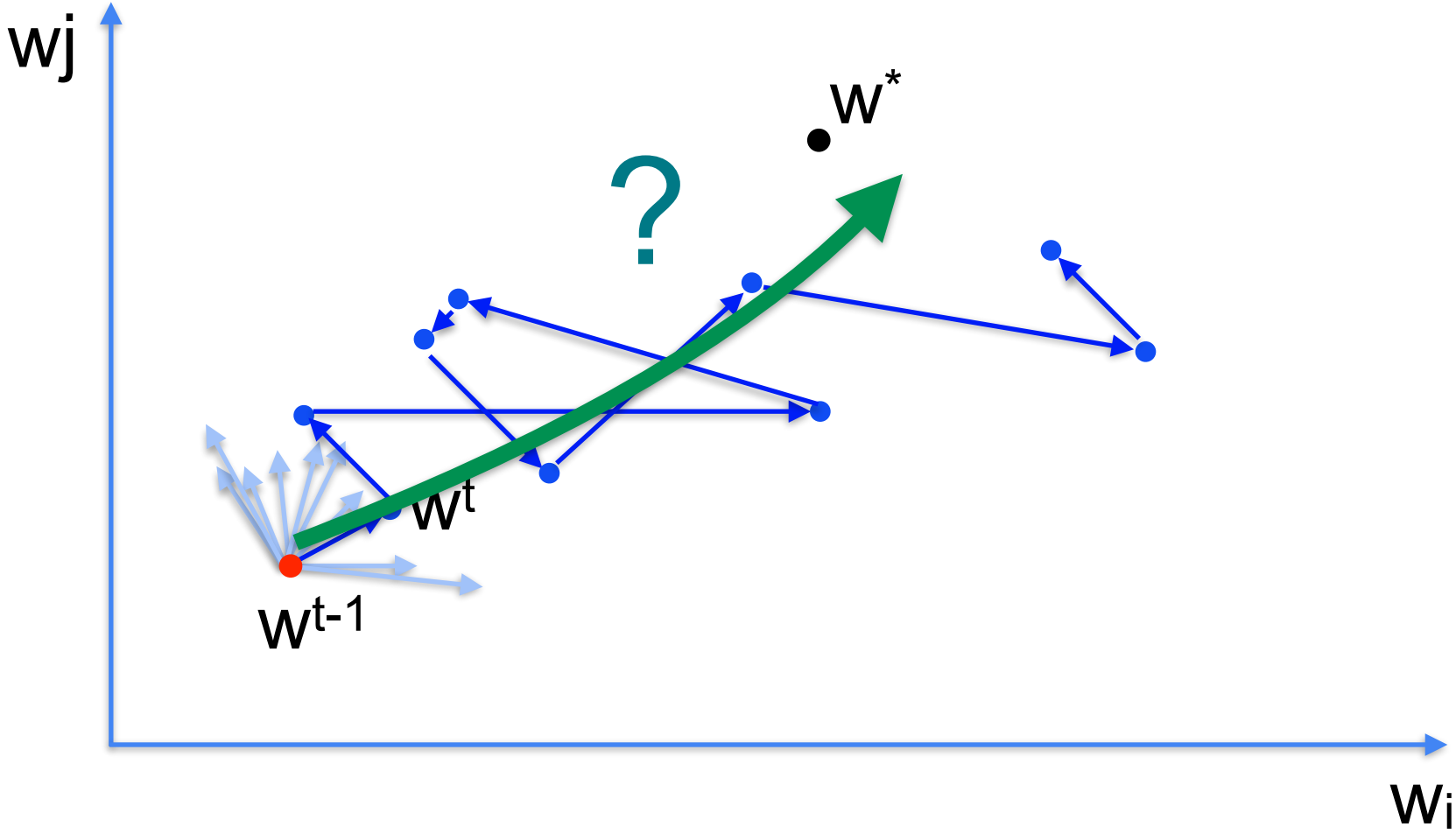
Mini-batch gradient descent

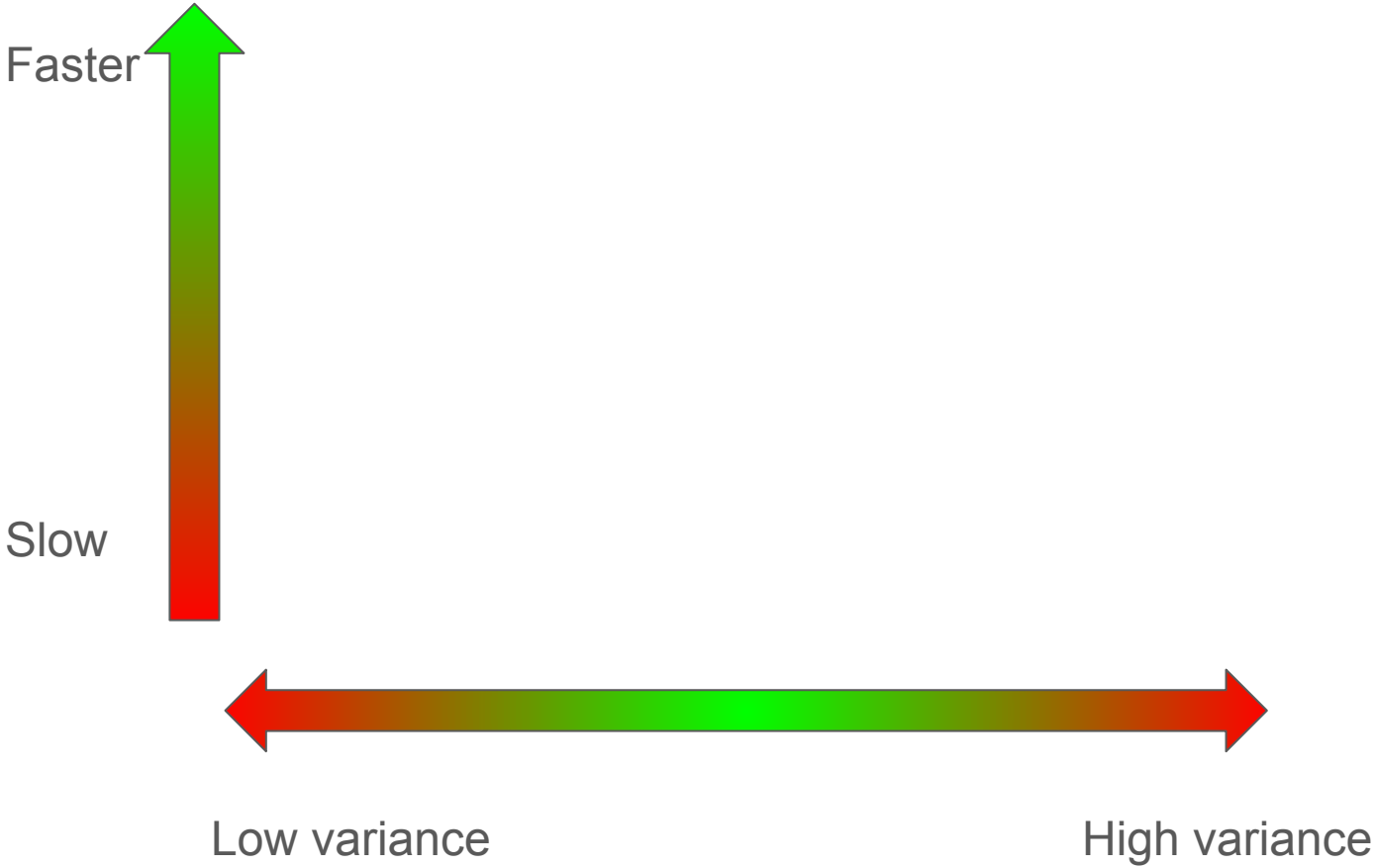
```
Input: dataset D, batch size B, learning rate η, initial parameters w
repeat until convergence:
    shuffle D
    partition D into batches of size B
    for each batch B_j:
        compute gradient g = (1/B) * Σ_(x,y)∈B_j ∇_w L(f(x, w), y)
        w ← w − η * g
return w
```

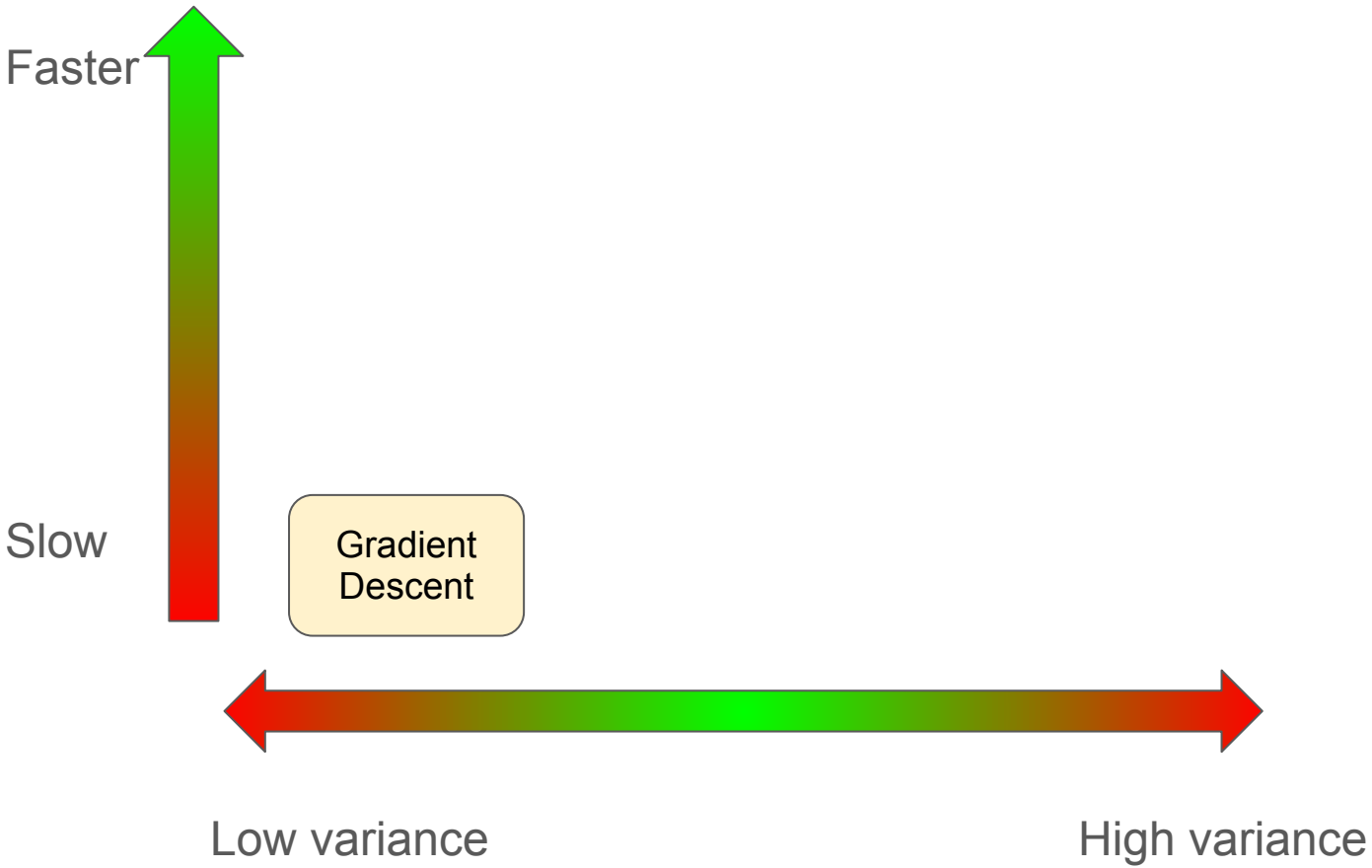
Idea for later: what about smoothing over time?

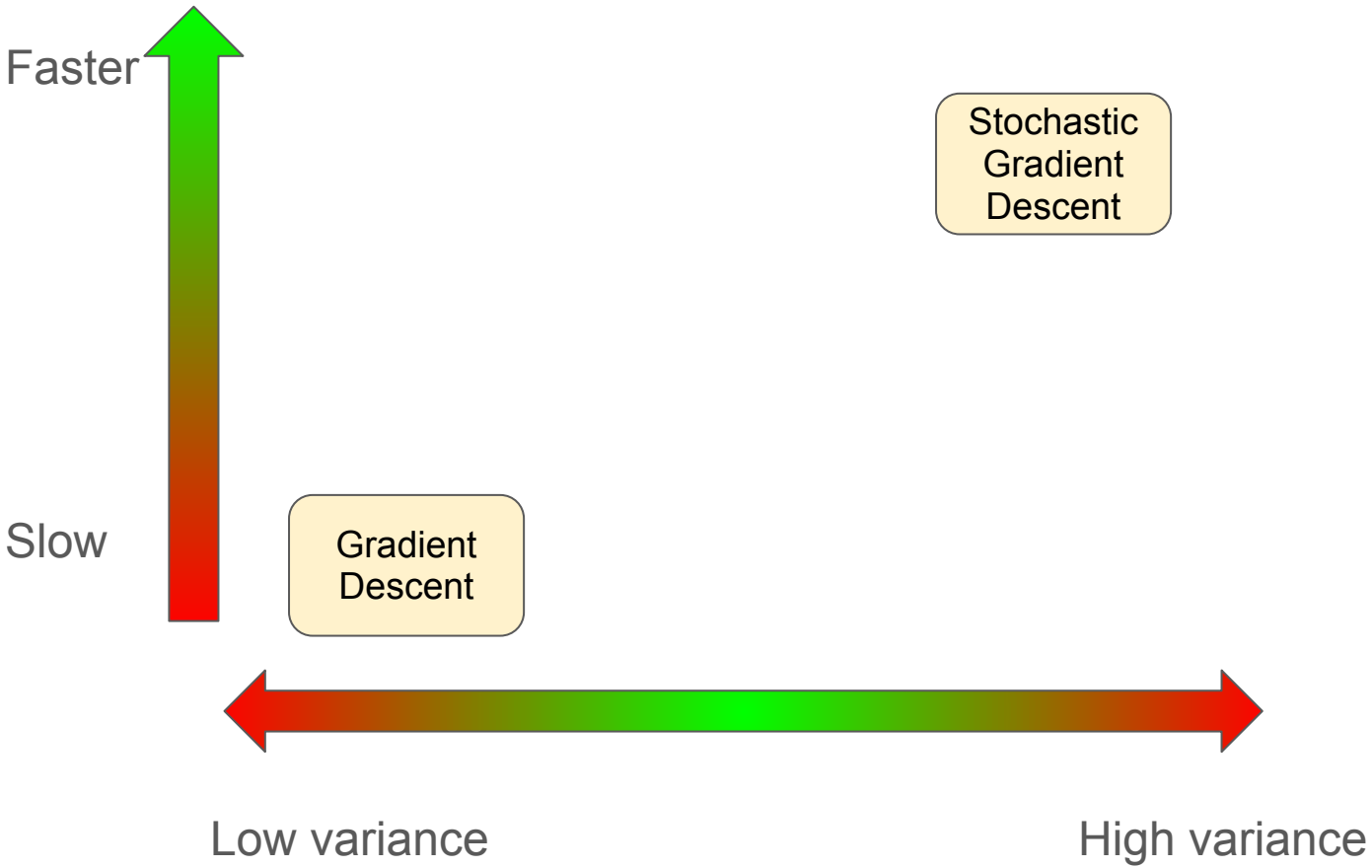


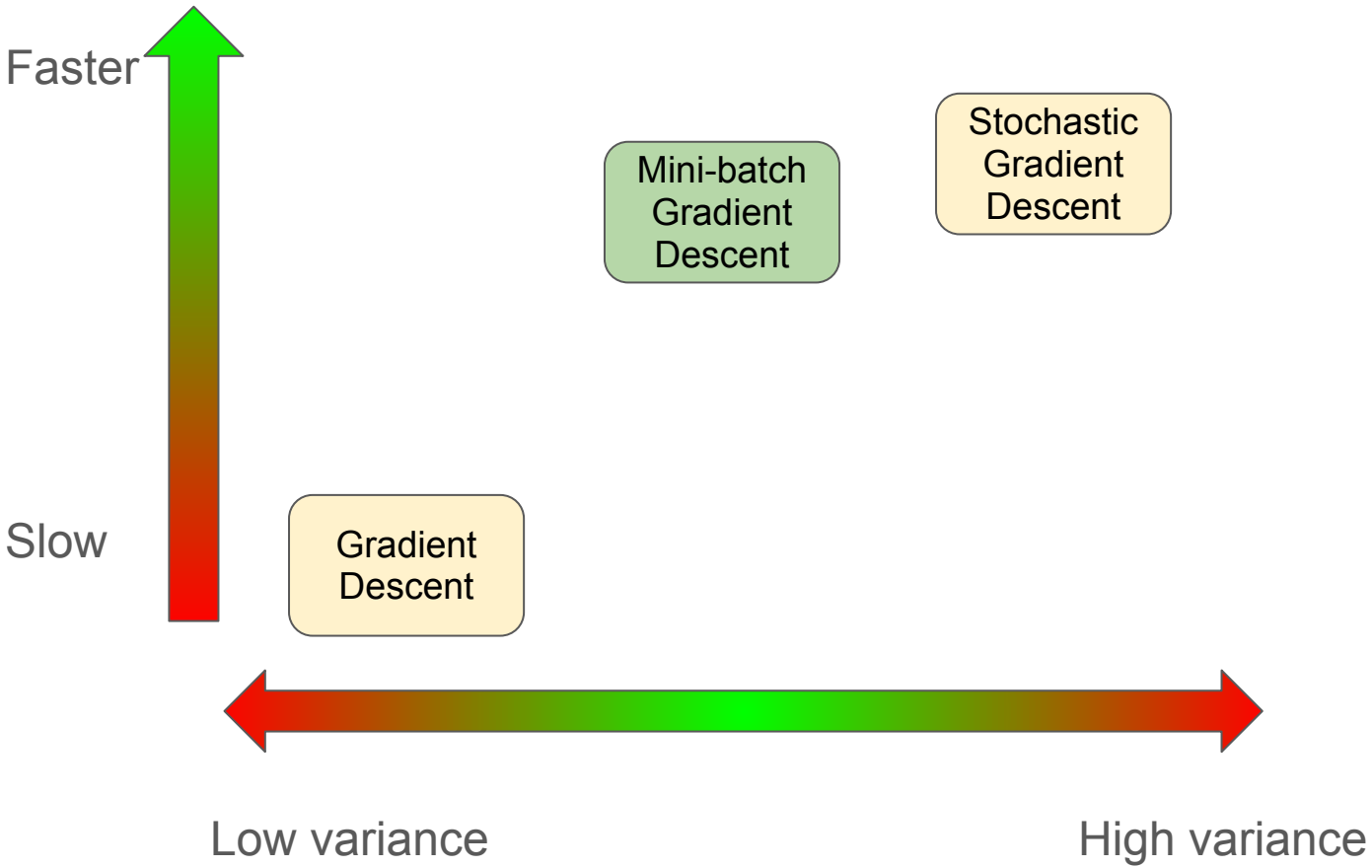
Idea for later: what about smoothing over time?




Faster
Slow
Low variance
High variance


Faster
Slow
Gradient Descent
Low variance
High variance


Faster
Slow
Stochastic Gradient Descent
Gradient Descent
Low variance
High variance


Faster
Slow
Mini-batch Gradient Descent
Stochastic Gradient Descent
Gradient Descent
Low variance
High variance